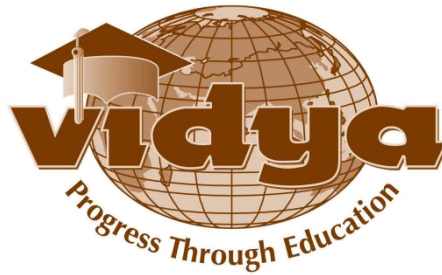


Vidya Academy of Science & Technology
(A unit of Vidya International Charitable Trust)
Approved by AICTE & Affiliated to APJ Abdul Kalam
Technological University



**DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING**

**CSL 331: SYSTEMS SOFTWARE AND
MICROPROCESSORS LAB**

LAB MANUAL

PREPARED BY: Ms. PRAJITHA M V

APPROVED BY:HOD, CSE

Vision of the Institution

Progress through education

Mission of the Institution

To seek, strive for and scale greater heights of quality education

Vision of the Department

Progress through quality education and become a centre of excellence in computing knowledge

Mission of the Department

To seek, to strive for advancements in technological and social aspects so as to create socially responsible engineers in the field of computer science, who will embark to further reduce gap between science and man-kind.

Program Educational Objectives (PEOs)

PEO-1: Graduates shall practice Computer Science and Engineering in IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.

PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.

PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

Program Specific Outcomes (PSOs)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

Program Outcomes (POs)

Computer Science & Engineering graduates will be able to:

PO-1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and Computer Science & Engineering specialization to the solution of complex engineering problems.

PO-2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO-3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO-4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO-5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO-6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO-7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO-8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO-9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary surroundings.

PO-10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO-11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO-12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

CSL 331	SYSTEM SOFTWARE AND MICROPROCESSOR S LAB	Category	L	T	P	Credit	Year of Introduction
		PCC	0	0	4	2	2019

Preamble: The aim of this course is to give hands-on experience in how microcontrollers, and microprocessors can be programmed. The course also aims to enable students to design and implement system software. The student should get familiar with assembly level programming of microprocessors and microcontrollers, interfacing of devices to microcontrollers, resource allocation algorithms in operating systems and design and implementation of system software.

Prerequisite: Sound knowledge in Operating systems

CO1	Develop 8086 programs and execute it using a microprocessor kit. (Cognitive Knowledge Level: Apply).
CO2	Develop 8086 programs and, debug and execute it using MASM assemblers (Cognitive Knowledge Level: Apply)
CO3	Develop and execute programs to interface stepper motor, 8255, 8279 and digital to analog converters with 8086 trainer kit (Cognitive Knowledge Level: Apply)
CO4	Implement and execute different scheduling and paging algorithms in OS (Cognitive Knowledge Level: Apply)
CO5	Design and implement assemblers, Loaders and macroprocessors. (Cognitive Knowledge Level: Apply)

Mapping of course outcomes with program outcomes

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	✓	✓	✓	✓				✓		✓		✓
CO2	✓	✓	✓	✓				✓		✓		✓
CO3	✓	✓	✓	✓				✓		✓		✓
CO4	✓	✓	✓	✓				✓		✓		✓
CO5	✓	✓	✓	✓				✓		✓		✓

Abstract POs defined by National Board of Accreditation

PO1	Engineering Knowledge	PO7	Environment and Sustainability
PO2	Problem Analysis	PO8	Ethics
PO3	Design/Development of solutions	PO9	Individual and team work
PO4	Conduct investigations of complex problems	PO10	Communication
PO5	Modern tool usage	PO11	Project Management and Finance
PO6	The Engineer and Society	PO12	Lifelong learning

Assessment Pattern

Bloom's Category	Continuous Assessment Test (Internal Exam) Percentage	End Semester Examination Percentage
Remember	20	20
Understand	20	20
Apply	60	60
Analyse		
Evaluate		
Create		

Mark Distribution

Total Marks	CIE Marks	ESE Marks	ESE Duration
150	75	75	3 hours

Continuous Internal Evaluation Pattern:

Attendance : 15 marks

Continuous Evaluation in Lab : 30 marks

Continuous Assessment Test : 15 marks

Viva-voce : 15 marks

Internal Examination Pattern: The marks will be distributed as Algorithm 30 marks, Program 20 marks, Output 20 marks and Viva 30 marks. Total 100 marks which will be converted out of 15 while calculating Internal Evaluation marks.

End Semester Examination Pattern: The marks will be distributed as Algorithm 30 marks, Program 20 marks, Output 20 marks and Viva 30 marks. Total 100 marks will be converted out of 75 for End Semester Examination.

Operating System to Use in Lab : Linux

Compiler/Software to Use in Lab : gcc

Programming Language to Use in Lab : Ansi C

Any compatible assembler can be used for implementation of 8086 programs

Fair Lab Record:

All Students attending the System Software and Microprocessors Lab should have a Fair Record. The fair record should be produced in the University Lab Examination. Every experiment conducted in the lab should be noted in the fair record. For every experiment in the fair record the right hand page should contain Experiment Heading, Experiment Number, Date of Experiment, Aim of Experiment, Details of Experiment including algorithm and Result of Experiment. The left hand page should contain a print out of the code used for the experiment and sample output obtained for a set of input.

SYLLABUS

MICROPROCESSOR LAB

- I. Assembly Language Programming Exercises/Experiments using 8086 Trainer kit
- II. Exercises/Experiments using MASM (PC required)
- III. Interfacing Exercises/Experiments with 8086 trainer kit through Assembly Language programming
- IV. Exercises/Experiments using 8051 trainer kit

SYSTEM SOFTWARE LAB:

- V. Experiments related to the operating system.
- VI. Exercises/Experiments related to the assemblers, loaders and macroprocessors

Text Books

1. Bhurchandi and Ray, Advanced Microprocessors and Peripherals, Third Edition McGraw Hill.
2. Andrew S Tanenbaum, "Modern Operating Systems", 4th Edition, Prentice Hall, 2015.
3. Leland L. Beck, System Software: An Introduction to Systems Programming, 3/E, Pearson Education Asia, 1997.

Reference Books

1. A. NagoorKani, Microprocessors and Microcontrollers, Second Edition, Tata McGraw Hill
2. Douglas V. Hall, SSSP Rao, Microprocessors and Interfacing, Third Edition, McGrawHill Education.
3. William Stallings, "Operating systems", 6th Edition, Pearson, Global Edition, 2015.

4. Garry Nutt, Nabendu Chaki, Sarmistha Neogy, "Operating Systems", 3rd Edition, Pearson Education.

5. D.M. Dhamdhare, Systems Programming and Operating Systems, Second Revised Edition, Tata McGraw Hill.

Practice Questions

MICROPROCESSORS LAB : List of Exercises/ Experiments

(Minimum 10 Exercises (at least 2 questions from each part I, II, III & IV)) : 2 Hrs/week

I. Assembly Language Programming Exercises/Experiments using 8086 Trainer kit

1. Implementation of simple decimal arithmetic and bit manipulation operations.
2. Implementation of code conversion between BCD, Binary, Hexadecimal and ASCII.
3. Implementation of searching and sorting of 16-bit numbers.

II. Exercises/Experiments using MASM (PC Required)

4. Study of Assembler and Debugging commands.
5. Implementation of decimal arithmetic (16 and 32 bit) operations.
6. Implementation of String manipulations.
7. Implementation of searching and sorting of 16-bit numbers.

III. Interfacing Exercises/Experiments with 8086 trainer kit through Assembly Language Programming

8. Interfacing with stepper motor - Rotate through any given sequence.
9. Interfacing with 8255 (mode0 and mode1 only).
10. Interfacing with 8279 (Rolling message, 2 key lockout and N-key rollover implementation).
11. Interfacing with Digital-to-Analog Converter.

IV. Exercises/Experiments using 8051 trainer kit

12. Familiarization of 8051 trainer kit by executing simple Assembly Language programs such as decimal arithmetic and bit manipulation.
13. Implementation of Timer programming (in mode1).

SYSTEM SOFTWARE LAB: List of Exercises/ Experiments

(Minimum 8 Exercises (at least 3 and 5 questions from each part V and VI)) : 2 Hrs/week

V. Exercises/Experiments from operating system

1. Simulate the following non-preemptive CPU scheduling algorithms to find turnaround time and waiting time.
a) FCFS b) SJF c) Round Robin (pre-emptive) d) Priority
2. Simulate the following file allocation strategies.
a) Sequential b) Indexed c) Linked
3. Implement the different paging techniques of memory management.
4. Simulate the following file organization techniques
a) Single level directory b) Two level directory c) Hierarchical
5. Implement the banker's algorithm for deadlock avoidance.
6. Simulate the following disk scheduling algorithms.
a) FCFS b) SCAN c) C-SCAN
7. Simulate the following page replacement algorithms:
a) FIFO b) LRU c) LFU

VI. Exercises/Experiments from assemblers, loaders and macroprocessor

1. Implement pass one of a two pass assembler.
2. Implement pass two of a two pass assembler.
3. Implement a single pass assembler.
4. Implement a two pass macro processor
5. Implement a single pass macro processor.
6. Implement an absolute loader.
7. Implement a relocating loader

INDEX

Exp No.	Expt. Name	Page Number
	Assembly Language Programming	
1	Implementation of simple decimal arithmetic operations	12
2	Implementation of code conversion between hexadecimal,BCD and decimal numbers	16
3	Implementation of searching and sorting of 16-bit numbers	28
4	Study of assembler and debugging commands in masm	34
5	Implementation of decimal arithmetic (16-bit and 32-bit) operations in masm	39
6	Implementation of string manipulation	44
7	Searching and sorting of 16-bit numbers using masm	50
8	Interfacing with stepper motor - rotate through any given sequence	56
9	Interfacing with 8255 (mode 0 and mode 1)	60
10	Interfacing with 8279 (rolling message, 2-key lockout, n-key rollover)	65
11	Interfacing with digital-to-analog converter (DAC)	72
12	Familiarization of 8051 trainer kit by executing simple assembly language programs(Decimal arithmetic and bit manipulation)	79
	System Software	
13	Simulation of FCFS,SJF,Round Robin & priority CPU scheduling algorithms	86
14	Simulation of sequential,indexed & linked file allocation strategies	104
15	Implementation of paging technique of memory management	118
16	Simulation of a single level directory,Two-level directory & Hierarchical (tree-structured) directory file organization technique	123
17	Implementation of the banker's algorithm for deadlock avoidance	141

18	Simulation of disk scheduling algorithms.-FCFS,SCAN & C-SCAN	147
19	Simulation of page replacement algorithms:-FIFO,LRU & LFU	161
	Exercises/Experiments from assemblers, loaders and macro processor	
20	Implement pass one of a two pass assembler	177
21	Implement pass two of a two pass assembler	184
22	Implement a single pass assembler	191
23	Implement a two pass macro processor	199
24	Implement a single pass macro processor	208
25	Implement an absolute loader	217
26	Implement a relocating loader	223

MICROPROCESSOR LAB – PART I

ASSEMBLY LANGUAGE PROGRAMMING

EXP NO: 1**IMPLEMENTATION OF SIMPLE DECIMAL ARITHMETIC OPERATIONS****AIM**

To perform simple decimal arithmetic operations – addition, subtraction, multiplication and division – on 8-bit and 16-bit hexadecimal numbers residing in memory using 8086 Assembly Language, and to store the result in the specified memory locations.

LEARNING OUTCOMES

Upon completion of this experiment, the student will be able to:

1. Write and hand-assemble simple 8086 Assembly Language programs.
2. Use data transfer instructions (MOV) to move data between registers and memory.
3. Use arithmetic instructions (ADD, SUB, MUL, DIV) to perform decimal arithmetic on 8-bit and 16-bit operands.
4. Handle the carry flag while performing addition and interpret the result stored in memory.
5. Store multi-byte results (quotient/remainder, higher/lower word) in successive memory locations.

DESCRIPTION

The 8086 microprocessor provides dedicated arithmetic instructions (ADD, SUB, MUL, DIV) that operate directly on register or memory operands. For addition of 8-bit numbers, the carry flag (CF) generated by the ADD instruction indicates whether the sum has exceeded the 8-bit range, and this is captured using a conditional jump (JNC) before storing the result. For 16-bit subtraction, the SUB instruction directly computes the difference of two word operands. Multiplication of two 16-bit numbers using MUL produces a 32-bit result, with the lower-order word in AX and the higher-order word in DX. Division of a 32-bit dividend (held in DX:AX) by a 16-bit divisor using DIV produces a 16-bit quotient in AX and a 16-bit remainder in DX.

ALGORITHM

1. Load the first operand from the specified memory location into a register (AL/AX).
2. Load the second operand from memory into another register (BL/BX) as required.
3. Perform the required arithmetic operation using the appropriate instruction (ADD, SUB, MUL or DIV).
4. For addition, check the carry flag using JNC; if a carry is generated, increment the carry byte.
5. Store the result (and carry/remainder, if any) in the specified memory location(s).

6. Terminate the program using the HLT instruction.

PROGRAM

(a) Addition of Two 8-bit Numbers (with Carry)

Address	Object Code	Mnemonics	Remarks
1000	8A 26 00 11	MOV AH,[1100]	First input byte in AH register
1004	8A 1E 01 11	MOV BL,[1101]	Second input byte in BL register
1008	C6 C0 00	MOV AL,00	
100B	00 DC	ADD AH,BL	
100D	73 02	JNC 1011	Jump if no carry
100F	FE C0	INC AL	
1011	88 06 02 11	MOV [1102],AL	Carry in location 1102
1015	88 26 03 11	MOV [1103],AH	Result in location 1103
1019	F4	HLT	

OUTPUT

Input: [1100] = FF [1101] = FF

Output: [1102] = 01 [1103] = FE

(b) Subtraction of Two 16-bit Numbers

Address	Object Code	Mnemonics	Remarks
1000	8A 06 00 11	MOV AX,[1100]	First input word in AX register
1004	2B 06 02 11	SUB AX,[1102]	Second input word in location 1102
1008	89 06 00 12	MOV [1200],AX	Result in location 1200
100C	F4	HLT	

OUTPUT

Input: [1100] = 9999 [1102] = 369C

Output: [1200] = 62FD

(c) Multiplication of Two 16-bit Numbers

Address	Object Code	Mnemonics	Remarks
1000	8B 06 00 11	MOV AX,[1100]	Move the first number to AX
1004	8B 06 02 11	MOV BX,[1102]	Move second number to BX
1008	F7 E3	MUL BX	Multiply the contents of BX and AX
100A	89 06 00 12	MOV [1200],AX	Lower word of result in 1200
100E	89 16 02 12	MOV [1202],DX	Higher order word of result in 1202
1012	F4	HLT	End of the program

OUTPUT

Input: [1100] = EF1A [1102] = CD50

Output: [1200] = 8A20 [1202] = BFC2

(d) Division of a 32-bit Number by a 16-bit Number

Address	Object Code	Mnemonics	Remarks
1000	8B 06 00 11	MOV AX,[1100]	Fetch lower order word of dividend
1004	2B 06 02 11	MOV DX,[1102]	Fetch higher order word of dividend
1008	89 06 02 12	MOV BX,[1104]	Pointer for divisor
100C	8B 06 04 11	DIV BX	
1010	1B 06 00 11	MOV [1200],AX	Store quotient in location 1200
1014	89 06 00 12	MOV [1202],DX	Store remainder in location 1202
1018	F4	HLT	End of the program

OUTPUT

Input (Dividend): [1100] = 580A [1102] = 71C2

Input (Divisor): [1104] = F6F2

Output: Quotient [1200] = 75EE Remainder [1202] = 290E

MODEL VIVA QUESTIONS

1. What is the purpose of the carry flag in 8-bit addition?
2. Which registers hold the result of a 16-bit MUL operation, and how?
3. Which registers must be loaded before executing a DIV instruction on a 32-bit dividend?
4. What is the difference between the ADD and ADC instructions?
5. Why is the HLT instruction used at the end of every program?
6. What happens if the divisor is zero during a DIV operation?

RESULT

The 8086 Assembly Language programs for addition, subtraction, multiplication and division of hexadecimal numbers were executed, and the results were verified against the expected outputs shown above.

EXP NO: 2**IMPLEMENTATION OF CODE CONVERSION BETWEEN HEXADECIMAL, BCD AND DECIMAL NUMBERS****AIM**

To convert a hexadecimal number to its equivalent BCD number, a BCD number to its equivalent hexadecimal number, and a decimal number to its equivalent hexadecimal number, using 8086 Assembly Language, and to store the result in the specified memory location(s).

LEARNING OUTCOMES

Upon completion of this experiment, the student will be able to:

1. Use logical instructions (AND) to mask and isolate individual nibbles of a byte.
2. Use the DAA and DAS instructions to perform decimal adjustment after addition and subtraction.
3. Implement repeated addition/subtraction as a substitute for multiplication and division during number-base conversion.
4. Convert numbers between hexadecimal, BCD and decimal representations using 8086 Assembly Language.
5. Trace and verify the intermediate register values used in a code-conversion routine.

DESCRIPTION

Code conversion routines translate a number from one representation to another using masking, shifting and decimal-adjust instructions. To convert a hexadecimal byte to BCD, the upper and lower nibbles are separated using AND masks and a right shift, and each nibble is accumulated in decimal form using repeated addition together with the DAA (Decimal Adjust after Addition) instruction. The reverse conversion, from BCD to hexadecimal, uses repeated subtraction of 16 together with the DAS (Decimal Adjust after Subtraction) instruction to determine how many multiples of 16 are contained in the BCD value, and a final multiplication reconstructs the hexadecimal equivalent. Decimal-to-hexadecimal conversion follows the same repeated-subtraction principle, applied to a 16-bit word instead of a single byte.

ALGORITHM

1. Load the input value from the specified memory location into a register.
2. Mask/shift the register as required to isolate the nibble or byte to be processed.
3. Repeatedly add or subtract 16 (using DAA/DAS for decimal adjustment) to determine the equivalent value, counting the number of repetitions in a register.
4. Combine the intermediate values (using multiplication and addition, where required) to obtain the final converted result.
5. Store the converted result in the specified memory location(s).
6. Terminate the program using the HLT instruction.

PROGRAM

(a) Conversion of a Hexadecimal Number to BCD

Address	Object Code	Mnemonics	Remarks
1000	8A 26 00 11	MOV AH,[1100]	Input byte in AH register
1004	80 E4 F0	AND AH,F0	Masking of bits with F0
1007	B1 04	MOV CL,04	
1009	B0 00	MOV AL,00	
100B	D2 EC	SHR AH,CL	Bits shifted right 4 times
100D	80 FC 00	CMP AH,00	
1010	74 0D	JE 101F	
1012	B7 00	MOV BH,00	
1014	04 16	ADD AL,16	
1016	27	DAA	Decimal adjust for addition
1017	73 02	JNC 101B	
1019	FE C7	INC BH	
101B	FE CC	DEC AH	
101D	75 F5	JNZ 1014	

101F	8A 26 00 11	MOV AH,[1100]	
1023	80 E4 0F	AND AH,0F	Masking of bits with 0F
1026	80 FC 0A	CMP AH,0A	Converting lower nibble to decimal
1029	72 03	JC 102E	
102B	80 C4 06	ADD AH,06	
102E	00 E0	ADD AL,AH	
1030	27	DAA	
1031	73 02	JNC 1035	
1033	FE C7	INC BH	
1035	88 3E 01 11	MOV [1101],BH	Higher byte of result in 1101
1039	A2 02 11	MOV [1102],AL	Lower byte of result in 1102
103C	F4	HLT	End of the program

INPUT

[1100]=99

OUTPUT

[1101]=01

[1102]=53

INPUT

[1100]=FF

OUTPUT

[1101]=02

[1102]=55

B.CONVERSION OF BCD NUMBER TO HEXADECIMAL — 8086 MICROPROCESSOR KIT

AIM

To write, execute and verify an Assembly Language Program (ALP) using the 8086 microprocessor kit to convert a given two-digit Packed BCD number into its equivalent Hexadecimal (Binary) number, and to store the result in memory / display it on the kit.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

1. Understand the difference between BCD representation and true binary (hexadecimal) representation of a number.
2. Apply arithmetic and logical instructions (AND, MUL, ADD, SHR) of the 8086 instruction set.
3. Perform nibble separation (splitting a byte into upper and lower 4 bits).
4. Use memory-referenced addressing to take input from and store output to specific memory locations on the kit.
5. Trace register contents (AL, AH, BL, CH) step-by-step to verify program correctness.
6. Execute and debug a program practically on an 8086 trainer kit using single-step/execute keys.

DESCRIPTION / THEORY

- BCD (Binary Coded Decimal): Each decimal digit (0–9) is represented using 4 bits. A packed BCD number stores two decimal digits in a single byte — the upper nibble is the tens digit, and the lower nibble is the units digit. Example: Decimal 25 → Packed BCD = 0010 0101 = 25H (but this is *not* equal to 25 in true binary value).
- Hexadecimal/Binary equivalent means the actual numeric value the number represents, computed as:

Hex Value=(Tens Digit×10)+Units Digit

$$\text{Hex Value} = (\text{Tens Digit} \times 10) + \text{Units Digit}$$

- Example: BCD 25H → Tens digit = 2, Units digit = 5
→ Hex value = $(2 \times 10) + 5 = 25$ (decimal) = 19H
- This conversion requires unpacking the BCD byte into two separate digits, multiplying the tens digit by 10, and adding the units digit — giving the true binary/hex equivalent.

ALGORITHM

1. Start.
2. Initialize the data segment (if using simulator) or directly load data into memory (if using kit).
3. Get the packed BCD number from the specified memory location into the accumulator (AL).
4. Make a copy of the number in another register (AH).
5. Mask the upper nibble (AND with 0FH) to extract the units digit. Store it temporarily (BL).
6. Take the original number again, mask the lower nibble (AND with F0H) to extract the tens digit.
7. Shift the tens digit right by 4 bits to align it as a normal number (0–9).
8. Multiply the tens digit by 10 (0AH) using the **MUL** instruction.
9. Add the units digit (stored in BL) to the product obtained.
10. Store the final result (true hexadecimal/binary value) into the specified output memory location.
11. Halt the program.

Memory Map used:

Address	Purpose
2000	Input – Packed BCD number
3000	Output – Hexadecimal equivalent

Address	Label	Mnemonic	Operand	Comments
2000		MOV	AL, [2000]	Get BCD number
		MOV	AH, AL	Copy number
		AND	AL, 0F	Mask lower nibble (units)
		MOV	BL, AL	Save units digit
		MOV	AL, AH	Restore original number
		AND	AL, F0	Mask upper nibble (tens)
		MOV	CL, 04	Load shift count
		SHR	AL, CL	Align tens digit
		MOV	CH, 0A	Load multiplier (10 decimal)
		MUL	CH	Tens digit $\times$ 10
		ADD	AL, BL	Add units digit
		MOV	[3000], AL	Store hex result
		HLT		Stop

PROCEDURE (For the 8086 Trainer Kit)

1. Switch ON the 8086 microprocessor kit.
2. Press RESET to clear the previous data/program.
3. Select the memory entry mode (usually the "MEM" key on the kit).
4. Enter the starting address of the program memory (e.g., 1000) using the keypad.
5. Enter the opcodes of the program instruction by instruction, pressing NEXT after each byte.

6. After entering the full program, switch to data entry mode and store the input BCD number at address 2000 (example: 25).
7. Return to execute mode, enter the starting address of the program (1000).
8. Press the EXECUTE / GO key to run the program (or use single-step key to trace instruction by instruction).
9. After execution, check the memory location 3000 for the result using the memory examine key.
10. Note down the result and compare with the manually calculated value.
11. Repeat with different BCD input values to verify the program.

SAMPLE OUTPUT

Input (BCD at 2000)	Output (Hex at 3000)	Verification
25H	19H	$(2 \times 10) + 5 = 25$ decimal = 19H ✓
99H	63H	$(9 \times 10) + 9 = 99$ decimal = 63H ✓
47H	2FH	$(4 \times 10) + 7 = 47$ decimal = 2FH ✓

MODEL VIVA QUESTIONS

1. What is the difference between BCD and hexadecimal number systems?
2. What is packed BCD and unpacked BCD?
3. Why do we multiply the tens digit by 10 (0AH) and not by 16?
4. What does the AND instruction do in this program, and why is masking needed?
5. Explain the working of the MUL instruction. Which registers does it use implicitly?
6. Why is AH used to store a copy of the original BCD number?
7. What is the purpose of the SHR instruction here?

8. What will be the result if the BCD input is 99H? Trace the execution.
9. What are the addressing modes used in this program?
10. Differentiate between MUL and IMUL.
11. How is data represented in memory in the 8086 (Little Endian format)?
12. What is the maximum decimal value a packed BCD byte can represent?
13. How would you modify the program to convert hexadecimal back to BCD (reverse conversion)?
14. What is the significance of the HLT instruction?
15. What are the flags affected by AND, ADD, and MUL instructions?

RESULT

The Assembly Language Program to convert a Packed BCD number into its equivalent Hexadecimal number was successfully written, executed, and verified using the 8086 microprocessor kit.

Thus, the given packed BCD number was successfully converted into its true hexadecimal equivalent, and the output was verified as correct.

C. CONVERSION OF DECIMAL NUMBER TO HEXADECIMAL

AIM

To write an Assembly Language Program (ALP) using the 8086 microprocessor kit to convert a given decimal number (up to 255) into its equivalent hexadecimal number and store the result in memory, with the input data also being fetched from memory.

LEARNING OUTCOMES

By the end of this experiment, the student will be able to:

1. Understand the internal number representation (BCD vs binary/hex) used by the 8086 microprocessor.
2. Learn to use arithmetic instructions like MUL, ADD, MOV for numeric conversions.
3. Understand memory addressing modes and how to fetch/store data from/to specific memory locations using an 8086 kit.
4. Learn the concept that a binary value computed in registers is inherently the hexadecimal equivalent of a decimal quantity.
5. Gain hands-on experience in entering, executing, and debugging programs on an 8086 trainer kit.
6. Understand register-level data flow (AL, AH, AX, BL, DX) during multiplication and addition operations.

DESCRIPTION

A decimal number (0–255) can be represented in memory as three separate unpacked digits: hundreds, tens, and units place. Since the 8086 processor performs all arithmetic in binary internally, if we compute:

$$\text{Value} = (\text{Hundreds digit} \times 100) + (\text{Tens digit} \times 10) + (\text{Units digit} \times 1)$$

using binary arithmetic instructions, the result obtained in the register is automatically the hexadecimal equivalent of the decimal number — no separate "conversion" logic is needed. For example, decimal 156 computed this way gives binary/hex value 9CH directly. This program fetches the three decimal digits from memory, computes the equivalent value, and stores the final hexadecimal byte back into memory.

ALGORITHM

1. Start.
2. Initialize data segment / memory pointer.
3. Fetch the hundreds digit from memory into AL.
4. Multiply AL by 100 (using MUL instruction) → result in AX.
5. Move the result (AX) into DX (used as accumulator/running sum).
6. Fetch the tens digit from memory into AL.
7. Multiply AL by 10 → result in AX.
8. Add AX to DX.
9. Fetch the units digit from memory into AL; clear AH.
10. Add AX (units) to DX.
11. Store the final result (lower byte, DL) into the specified memory location.
12. Terminate the program (HLT)

PROGRAM

Memory Map:

Address	Content	Description
2000H	Hundreds digit	Input
2001H	Tens digit	Input
2002H	Units digit	Input
2003H	Result	Output (Hex)

assembly

MOV AL, [2000] ; get hundreds digit

MOV BL, 100

MUL BL ; AX = hundreds * 100

MOV DX, AX ; DX = running total

```
MOV AL, [2001]    ; get tens digit
MOV BL, 10
MUL BL            ; AX = tens * 10
ADD DX, AX        ; DX = DX + AX

MOV AL, [2002]    ; get units digit
MOV AH, 00
ADD DX, AX        ; DX = DX + units

MOV [2003], DL    ; store hex result byte
HLT
```

PROCEDURE (to run on 8086 kit)

1. Switch ON the 8086 microprocessor trainer kit.
2. Press RESET to initialize the kit.
3. Select memory modify/examine mode and enter the input data:
 - At 2000H → hundreds digit
 - At 2001H → tens digit
 - At 2002H → units digit
4. Switch to program entry (opcode entry) mode.
5. Enter the machine/opcode instructions for the program starting from a chosen starting address (e.g., 1000H), as translated from the mnemonics above using the kit's opcode table.
6. After entering the full program, verify the entered opcodes by examining memory.
7. Set the Instruction Pointer (IP) to the starting address of the program (1000H).
8. Press EXECUTE / GO to run the program (or use Single Step to trace execution instruction by instruction).
9. Once execution halts (HLT encountered), go to memory examine mode.
10. Check memory location 2003H to view the resulting hexadecimal value.
11. Note down the result and switch OFF the kit.

SAMPLE INPUT / OUTPUT

Decimal Input	Digits Stored (2000H,2001H,2002H)	Result at 2003H
156	01, 05, 06	9CH
99	00, 09, 09	63H
255	02, 05, 05	FFH

MODEL VIVA QUESTIONS

1. What is the difference between BCD and binary/hexadecimal representation of a number?
2. Why does computing (digit $\times$ place value) and adding automatically give the hexadecimal equivalent?
3. What is the function of the MUL instruction? Which registers does it use implicitly?
4. Why is AH cleared before adding the units digit?
5. What is the maximum value that can be stored in a single byte (register like DL)? What happens if it's exceeded?
6. Why can decimal 256 not fit into an 8-bit (byte) register? How would you handle it?
7. Explain the difference between MOV AL, [2000] and MOV AL, 2000.
8. Differentiate between ADD and ADC instructions.
9. What is the purpose of the HLT instruction?
10. What is the significance of segment registers (DS, CS) in addressing memory in 8086?

RESULT

The Assembly Language Program to convert a decimal number into its equivalent hexadecimal number was executed successfully on the 8086 microprocessor kit. The input decimal digits were fetched from memory, converted, and the resultant hexadecimal value was successfully stored in the specified memory location.

EXP NO:3**IMPLEMENTATION OF SEARCHING AND SORTING OF 16-BIT NUMBERS****A.SEARCHING A 16-BIT NUMBER IN AN ARRAY****AIM**

To write and execute an Assembly Language Program (ALP) using the 8086 microprocessor kit to search for a given 16-bit number in an array of 16-bit numbers and display its position if found.

LEARNING OUTCOMES

By the end of this experiment, the student will be able to:

1. Understand the concept of linear search and its implementation in assembly language.
2. Learn the use of registers (SI, CX, BX, DI) for pointer arithmetic and looping.
3. Understand conditional and unconditional jump instructions (JZ, JMP, LOOP).
4. Learn to access and manipulate array elements stored in memory using indirect addressing.
5. Gain practical exposure to entering, executing, and debugging programs on an 8086 trainer kit.

DESCRIPTION

Searching is a fundamental operation in computer science used to locate a specific value within a collection of data. In this program, a linear search technique is used, where each element of the array is compared sequentially with the number to be searched until a match is found or the entire array has been scanned. The 8086 microprocessor provides string/loop instructions and conditional jumps that make such repetitive comparison operations efficient.

ALGORITHM

1. Initialize SI with the base address of the array.
2. Load the count of array elements into CX.
3. Load the number to be searched into BX.
4. Initialize DI = 0 (position counter).
5. Repeat:
 - a. Load the array element pointed to by SI into AX.
 - b. Compare AX with BX.
 - c. If equal, go to step 7.
 - d. Increment SI by 2 (to point to next 16-bit element).
 - e. Increment DI.
 - f. Decrement CX; if CX $\neq$ 0, repeat from step 5(a).
6. If loop completes without match, set DI = FFFFH (not found).
7. Store DI (position or FFFFH) at the result memory location.
8. Halt.

PROGRAM

```
MOV SI, 0500    ; SI = address of count
MOV CL, [SI]    ; CL = number of elements
MOV CH, 00
INC SI
INC SI
MOV BX, [050C]  ; BX = number to search
MOV DI, 0000    ; DI = position counter
```

```
BACK: MOV AX, [SI]
      CMP AX, BX
      JZ FOUND
      INC SI
      INC SI
      INC DI
      LOOP BACK
```

```
      MOV DI, FFFF
      JMP STORE
```

```
FOUND: NOP
STORE: MOV [050E], DI
      HLT
```

PROCEDURE

1. Switch ON the 8086 microprocessor trainer kit.
2. Enter the program using the keyboard monitor in the "Enter Program" mode, starting from the specified starting address (e.g., 0400H, as per kit convention).
3. Enter the array data (count + elements) and the number to be searched at the specified data memory locations (0500H onward).
4. Verify the entered opcodes by displaying them back.
5. Execute the program using the "Execute/Go" command from the starting address.
6. After execution, examine the memory location 050EH to view the result.
7. Note down the result and compare it with the expected (manually calculated) output.
8. Repeat with different data sets (number present, number absent) to verify correctness.

MODEL VIVA QUESTIONS

1. What is the difference between linear search and binary search?
2. What is the time complexity of linear search?
3. Why is CX used as a loop counter in this program?
4. What does the LOOP instruction do internally?
5. What is the difference between JZ and JNZ?
6. Why do we increment SI by 2 instead of 1 in this program?
7. What value is stored if the number is not found, and why?
8. What is the purpose of the DI register in this program?
9. Differentiate between direct and indirect addressing modes used here.
10. What are the flags affected by the CMP instruction?

RESULT

The assembly language program to search a 16-bit number in an array was written, executed successfully on the 8086 microprocessor trainer kit, and the position of the searched number was obtained correctly at memory location 050EH. When the number was not present, the result FFFFH was correctly displayed.

B.SORTING 16-BIT NUMBERS IN ASCENDING ORDER

AIM

To write and execute an Assembly Language Program using the 8086 microprocessor kit to sort an array of 16-bit numbers in ascending order using the bubble sort technique.

LEARNING OUTCOMES

By the end of this experiment, the student will be able to:

1. Understand the bubble sort algorithm and its implementation at the assembly level.
2. Learn to implement nested loops using registers in 8086 assembly.
3. Understand comparison and conditional swapping using CMP, JBE/JLE, and XCHG.
4. Learn to manage multiple counters (outer pass count and inner comparison count) using memory and registers.
5. Develop debugging skills for multi-loop programs on a trainer kit.

DESCRIPTION

Sorting arranges data in a particular order (ascending or descending). Bubble sort is one of the simplest sorting techniques, where adjacent elements are repeatedly compared and swapped if they are in the wrong order. This process is repeated for $(n-1)$ passes over the array, ensuring that after each pass, the next largest unsorted element "bubbles" to its correct position at the end of the array.

ALGORITHM

1. Initialize SI with the base address of the array and load the count (n) into CX.
2. Set outer loop counter = $n - 1$.
3. For each outer pass:
 - a. Reset SI to the start of the array.
 - b. Set inner loop counter = (remaining outer count).
 - c. For each inner comparison:
 - Load two adjacent elements into AX and BX.
 - Compare AX and BX.
 - If $AX > BX$, swap the two elements in memory.
 - Move the pointer to the next pair.
 - Decrement inner counter; repeat until zero.
4. Decrement outer counter; repeat step 3 until outer counter = 0.
5. Halt — array is now sorted in ascending order.

PROGRAM

```
MOV SI, 0500
MOV CL, [SI]
MOV CH, 00
DEC CX
MOV [0600], CX

OUTER: MOV CX, [0600]
      MOV SI, 0502
      MOV DX, CX

INNER: MOV AX, [SI]
      MOV BX, [SI+2]
      CMP AX, BX
      JBE NOSWAP
      XCHG AX, BX
      MOV [SI], AX
      MOV [SI+2], BX

NOSWAP: ADD SI, 2
      DEC DX
      JNZ INNER

      MOV CX, [0600]
      DEC CX
      MOV [0600], CX
      JNZ OUTER

      HLT
```

PROCEDURE

1. Switch ON the trainer kit and reset it.
2. Enter the program opcodes starting from the given program memory address.
3. Enter the unsorted array data (count + elements) at the specified data memory location (0500H onward).
4. Verify the entered program by displaying the memory contents.
5. Execute the program.
6. After execution, examine the array memory locations (0502H onward) to view the sorted result.
7. Compare the sorted output with the manually sorted array to verify correctness.
8. Repeat with different sets of unsorted data (already sorted, reverse sorted, random) to test all cases.

MODEL VIVA QUESTIONS

1. Explain the bubble sort algorithm with an example.
2. What is the time complexity of bubble sort (best, average, worst case)?
3. Why are two nested loops required in bubble sort?
4. What is the purpose of the XCHG instruction?
5. What is the difference between JBE and JLE? When would you use each?
6. How many passes are required to sort n elements using bubble sort?
7. What are the advantages and disadvantages of bubble sort compared to other sorting algorithms (like quick sort)?
8. Why do we decrement CX after every outer pass in this program?
9. Is bubble sort a stable sorting algorithm?
10. What modification would be needed to sort the array in descending order?

RESULT

The assembly language program to sort an array of 16-bit numbers in ascending order using the bubble sort technique was written, executed successfully on the 8086 microprocessor trainer kit, and the array was verified to be correctly sorted in ascending order at the specified memory locations.

EXP NO.4**STUDY OF ASSEMBLER AND DEBUGGING COMMANDS IN MASM****AIM**

To study the various assembler directives, and debugging commands used in MASM (Microsoft Macro Assembler) for writing, assembling, linking, executing, and debugging 8086 assembly language programs, and to execute a sample program demonstrating their use.

LEARNING OUTCOMES

By the end of this experiment, the student will be able to:

1. Understand the structure of an assembly language program written in MASM (segments, directives, procedures).
2. Differentiate between assembler directives and machine instructions.
3. Learn the steps involved in editing, assembling, linking, and executing an ALP using MASM.
4. Understand the use of DEBUG utility commands (D, U, T, G, R, A, E, Q) for tracing and verifying program execution.
5. Develop the ability to view and modify register contents, memory contents, and flags during program execution.
6. Gain skills in identifying and correcting assembly/logical errors using debugging tools.

DESCRIPTION

MASM (Microsoft Macro Assembler) is a software tool used to convert 8086 assembly language source code into machine-executable object code. A typical MASM program consists of segments (CODE, DATA, STACK) defined using assembler directives, which instruct the assembler on how to organize the program but are not translated into machine instructions themselves.

The process of running a program in MASM involves:

- **Editing** the source file (.ASM) using a text editor.
- **Assembling** it using MASM to produce an object file (.OBJ) and listing file (.LST).
- **Linking** the object file using LINK to produce an executable file (.EXE).
- **Debugging/Executing** the program using DEBUG or a similar debugger to trace execution, inspect registers/memory, and verify correctness.

DEBUG is a command-line utility that allows the programmer to load, examine, and step through a program at the machine-code level. It is extremely useful for verifying register values, memory contents, and flag conditions during execution — especially important for microprocessor lab exercises.

COMMON ASSEMBLER DIRECTIVES

Directive	Purpose
.MODEL	Specifies memory model (SMALL, MEDIUM, LARGE, etc.)
.STACK	Defines the size of the stack segment
.DATA	Defines the start of the data segment
.CODE	Defines the start of the code segment
DB	Define Byte – allocates one byte of storage
DW	Define Word – allocates two bytes of storage
DD	Define Doubleword – allocates four bytes
EQU	Assigns a symbolic name to a constant
ORG	Sets the starting offset address
END	Marks the end of the program
PROC / ENDP	Defines the start/end of a procedure
ASSUME	Tells assembler which segment registers to associate with which segments

COMMON DEBUG COMMANDS

Command	Function
A	Assemble – enter assembly instructions directly
U	Unassemble – displays machine code as mnemonics
D	Dump – displays contents of memory
E	Enter – modify contents of memory

R	Register – display/modify register contents
T	Trace – execute one instruction at a time
G	Go – execute program till a breakpoint
Q	Quit – exit DEBUG
N	Name – specify a program name for load/save
L	Load – load a program from disk into memory
W	Write – write program from memory to disk
C	Compare – compare two blocks of memory

ALGORITHM (Procedure for Studying Assembler & Debug Commands)

1. Open the text editor and write a sample 8086 ALP using MASM directives (.MODEL, .DATA, .CODE, etc.).
2. Save the file with a .ASM extension.
3. Assemble the program using the command: MASM filename.asm
4. Correct any syntax errors shown in the listing, and reassemble if necessary.
5. Link the object file using the command: LINK filename.obj to generate the .EXE file.
6. Load the .EXE file into DEBUG using: DEBUG filename.exe
7. Use the U command to unassemble and view the machine code.
8. Use the R command to view/modify register contents before execution.
9. Use the T command to trace execution instruction-by-instruction, observing register and flag changes.
10. Use the D command to view memory contents (data segment) before and after execution.
11. Use the G command with a breakpoint address to run the program up to a specific point.
12. Use the Q command to quit DEBUG after verifying the output.

SAMPLE PROGRAM (Addition of Two 8-bit Numbers using MASM)

asm

MODEL SMALL

.STACK 100H

.DATA

NUM1 DB 25H

NUM2 DB 35H

RESULT DB ?

.CODE

MAIN PROC

MOV AX, @DATA

MOV DS, AX

MOV AL, NUM1

ADD AL, NUM2

MOV RESULT, AL

MOV AH, 4CH

INT 21H

MAIN ENDP

END MAIN

PROCEDURE

1. Open the editor (EDIT or Notepad in DOS environment) and type the above sample program. Save as ADD.ASM.
2. At the DOS prompt, assemble the program:

MASM ADD.ASM

This generates ADD.OBJ and, optionally, ADD.LST.

3. Link the object file:

LINK ADD.OBJ

This generates ADD.EXE.

4. Load the executable into DEBUG:

DEBUG ADD.EXE

5. At the DEBUG prompt (-), use U 100 110 to unassemble and view the instructions in machine code form.
6. Use R to display all register contents before execution.
7. Use D DS:0000 to view the data segment contents (NUM1, NUM2, RESULT) before execution.

8. Use T repeatedly to trace through the program instruction-by-instruction, observing how AL and the flags change after each step.
9. After execution, use D DS:0000 again to verify that RESULT contains the correct sum ($25H + 35H = 5AH$).
10. Use Q to quit DEBUG.

MODEL VIVA QUESTIONS

1. What is the difference between an assembler directive and a machine instruction?
2. What is the function of the .MODEL directive, and what are the different memory models available?
3. Differentiate between DB, DW, and DD directives.
4. What is the purpose of the ASSUME directive?
5. Explain the difference between assembling, linking, and execution.
6. What is the difference between the T (Trace) and G (Go) commands in DEBUG?
7. How do you set a breakpoint in DEBUG, and what command is used?
8. What information does the R command display?
9. What is the purpose of the U (Unassemble) command?
10. What is the difference between a .COM file and an .EXE file?
11. What does the INT 21H with AH=4CH do at the end of the program?
12. How can you modify the contents of a register using DEBUG?
13. What is the significance of the .LST file generated after assembly?
14. What are segment registers, and why is MOV AX, @DATA followed by MOV DS, AX necessary?

RESULT

The various assembler directives and debugging commands used in MASM were studied successfully. A sample 8086 assembly language program for addition of two 8-bit numbers was written, assembled, linked, and executed using MASM and DEBUG utilities. The program was traced step-by-step using the T command, and the correct result (5AH) was verified in the data segment using the D command.

EXP NO.5**IMPLEMENTATION OF DECIMAL ARITHMETIC (16-BIT AND 32-BIT) OPERATIONS IN MASM****AIM**

To write, assemble, and execute MASM assembly language programs to perform decimal (BCD) arithmetic operations — addition and subtraction — on 16-bit (4-digit) and 32-bit (8-digit) packed BCD numbers using the 8086 instruction set.

LEARNING OUTCOMES

By the end of this experiment, the student will be able to:

1. Understand the concept of Binary Coded Decimal (BCD) representation — packed and unpacked.
2. Learn the purpose and working of decimal adjust instructions: DAA (Decimal Adjust after Addition) and DAS (Decimal Adjust after Subtraction).
3. Understand how the Auxiliary Carry Flag (AF) and Carry Flag (CF) are used internally for BCD correction.
4. Learn to extend arithmetic operations beyond a single byte to multi-byte (16-bit and 32-bit) decimal quantities using carry propagation.
5. Develop the ability to write modular, loop-based multi-precision arithmetic routines in MASM.
6. Gain practical understanding of the difference between binary arithmetic and decimal arithmetic at the processor level.

DESCRIPTION

Decimal arithmetic in 8086 is performed using **packed BCD** format, where each byte holds two decimal digits (0–9 each), rather than pure binary values. Since the ALU of the 8086 performs binary addition/subtraction internally, the result of adding two BCD digits may not be a valid decimal digit (e.g., $9 + 8 = 11\text{H}$, which is not a valid BCD digit pair). The DAA and DAS instructions correct (adjust) the result in AL after an ADD/SUB operation so that it represents a valid packed BCD result.

- **DAA:** Used immediately after ADD/ADC on two packed BCD bytes in AL, to adjust AL into valid packed BCD.
- **DAS:** Used immediately after SUB/SBB on two packed BCD bytes in AL, to adjust AL into valid packed BCD.

For **16-bit BCD numbers** (4 decimal digits = 2 bytes), and **32-bit BCD numbers** (8 decimal digits = 4 bytes), the operation is extended by processing the numbers **byte-by-byte from the least significant byte to the most significant byte**, using ADC/SBB (add/subtract with carry) along with DAA/DAS for each byte, so that carry/borrow propagates correctly across bytes.

ALGORITHM

(A) 16-bit (4-digit) BCD Addition

1. Initialize DS with data segment address.
2. Clear the carry flag.
3. Load the lower byte of BCD1 and BCD2 into AL and BL respectively.
4. Add AL and BL using ADC (to include carry from any previous step, initially 0).
5. Apply DAA to adjust the result into valid packed BCD.
6. Store the corrected lower byte of the result.
7. Load the upper byte of BCD1 and BCD2 into AL and BL.
8. Add using ADC (carries in from step 5's carry flag).
9. Apply DAA to adjust the result.
10. Store the corrected upper byte of the result.
11. Display/store the final 4-digit BCD sum.
12. Halt.

(B) 32-bit (8-digit) BCD Addition

Same as above, but repeated for 4 bytes (LSB to MSB) instead of 2, using a loop with ADC + DAA at each byte, propagating carry from one byte to the next.

(C) BCD Subtraction (16-bit / 32-bit)

Same procedure as addition, but using SBB (subtract with borrow) followed by DAS at each byte, propagating borrow from LSB to MSB.

PROGRAM

(A) 16-bit Packed BCD Addition

```
asm
MODEL SMALL
.STACK 100H

.DATA
BCD1 DB 45H, 67H ; 4-digit BCD number: 6745
BCD2 DB 38H, 29H ; 4-digit BCD number: 2938
RESULT DB 2 DUP(?)

.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX

CLC ; clear carry
MOV AL, BCD1[0] ; lower byte of BCD1
MOV BL, BCD2[0] ; lower byte of BCD2
ADC AL, BL
DAA
```

```

MOV RESULT[0], AL    ; store lower byte of result

MOV AL, BCD1[1]      ; upper byte of BCD1
MOV BL, BCD2[1]      ; upper byte of BCD2
ADC AL, BL
DAA
MOV RESULT[1], AL    ; store upper byte of result

MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN

```

Expected Result: 6745 + 2938 = 9683 → RESULT = 83H, 96H

(B) 32-bit Packed BCD Addition (Loop-based, 4 bytes)

```

asm
MODEL SMALL
.STACK 100H

.DATA
BCD1 DB 12H, 34H, 56H, 78H ; 8-digit BCD: 78563412 (LSB first)
BCD2 DB 11H, 22H, 33H, 44H ; 8-digit BCD: 44332211 (LSB first)
RESULT DB 4 DUP(?)
COUNT DB 04H

.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX

LEA SI, BCD1
LEA DI, BCD2
LEA BX, RESULT
MOV CL, COUNT
MOV CH, 00
CLC                ; clear carry before first byte

NEXT_BYTE:
MOV AL, [SI]
MOV AH, [DI]
ADC AL, AH
DAA
MOV [BX], AL
INC SI
INC DI
INC BX
LOOP NEXT_BYTE    ; Note: LOOP does not disturb CF (safe here)

```

```
MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN
```

Expected Result: $78563412 + 44332211 = 122895623 \rightarrow$ since it's 32-bit (8-digit) BCD, result wraps/stores across the 4 bytes with final carry (if any) indicating a 9th digit overflow.

(C) 16-bit Packed BCD Subtraction

asm

```
.MODEL SMALL
.STACK 100H
```

```
.DATA
```

```
BCD1 DB 45H, 67H ; Minuend: 6745
BCD2 DB 38H, 29H ; Subtrahend: 2938
RESULT DB 2 DUP(?)
```

```
.CODE
```

```
MAIN PROC
```

```
MOV AX, @DATA
MOV DS, AX
```

```
CLC
```

```
MOV AL, BCD1[0]
```

```
MOV BL, BCD2[0]
```

```
SBB AL, BL
```

```
DAS
```

```
MOV RESULT[0], AL
```

```
MOV AL, BCD1[1]
```

```
MOV BL, BCD2[1]
```

```
SBB AL, BL
```

```
DAS
```

```
MOV RESULT[1], AL
```

```
MOV AH, 4CH
```

```
INT 21H
```

```
MAIN ENDP
```

```
END MAIN
```

Expected Result: $6745 - 2938 = 3807 \rightarrow \text{RESULT} = 07\text{H}, 38\text{H}$

PROCEDURE

1. Open the editor and type the desired program (16-bit addition / 32-bit addition / subtraction). Save with a .ASM extension (e.g., BCD16.ASM).
2. Assemble the program:

MASM BCD16.ASM

3. Link the object file:

LINK BCD16.OBJ

4. Run the program under DEBUG:

DEBUG BCD16.EXE

5. Use D DS:0000 to view the initial BCD operand values in the data segment.
6. Use T to trace instruction-by-instruction, observing:
 - Changes in AL after ADC/SBB.
 - Changes in AL after DAA/DAS (compare before and after adjustment).
 - Status of Carry Flag (CF) and Auxiliary Carry Flag (AF) after each step.
7. After execution, use D DS:0000 again to verify that the RESULT array contains the correct packed BCD sum/difference.
8. Repeat with different BCD values, including cases that generate a carry/borrow out of the most significant digit, to verify correct multi-byte propagation.

MODEL VIVA QUESTIONS

1. What is packed BCD and unpacked BCD? Give examples.
2. Why is DAA needed after a normal binary ADD when working with BCD numbers?
3. What is the difference between DAA and AAA?
4. What is the difference between DAS and AAS?
5. Which flags are affected/used by the DAA instruction?
6. Why is ADC used instead of ADD for the higher bytes in multi-byte BCD addition?
7. What happens if you don't clear the carry flag (CLC) before starting the addition of the first byte?
8. How would you detect a final overflow (9th digit) in the 32-bit BCD addition program?
9. What is the significance of the Auxiliary Carry Flag (AF) in decimal arithmetic?
10. Why is BCD arithmetic needed at all when the processor is fundamentally binary?
11. How would you modify the 16-bit addition program to work for 32-bit numbers?
12. Can DAA/DAS be used with 16-bit registers (AX) directly? Why or why not?

RESULT

The MASM assembly language programs for performing decimal (BCD) arithmetic operations on 16-bit and 32-bit packed BCD numbers were written, assembled, linked, and executed successfully. The DAA and DAS instructions correctly adjusted the results after binary addition/subtraction, and the final results verified using the DEBUG utility matched the expected decimal sums and differences.

EXP NO.6**IMPLEMENTATION OF STRING MANIPULATION****AIM**

To write, assemble, and execute Assembly Language Programs (ALPs) using MASM (Microsoft Macro Assembler) to perform basic string manipulation operations — finding the length of a string, reversing a string, and checking whether a string is a palindrome.

LEARNING OUTCOME

After completing this experiment, the student will be able to:

1. Understand the internal representation of strings in 8086 assembly language.
2. Use index registers (SI, DI) and string-related instructions to traverse and process character data.
3. Apply looping and conditional jump instructions to implement iterative logic.
4. Use DOS interrupt INT 21H for character/string input and output.
5. Understand the difference between .OBJ, .EXE generation and the assemble–link–execute cycle for MASM programs.

DESCRIPTION

In 8086 assembly language, a string is typically stored as a sequence of bytes in the data segment, terminated by a special character (commonly \$ when using DOS function calls for display). String manipulation programs traverse this byte sequence using a pointer register (SI or DI), performing operations character by character until the terminator is reached.

Three classic string operations are implemented below:

- **Program 1:** Find the length of a string.
- **Program 2:** Reverse a given string.
- **Program 3:** Check if a given string is a palindrome.

ALGORITHM**Program 1: Find the Length of a String****Algorithm:**

1. Start
2. Initialize data segment
3. Load the address of the string into a register (SI), using LEA or MOV with OFFSET
4. Initialize a counter (CX or a memory variable) to 0
5. Repeat:
 - Read the byte pointed to by SI
 - If the byte equals the terminating character (e.g., '\$' or 0), stop the loop
 - Otherwise, increment the counter and increment SI

6. Store/display the counter value as the length
7. Stop

Key registers: SI (string pointer), CX/AL (counter/comparison), terminator check (\$ for DOS string, or 0 for null-terminated)

Program 2: Reverse a Given String

Algorithm:

1. Start
2. Initialize data segment
3. Load the address of the string into SI
4. Find the length of the string (traverse until terminator, count characters) — call it n
5. Set SI to point to the start of the string, and DI to point to the last character (start + n - 1)
6. Repeat n/2 times (or until SI ≥ DI):
 - Exchange the byte at [SI] with the byte at [DI]
 - Increment SI, decrement DI
7. Display/store the reversed string
8. Stop

Key registers: SI (left pointer), DI (right pointer), CX (loop counter = n/2), AL/temp register (for swapping)

Program 3: Check if a Given String is a Palindrome

Algorithm:

1. Start
2. Initialize data segment
3. Load the address of the string into SI
4. Find the length of the string n (traverse until terminator)
5. Set SI to point to the start, and DI to point to the last character (start + n - 1)
6. Set a flag = "palindrome" (e.g., a byte set to 1)
7. Repeat n/2 times:
 - Compare byte at [SI] with byte at [DI]
 - If they are not equal:
 - Set flag = "not palindrome"
 - Exit loop (break)
 - Else, increment SI, decrement DI
8. After the loop, check the flag:
 - If flag = "palindrome", display "String is Palindrome"
 - Else, display "String is Not Palindrome"
9. Stop

Key registers: SI (left pointer), DI (right pointer), CX (loop counter), AL/BL/flag variable

(comparison result)

PROGRAM 1: FIND THE LENGTH OF A STRING

asm

.MODEL SMALL

.STACK 100H

.DATA

STR1 DB 'HELLO\$'

LEN DB ?

MSG DB 10,13,'LENGTH OF STRING IS : \$'

.CODE

MAIN PROC

MOV AX, @DATA

MOV DS, AX

LEA SI, STR1

MOV CL, 0

UP:

CMP BYTE PTR [SI], '\$'

JE DOWN

INC SI

INC CL

JMP UP

DOWN:

MOV LEN, CL

LEA DX, MSG

MOV AH, 09H

INT 21H

MOV DL, LEN

ADD DL, 30H ; convert to ASCII (works for single-digit length)

MOV AH, 02H

INT 21H

MOV AH, 4CH

INT 21H

MAIN ENDP

END MAIN

OUTPUT:

LENGTH OF STRING IS : 5

PROGRAM 2: REVERSE A STRING**asm**

```
.MODEL SMALL
.STACK 100H
.DATA
    STR1 DB 'HELLO$'
    MSG1 DB 10,13,'ENTERED STRING : $'
    MSG2 DB 10,13,'REVERSED STRING : $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, MSG1
    MOV AH, 09H
    INT 21H
    LEA DX, STR1
    MOV AH, 09H
    INT 21H

    LEA SI, STR1
    MOV CX, 0
UP:
    CMP BYTE PTR [SI], '$'
    JE DOWN
    INC SI
    INC CX
    JMP UP

DOWN:
    DEC SI
    LEA DX, MSG2
    MOV AH, 09H
    INT 21H

REV:
    MOV DL, [SI]
    MOV AH, 02H
    INT 21H
    DEC SI
    LOOP REV

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

OUTPUT

ENTERED STRING : HELLO
REVERSED STRING : OLLEH

PROGRAM 3: CHECK IF A STRING IS A PALINDROME

asm

```
.MODEL SMALL
.STACK 100H
.DATA
    STR1 DB 'MADAMS$'
    MSG1 DB 10,13,'STRING IS A PALINDROMES$'
    MSG2 DB 10,13,'STRING IS NOT A PALINDROMES$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, STR1
    LEA DI, STR1
    MOV CX, 0

UP:
    CMP BYTE PTR [SI], '$'
    JE DOWN
    INC SI
    INC CX
    JMP UP

DOWN:
    DEC SI                ; SI now points to last character

CHECK:
    CMP CX, 0
    JLE PALIN
    MOV AL, [SI]
    CMP AL, [DI]
    JNE NOTPALIN
    DEC SI
    INC DI
    SUB CX, 2
    JMP CHECK

PALIN:
    LEA DX, MSG1
    JMP DISPLAY

NOTPALIN:
    LEA DX, MSG2
```

DISPLAY:

```
MOV AH, 09H  
INT 21H
```

```
MOV AH, 4CH  
INT 21H
```

```
MAIN ENDP
```

```
END MAIN
```

OUTPUT:

STRING IS A PALINDROME

PROCEDURE

1. Open the MASM/TASM working environment (commonly run inside **DOSBox** on modern systems).
2. Mount the working folder as a virtual drive: MOUNT C C:\MASM
3. Switch to the mounted drive: C:
4. Open a text editor and type the program: EDIT PROG1.ASM
5. Save the file and exit the editor.
6. Assemble the source file to generate the object file: MASM PROG1.ASM
7. Link the object file to generate the executable file: LINK PROG1.OBJ
8. Run the executable file: PROG1.EXE
9. Observe and note the output displayed on the screen.
10. Repeat steps 4–9 for each of the three programs.

MODEL VIVA QUESTIONS

1. What is MASM? How does it differ from TASM?
2. What are the DOS-provided string instructions (MOVS, CMPS, SCAS, LODS, STOS) and how do they differ from a manually written loop?
3. What is the role of the Direction Flag (DF), and what do CLD and STD do?
4. Why are SI and DI called index registers, and which segment does each default to (DS and ES)?
5. Why is \$ used as a string terminator in DOS assembly programs?
6. What is the significance of INT 21H? What do the function codes AH=09H, AH=02H, and AH=4CH represent?
7. What is the difference between a .COM file and an .EXE file?
8. What is the purpose of the LOOP instruction, and which register does it implicitly use as a counter?
9. What is a palindrome, and what is the logic used here to check it?
10. Explain segmented memory addressing in the 8086 (CS, DS, SS, ES) and how a physical address is calculated.

RESULT

The Assembly Language Programs to find the length of a string, reverse a string, and check whether a string is a palindrome were successfully written, assembled, executed using MASM, and the outputs were verified to be correct.

EXP.NO.7**SEARCHING AND SORTING OF 16-BIT NUMBERS USING MASM****AIM**

To write, assemble, and execute an 8086 Assembly Language Program (ALP) using MASM to:

1. Search for a given 16-bit number in an array of 16-bit numbers (Linear Search).
2. Sort a given array of 16-bit numbers in ascending order (Bubble Sort).

LEARNING OUTCOME

After completing this experiment, the student will be able to:

- Understand the internal architecture and addressing modes of the 8086 microprocessor.
- Write and debug assembly language programs using MASM/TASM assembler directives.
- Implement fundamental data structure operations (searching and sorting) at the register/memory level.
- Understand how 16-bit data is stored, compared, and exchanged in memory using registers such as AX, BX, CX, SI, DI.
- Apply looping, comparison, and conditional jump instructions (CMP, JE, JNE, JG, JL, LOOP, JMP) to solve real problems.
- Analyze time and space complexity of searching and sorting at a low level (register/memory operations).

DESCRIPTION

- Searching means finding whether a particular element (16-bit number) is present in a given array, and if present, finding its position. Here, Linear Search is used — each element of the array is compared one by one with the key element until a match is found or the array ends.
- Sorting means arranging the elements of an array in a particular order (ascending here). Here, Bubble Sort is used — repeatedly comparing adjacent elements and swapping them if they are in the wrong order, until the entire array is sorted.
- Since the numbers are 16-bit, each element occupies 2 bytes in memory, and the DW (Define Word) directive is used to declare them. Register pairs and word-sized instructions (CMP AX, [SI], XCHG, MOV WORD PTR) are used instead of byte-sized ones.

1. LINEAR SEARCH (16-bit Numbers)**ALGORITHM**

1. Start
2. Initialize SI to point to the base address of the array; initialize CX with the number of elements (count).
3. Load the key (number to be searched) into AX.

4. Compare AX with the array element pointed to by SI.
5. If equal, display "Number Found" along with its position, and go to step 8.
6. Else, increment SI by 2 (word size) and decrement CX.
7. Repeat steps 4–6 until CX = 0.
8. If loop ends without a match, display "Number Not Found".
9. Stop

PROGRAM

asm

DATA SEGMENT

```

ARRAY  DW 0010H, 0025H, 0032H, 0047H, 0055H, 0060H, 0075H, 0089H
COUNT EQU 08H
KEY    DW 0047H
FOUND_MSG DB 10,13,'NUMBER FOUND AT POSITION: $'
NOTFOUND_MSG DB 10,13,'NUMBER NOT FOUND$'
POS     DB 2 DUP('$')

```

DATA ENDS

CODE SEGMENT

```
ASSUME CS:CODE, DS:DATA
```

START:

```

MOV AX, DATA
MOV DS, AX

```

```

LEA SI, ARRAY
MOV CX, COUNT
MOV AX, KEY
MOV BX, 01H      ; position counter

```

SEARCH_LOOP:

```

CMP AX, [SI]
JE  MATCH_FOUND
INC SI
INC SI      ; move to next word (16-bit = 2 bytes)
INC BX
LOOP SEARCH_LOOP

```

```

; Not found
LEA DX, NOTFOUND_MSG
MOV AH, 09H
INT 21H
JMP EXIT

```

MATCH_FOUND:

```

LEA DX, FOUND_MSG
MOV AH, 09H
INT 21H

```

```

; Convert position (BX) to ASCII and display
MOV AX, BX
AAM

```

```
ADD AX, 3030H
MOV DL, AH
MOV AH, 02H
INT 21H
MOV DL, AL
MOV AH, 02H
INT 21H
```

EXIT:

```
MOV AH, 4CH
INT 21H
CODE ENDS
END START
```

OUTPUT

NUMBER FOUND AT POSITION: 04

(If KEY = 0047H is changed to a value not present in the array, output will be:)

NUMBER NOT FOUND

2. BUBBLE SORT (16-bit Numbers – Ascending Order)

ALGORITHM

1. Start
2. Initialize CX1 = (n - 1) for the outer loop, where n is number of elements.
3. Initialize SI to point to the base address of the array for the inner loop.
4. Compare the current element with the next element.
5. If the current element is greater than the next element, swap (exchange) them.
6. Move to the next pair and repeat step 4–5 for all elements in the current pass (inner loop).
7. Decrement the outer loop counter and repeat steps 3–6 until CX1 = 0.
8. Display the sorted array.
9. Stop

PROGRAM

asm

DATA SEGMENT

```
ARRAY DW 0089H, 0025H, 0047H, 0010H, 0075H, 0032H, 0060H, 0055H
COUNT EQU 08H
RES_MSG DB 10,13,'SORTED ARRAY (HEX): $'
SPACE DB '$'
```

DATA ENDS

CODE SEGMENT

```
ASSUME CS:CODE, DS:DATA
```

START:

```
MOV AX, DATA
MOV DS, AX
```

```
    MOV CX, COUNT-1      ; outer loop counter = n-1
OUTER_LOOP:
    PUSH CX
    LEA SI, ARRAY
    MOV CX, COUNT-1

INNER_LOOP:
    MOV AX, [SI]
    MOV BX, [SI+2]
    CMP AX, BX
    JLE NO_SWAP
    XCHG AX, BX
    MOV [SI], AX
    MOV [SI+2], BX

NO_SWAP:
    INC SI
    INC SI
    LOOP INNER_LOOP

    POP CX
    LOOP OUTER_LOOP

; Display sorted array in Hex
    LEA DX, RES_MSG
    MOV AH, 09H
    INT 21H

    LEA SI, ARRAY
    MOV CX, COUNT
PRINT_LOOP:
    MOV AX, [SI]
    CALL PRINT_HEX
    LEA DX, SPACE
    MOV AH, 09H
    INT 21H
    INC SI
    INC SI
    LOOP PRINT_LOOP

    MOV AH, 4CH
    INT 21H

; --- Procedure to print a 16-bit number in AX as 4 hex digits ---
PRINT_HEX PROC
    PUSH CX
    PUSH DX
    MOV CX, 4
HEX_DIGIT:
    ROL AX, 4
```

```
PUSH AX
AND AL, 0FH
CMP AL, 0AH
JL IS_DIGIT
ADD AL, 07H
IS_DIGIT:
ADD AL, 30H
MOV DL, AL
MOV AH, 02H
INT 21H
POP AX
LOOP HEX_DIGIT
POP DX
POP CX
RET
PRINT_HEX ENDP

CODE ENDS
END START
```

OUTPUT

SORTED ARRAY (HEX): 0010 0025 0032 0047 0055 0060 0075 0089

PROCEDURE (How to Run in MASM/TASM/DOSBox)

1. Open the MASM/TASM working directory (or mount the folder in DOSBox using MOUNT C: C:\MASM).
2. Type EDIT SEARCH.ASM (or use any text editor) and enter the linear search program; save it. Repeat for SORT.ASM with the bubble sort program.
3. Assemble the program: MASM SEARCH.ASM (creates SEARCH.OBJ)
4. Link the object file: LINK SEARCH.OBJ (creates SEARCH.EXE)
5. Run the executable: SEARCH.EXE (or SEARCH at the DOS prompt)
6. Observe and note the output on screen.
7. Repeat steps 3–6 for SORT.ASM.

MODEL VIVA QUESTIONS

1. What is the difference between MASM and TASM?
2. Why is DW used instead of DB in this program?
3. What is the time complexity of Linear Search and Bubble Sort?
4. Why do we increment SI by 2 and not by 1 while traversing a 16-bit array?
5. What is the purpose of the LOOP instruction? Which register does it implicitly use?

6. Differentiate between CMP and SUB instructions.
7. What are the different addressing modes used in this program?
8. How does the XCHG instruction work, and why is it used in Bubble Sort?
9. What is the significance of the ASSUME directive?
10. How can Bubble Sort be optimized (e.g., using a flag to detect an already-sorted array)?
11. What is the difference between Bubble Sort and Selection Sort?
12. What are the various interrupt services (INT 21H) used in this program, and what do AH=09H and AH=02H represent?
13. Explain the difference between JE, JL, JG, JLE, JGE conditional jumps.
14. What is a segment override, and is it needed here?
15. How would you modify this program to perform a Binary Search instead of Linear Search?

RESULT

The 8086 Assembly Language Programs (using MASM) for Linear Search and Bubble Sort of 16-bit numbers were successfully written, assembled, executed, and verified. The search program correctly located the position of the given key element, and the sort program successfully arranged the 16-bit array in ascending order.

EXP.NO.8**INTERFACING WITH STEPPER MOTOR - ROTATE THROUGH ANY GIVEN SEQUENCE.****AIM**

To interface a stepper motor with 8086 microprocessor kit.

LEARNING OUTCOME

By the end of this experiment, the student will be able to:

1. Understand the working principle of a stepper motor and how discrete angular steps are achieved through sequential winding excitation.
2. Differentiate between the various stepper motor excitation schemes — wave scheme, 2-phase scheme, half-stepping, and wave switching scheme — and identify their respective step angles.
3. Interface a stepper motor to the 8086 microprocessor kit through a peripheral port (e.g., 8255 PPI) and drive it using digital output patterns.
4. Write and execute 8086 assembly language programs to generate the required switching sequences for clockwise and anticlockwise rotation.
5. Implement software time delays using nested loop/counter routines to control the stepping rate (speed) of the motor.
6. Analyze how reversing the excitation sequence reverses the direction of motor rotation.
7. Correlate step count, step angle, and switching pattern with the actual mechanical rotation of the motor shaft.
8. Debug and verify hardware-software interfacing issues (port addressing, data format, timing) in a real-time control application.

DESCRIPTION

A motor in which the rotor is able to assume only discrete stationary angular position is a stepper motor. The rotary motion occurs in a stepwise manner from one equilibrium position to the next.

Stepper motor control is a very popular application of microprocessor in control area. They are widely used in (simple position control systems in the open and closed loop mode) a variety of applications such as computer peripherals and in the areas of process control machine tools, medicine, numerically controlled machines and Robotics.

a) WAVE SCHEME

The stepper motor windings A1, B1, A2, B2 can be cyclically excited with a DC current to run the motor in the clockwise direction. By reversing the phase sequences as A1, B2, A2, B1. we can obtain anticlockwise stepping.

ANTI CLOCK WISE

STEP1 A1 ON
STEP2 B2 ON

CLOCK WISE

STEP1 A1 ON
STEP2 B1 ON

STEP3 A2 ON
STEP4 B1 ON

STEP3 A2 ON
STEP4 B2 ON

ANTICLOCK WISE

STEP A1 A2 A3 A4

1	1	0	0	0
2	0	0	0	1
3	0	1	0	0
4	0	0	1	0

CLOCK WISE

STEP A1 A2 A3 A4

1	1	0	0	0
2	0	0	1	0
3	0	1	0	0
4	0	0	0	1

b) 2 – PHASE SCHEME

In this scheme, any two adjacent stator windings are energized. There are two magnetic fields active in the quadrature and none of the rotor pole faces can be in direct alignment with the stator poles. A partial but symmetric alignment of the rotor poles is possible. The step angle is 30 degree and the rotor is offset by 15 degree with respect to the wave scheme.

Anti clock Wise

STEP1 A1 ON, B2 ON
STEP2 B2 ON, B2 ON
STEP3 A2 ON, B1 ON
STEP4 B1 ON, B1 ON

Clock Wise

STEP1 A1 ON, B1 ON
STEP2 B1 ON, B1 ON
STEP3 A2 ON, B2 ON
STEP4 B2 ON, B2 ON

ANTICLOCK WISE

STEP A1 A2 A3 A4

1	1	0	0	1
2	0	1	0	1
3	0	1	1	0
4	1	0	1	0

CLOCK WISE

STEP A1 A2 A3 A4

1	1	0	1	0
2	0	1	1	0
3	0	1	0	1
4	1	0	0	1

c) HALF STEPPING

This wave scheme and 2 phase scheme give step size of 30 degree. There is an offset of 15 degree in between the two states. By interleaving the 2 schemes the step size can be reduced to 15 degree thereby improving the accuracy of the stepper motor.

The switching sequence is:

- A1 (ON)
- A1 and B1 (ON)
- B1 (ON)
- B1 and A2 (ON)
- A2 (ON)
- A2 and B2 (ON)
- B2 (ON)
- B2 and A1 (ON)
- A1 (ON), etc...

Eight steps are required to move the shaft by 120 degree and 24 steps for one complete revolution. By reversing the switching sequence, we can reverse the direction of rotation.

d) WAVE SWITCHING SCHEME

Here only one pole is energized by the microprocessor. When A2 alone is energized, the nearest rotor pole locks with it. When B2 alone is energized the nearest rotor pole is locked with it. Totally 12 such steps cause a displacement of 360 degree. Thus the step angle is 30 degree.

STEP	A1	A2	B1	B2
1	1	0	0	0
2	1	0	1	0
3	0	0	1	0
4	0	1	1	0
5	0	1	0	0
6	0	1	0	1
7	0	0	0	1
8	1	0	0	1

PROGRAM

TO RUN THE STEPPER MOTOR IN BOTH FORWARD AND REVERSE DIRECTION WITH DELAY

Address	Machine Code	Mnemonic
1000	C6 C3 20	MOV BL, 20
1003	C7 C6 40 10	MOV SI, 1040
1007	E8 13 00	CALL 101D
100A	E8 2B 00	CALL 1038
100D	C6 C3 20	MOV BL, 20
1010	C7 C6 44 10	MOV SI, 1044
1014	E8 06 00	CALL 101D
1017	E8 1E 00	CALL 1038
101A	E9 E3 FF	JMP 1000
101D	89 F7	MOV DI, SI
101F	C6 C1 04	MOV CL, 04
1022	8A 05	MOV AL, [DI]
1024	E6 C0	OUT C0, AL
1026	C7 C2 10 10	MOV DX, 1010
102A	4A	DEC DX
102B	75 FD	JNZ 102A
102D	FE CB	DEC BL
102F	74 06	JZ 1037
1031	47	INC DI
1032	E2 EE	LOOP 1022
1034	E9 E6 FF	JMP 101D
1037	C3	RET
1038	C7 C2 FF FF	MOV DX, 0FFFF

103C	4A	DEC DX
103D	75 FD	JNZ 103C
103F	C3	RET
1040	09 05 06 0A	FORWARD DATA
1044	0A 06 05 09	REVERSE DATA

Program Logic Summary

1. 1000–100A: Loads step count (20H = 32 steps) into BL, points SI to FORWARD data table (1040), calls the stepping routine (101D), then calls the delay routine (1038) — driving the motor forward.
2. 100D–1017: Repeats the same process but points SI to REVERSE data table (1044) — driving the motor in reverse.
3. 101A: Jumps back to 1000, so the sequence (forward → reverse) repeats indefinitely.
4. 101D–1037: The stepping subroutine — copies SI into DI, sets CL = 4 (4-step pattern per table), fetches each byte from the table via [DI], outputs it to port C0 (driving the motor windings), executes a small delay, decrements BL (overall step counter), checks if BL = 0 to exit, else increments DI to point to the next pattern byte and loops (LOOP 1022) 4 times before repeating the whole 4-byte pattern again (via JMP 101D) until BL reaches 0.
5. 1038–103F: A simple delay subroutine using DX loaded with FFFFH, decremented in a tight loop — inserted between motor commands to control stepping speed.
6. 1040 & 1044: Data tables holding the excitation pattern bytes for forward and reverse rotation, sent out to the port to energize the stator windings in sequence.

Model Viva Questions

1. What is a stepper motor? How does it differ from a normal DC or AC motor?
2. What is meant by "step angle"? How is it calculated?
3. Explain the wave scheme of stepper motor excitation. What is its step angle?
4. What is the 2-phase excitation scheme? Why is the rotor said to be "offset by 15°" compared to the wave scheme?
5. What is half-stepping, and why is it used? How does interleaving the wave and 2-phase schemes reduce the step size?
6. Explain the wave switching scheme. How many steps are required for one full revolution, and what is the resulting step angle?
7. How is the direction of rotation (clockwise vs. anticlockwise) reversed in software?
8. What are the practical applications of stepper motors (mention at least 3–4 fields)?
9. Why can't a stepper motor rotate continuously like a DC motor — why does it move only in discrete steps?
10. What determines the torque and holding capability of a stepper motor at a given step?

RESULT

The stepper motor was interfaced with 8086 microprocessor and related programming experiments have been carried out.

EXP.NO.9**INTERFACING WITH 8255 (MODE 0 AND MODE 1)****AIM**

To study the architecture of the 8255 Programmable Peripheral Interface (PPI) and interface it with the 8086 microprocessor kit, and to write assembly language programs to operate the 8255 in Mode 0 (Simple I/O) and Mode 1 (Strobed I/O).

LEARNING OUTCOMES

By the end of this experiment, the student will be able to:

1. Understand the internal architecture of the 8255 PPI and the function of Ports A, B, and C.
2. Understand the concept and format of the Control Word Register (CWR) and compute control words for different modes.
3. Differentiate between Mode 0 (Basic Input/Output), Mode 1 (Strobed Input/Output), and Mode 2 (Bidirectional, not covered here).
4. Interface the 8255 with the 8086 kit and identify the port addresses used for Port A, B, C, and the Control Register.
5. Write 8086 assembly programs to configure the 8255 in Mode 0 and perform simple input/output operations.
6. Understand handshaking signals (STB, IBF, OBF, ACK, INTR) used in Mode 1 operation.
7. Analyze the difference in timing and control requirements between Mode 0 and Mode 1 operation.
8. Debug hardware-software interfacing problems related to port addressing and control word programming.

DESCRIPTION

The **8255 Programmable Peripheral Interface (PPI)** is a general-purpose I/O device designed to transfer data between the microprocessor and peripheral devices such as printers, keyboards, displays, stepper motors, ADC/DAC, etc. It has:

- **24 I/O pins** organized as three 8-bit ports: **Port A**, **Port B**, and **Port C**.
- **Port C** can be split into two 4-bit ports: **Port C upper (CU)** and **Port C lower (CL)**, often used for control/status signals in Mode 1 and Mode 2.
- Two internal groups: **Group A** (Port A + Port C upper) and **Group B** (Port B + Port C lower), each independently programmable.

Control Word Register (CWR):

D7	D6 D5	D4	D3	D2	D1	D0
1=Active(Mode Set Flag)	Mode select Group A (00=Mode0,01=Mode1,1X=Mode2)	Port A (1=Input, 0=Output)	Port C Upper (1=Input,0=Output)	Mode select Group B (0=Mode0,1=Mode1)	Port B (1=Input,0=Output)	Port C Lower (1=Input,0=Output)

Mode 0 (Basic I/O):

- Ports A, B, C (upper and lower) function as simple latched input or output ports.
- No handshaking; suitable for simple data transfer applications.

Mode 1 (Strobed I/O):

- Ports A and B function as input or output ports with handshaking.
- Port C lines are used as control/status signals: **STB** (Strobe, input control), **IBF** (Input Buffer Full), **OBF** (Output Buffer Full), **ACK** (Acknowledge), **INTR** (Interrupt Request).
- Used where synchronization between the microprocessor and peripheral is required (e.g., keyboard input, printer output).

ALGORITHM**(a) Mode 0 – Simple Output (Port A as output, Port B as input)**

1. Compute and load the control word for Mode 0 with Port A = Output, Port B = Input.
2. Output the control word to the Control Register address.
3. Read data from Port B (input device/switches).
4. Send/move the data to Port A (output device/LEDs).
5. Repeat steps 3–4 continuously (or as required).
6. Stop.

(b) Mode 1 – Strobed Input (Port A as input with handshaking)

1. Compute and load the control word for Mode 1 with Port A = Input.
2. Output the control word to the Control Register.
3. Read the status word from Port C to check the IBF (Input Buffer Full) flag.
4. If IBF = 1, data is ready; read data from Port A.
5. Store/display/process the data received.
6. Repeat as required.
7. Stop.

(c) Mode 1 – Strobed Output (Port A as output with handshaking)

1. Compute and load the control word for Mode 1 with Port A = Output.
2. Output the control word to the Control Register.
3. Load data to be sent into Port A.

4. Check the OBF (Output Buffer Full) / ACK status via Port C.
5. Once the peripheral acknowledges (ACK), send the next data byte.
6. Repeat until all data is transferred.
7. Stop.

PROGRAM

Assumed Port Addresses (typical for 8086 trainer kits):

- Port A = C0H
- Port B = C2H
- Port C = C4H
- Control Register = C6H

(a) Mode 0 – Read switches on Port B, display on Port A (LEDs)

Address	Mnemonic	Comments
1000	MOV AL, 90H	Control word: Port A=Output, Port B=Input, Mode 0
1002	OUT C6, AL	Load control word to CWR
1004	IN AL, C2	Read data from Port B (input)
1006	OUT C0, AL	Send data to Port A (output)
1008	JMP 1004	Repeat continuously

(Control word 90H = 1001 0000: D7=1 active, Group A Mode0, Port A=Output(0)... adjust bits per exact port assignment used in your lab manual.)

(b) Mode 1 – Strobed Input on Port A

Address	Mnemonic	Comments
1000	MOV AL, B4H	Control word: Port A = Mode 1, Input
1002	OUT C6, AL	Load control word
1004	IN AL, C4	Read Port C (status)

1006	AND AL, 20H	Mask to check IBF (bit D5)
1008	JZ 1004	Wait until IBF = 1 (data ready)
100A	IN AL, C0	Read data from Port A
100C	OUT C2, AL	Display data on Port B (LEDs)
100E	JMP 1004	Repeat

(Exact control word and status bit positions must be confirmed against your kit's 8255 datasheet/manual, since bit assignment for IBF/OBF varies slightly with Group A/B configuration.)

PROCEDURE

1. Connect the 8255 interfacing card/trainer to the 8086 microprocessor kit.
2. Identify and note the port addresses (Port A, B, C, Control Register) as per the kit's memory/IO map.
3. Enter the program using the keyboard monitor or an assembler/editor provided with the kit.
4. Assemble/verify the machine code and enter it into memory starting from the given address.
5. Connect switches to the input port and LEDs (or another output device) to the output port as required by the mode.
6. Execute the program in single-step or continuous run mode.
7. Observe the output response corresponding to the input applied.
8. For Mode 1, verify the handshaking signals (STB, IBF, ACK, OBF) using an LED/CRO if available.
9. Note the results and verify correct operation for both Mode 0 and Mode 1 configurations.

OUTPUT

- **Mode 0:** The LEDs connected to Port A reflect the switch positions given at Port B instantaneously (simple I/O, no handshaking delay).
- **Mode 1:** Data is transferred to/from Port A only when the corresponding handshake signal (IBF/OBF/ACK) is active, demonstrating synchronized/strobed data transfer.

MODEL VIVA QUESTIONS

Architecture & Theory

1. What is the 8255 PPI? Why is it called "programmable"?
2. List the three ports of 8255 and their possible groupings.
3. What is the function of Port C when the 8255 operates in Mode 1?
4. Explain the format of the Control Word Register (CWR) with a bit diagram.
5. What is the difference between Mode 0, Mode 1, and Mode 2 of 8255?
6. What is the "BSR mode" (Bit Set/Reset mode) of the 8255? How is it different from I/O modes?
7. What are STB, IBF, OBF, ACK, and INTR signals? Explain their role in Mode 1.
8. Why is Mode 0 called "Basic I/O" and Mode 1 called "Strobed I/O"?
9. Can Port A and Port B be configured in different modes simultaneously? Explain Group A and Group B.
10. What is the significance of D7 (Mode Set Flag) in the control word?

Interfacing & Addressing

11. How many address lines are needed to select the internal registers of 8255? Why?
12. How are the port addresses (Port A, B, C, Control Register) determined in an 8086 system?
13. What type of decoding (absolute/partial) is typically used to interface 8255 with 8086?
14. Why must the control word be written to the Control Register before any data transfer?

Program-Specific

15. What is the control word used in your Mode 0 program, and how did you compute it?
16. In the Mode 1 program, which bit of Port C indicates IBF, and why do we mask and check it?
17. Why is a polling loop used to check the status bit before reading/writing data in Mode 1?
18. How would you modify the program to use interrupts instead of polling for handshaking?
19. What happens if the control word is not loaded before attempting a data transfer?
20. How would you configure 8255 for Port A = Output, Port B = Input, both in Mode 0? Compute the control word.

Analytical

21. What are the real-world applications of Mode 1 strobed I/O (e.g., keyboard interfacing, printer interfacing)?
22. What is the advantage of Mode 1 over Mode 0 in terms of data integrity?
23. Why can't Mode 2 be used on Port B? (Mode 2 is only available for Port A)
24. How would you verify experimentally that the IBF/OBF handshaking is functioning correctly?

RESULT

The 8255 Programmable Peripheral Interface was successfully interfaced with the 8086 microprocessor kit, and its operation in Mode 0 (Simple I/O) and Mode 1 (Strobed I/O) was studied and verified through assembly language programs.

EXP:10**INTERFACING WITH 8279 (Rolling Message, 2-Key Lockout, N-Key Rollover)****AIM**

To study the architecture and operating modes of the Programmable Keyboard/Display Interface 8279, interface it with the 8086 microprocessor kit, and write assembly language programs to:

- (a) Display a rolling (scrolling) message on the seven-segment display, and
- (b) Implement keyboard scanning using 2-Key Lockout mode and N-Key Rollover mode.

LEARNING OUTCOMES

By the end of this experiment, the student will be able to:

1. Understand the internal architecture of the 8279 Keyboard/Display Controller and its dual role in display refreshing and keyboard scanning.
2. Understand the Command Word format used to configure the 8279 (Keyboard/Display mode, Clock, FIFO, etc.).
3. Interface the 8279 with a multiplexed seven-segment display and a matrix keyboard on the 8086 kit.
4. Write and execute a program to display a scrolling/rolling message using the display RAM of 8279.
5. Differentiate between 2-Key Lockout mode and N-Key Rollover mode of keyboard scanning.
6. Understand how key debouncing and simultaneous key-press handling are managed internally by the 8279.
7. Analyze the FIFO/Sensor RAM status and read keyboard data through the 8279.
8. Debug hardware-software interfacing issues related to display refresh timing and keyboard scan codes.

DESCRIPTION

The **8279** is a programmable **Keyboard/Display Interface** chip that off-loads the microprocessor from the repetitive tasks of scanning a keyboard and refreshing a display. It supports:

- **Keyboard section:** Scans an 8×8 (or smaller) keyboard matrix, debounces keys, and stores key codes in an internal 8-byte FIFO RAM. Two special modes handle simultaneous key presses:
 - **2-Key Lockout Mode:** If two keys are pressed at nearly the same time, only the first detected valid key is recognized; the second is locked out (ignored) until the first is released. This prevents ambiguous/rollover entries and is common in calculator-type keypads.
 - **N-Key Rollover Mode:** Allows any number of keys to be pressed simultaneously (in any order), and each valid key closure is entered into the FIFO in the order it was pressed/released — used in typewriter/computer-style keyboards where fast typing may cause overlapping key presses.
- **Display section:** Supports up to 16-character (8-bit) or 16-character (7-segment ×2) multiplexed LED displays via a 16×8 internal Display RAM, refreshed automatically by

internal scan logic — the microprocessor only needs to write display codes to this RAM once; the 8279 handles refresh.

Command Words (written to Control/Command register) commonly used:

- **Mode Set Command** — selects display mode (8/16-character, left/right entry) and keyboard mode (Encoded/Decoded scan, 2-key lockout / N-key rollover).
- **Clock Command** — sets the internal clock prescaler for correct scan frequency.
- **Read FIFO / Write Display Command** — used to access keyboard data or write display data.

Rolling Message Display: A message longer than the display width is stored in memory; the program repeatedly shifts the message string across the display window with a small delay between shifts, giving the visual effect of the text "scrolling."

ALGORITHM

(a) Rolling Message Display

1. Initialize the 8279 with the Mode Set command (e.g., 8-character display, left entry, encoded scan, 2-key lockout).
2. Load the Clock Command Word to set the internal scan frequency.
3. Store the message (with leading/trailing blanks for smooth scroll) in a memory buffer.
4. Initialize a pointer to the start of the message buffer.
5. Take the display-window-length (e.g., 8 characters) of data starting from the pointer, convert to seven-segment codes, and write to the Display RAM via the FIFO/Display Write command.
6. Call a delay subroutine for visible scroll speed.
7. Increment the pointer by one character.
8. Repeat steps 5–7 until the entire message has scrolled through the display.
9. Reset pointer to the beginning and repeat (loop continuously) or stop after one pass.

(b) 2-Key Lockout Mode

1. Initialize 8279 with Mode Set command specifying 2-Key Lockout keyboard mode.
2. Continuously monitor the FIFO status (via Read FIFO Status command) to check if a key entry is available.
3. If FIFO is not empty, read the key code (Read FIFO command).
4. Decode the key code to the corresponding key value.
5. Display the key value on the seven-segment display.
6. If two keys are pressed simultaneously, verify only the first detected key is registered (second is locked out) — observe behavior.
7. Repeat steps 2–6 continuously.

(c) N-Key Rollover Mode

1. Initialize 8279 with Mode Set command specifying N-Key Rollover keyboard mode.
2. Continuously monitor the FIFO status.
3. Whenever a key is pressed/released, read the resulting code from FIFO (each key generates a make/break code independent of other keys).
4. Decode and display/store each key code in the order received.

5. Verify that multiple simultaneously pressed keys are all correctly registered (no lockout).
6. Repeat continuously.

PROGRAM

Assumed Port Addresses (typical trainer kit):

- 8279 Data Port = C0H
- 8279 Command/Status Port = C2H

(a) Rolling Message Display (Skeleton)

Address	Mnemonic	Comments
1000	MOV AL, 00H	Mode Set: 8-char display, left entry, encoded scan, 2-key lockout
1002	OUT C2, AL	Write command to 8279
1004	MOV AL, 31H	Clock command word (prescaler value)
1006	OUT C2, AL	Write clock command
1008	MOV SI, 2000H	SI → start of message buffer in memory
100B	MOV DI, 00	DI = window offset / character count
100D	MOV CL, 08	CL = number of visible display positions
100F	MOV AL, 90H	Write Display RAM command (start at addr 0)
1011	OUT C2, AL	Send command
1013	MOV AL, [SI+DI]	Get character from buffer
1016	(Lookup 7-seg code)	Convert ASCII/char to 7-segment pattern
—	OUT C0, AL	Send segment code to Data Port

—	INC DI	Move to next character in window
—	LOOP 1013	Repeat for 8 positions
—	CALL DELAY	Delay for visible scroll
—	INC SI	Shift buffer start by 1 (scroll effect)
—	CMP SI, END_ADDR	Check if end of message reached
—	JNZ 100B	Repeat scroll
—	JMP 1008	Restart message (continuous loop)

(This is a structural skeleton — the exact 7-segment lookup table and opcodes must be entered as per your kit's monitor program and assembler, since actual hex codes vary by trainer.)

(b) 2-Key Lockout Key Read (Skeleton)

Address	Mnemonic	Comments
1000	MOV AL, 00H	Mode Set: Encoded scan, 2-key lockout
1002	OUT C2, AL	Write command
1004	IN AL, C2	Read status register
1006	AND AL, 07H	Mask FIFO status bits (check if data available)
1008	JZ 1004	Wait until key data ready
100A	IN AL, C0	Read key code from FIFO (Data Port)
100C	(Decode routine)	Convert scan code to display code

—	OUT C0, AL	Display key value on 7-segment
—	JMP 1004	Repeat

(c) N-Key Rollover Key Read (Skeleton)

Address	Mnemonic	Comments
1000	MOV AL, 08H	Mode Set: Encoded scan, N-key rollover
1002	OUT C2, AL	Write command
1004	IN AL, C2	Read status register
1006	AND AL, 07H	Check FIFO status
1008	JZ 1004	Wait for key data
100A	IN AL, C0	Read key code
100C	(Decode & Store)	Store/display each key code as it arrives
—	JMP 1004	Repeat continuously

PROCEDURE

1. Connect the 8279 keyboard/display interfacing module to the 8086 microprocessor kit.
2. Identify the Data Port and Command/Status Port addresses from the kit manual.
3. Enter the Mode Set and Clock command program to initialize the 8279.
4. For **rolling message**: store the desired message string (with padding blanks) in memory, enter the display program, and run it — observe the message scrolling across the seven-segment display.
5. For **2-key lockout**: enter the keyboard-read program with the 2-key lockout mode set, then press single and simultaneous keys on the connected keypad; observe that only one key registers when two are pressed together.
6. For **N-key rollover**: change the mode set command to N-key rollover, re-run the program, press multiple keys simultaneously, and verify all key presses are correctly captured in sequence.
7. Note the display outputs in each case and record observations.

OUTPUT

- **Rolling Message:** The programmed text message scrolls continuously (left to right or right to left) across the seven-segment display array.
- **2-Key Lockout:** When two keys are pressed simultaneously, only the first key press is recognized and displayed; the second is ignored until the first key is released.
- **N-Key Rollover:** When multiple keys are pressed simultaneously or in quick succession, all key presses are correctly detected and displayed/stored in the order of activation, with no key ignored.

MODEL VIVA QUESTIONS

Architecture & Theory

1. What is the 8279? What are its two main functional sections?
2. Draw and explain the block diagram of the 8279.
3. What is the size of the internal FIFO/Sensor RAM in 8279, and what is it used for?
4. What is the size of the internal Display RAM, and how many characters can it support?
5. Explain the format of the Mode Set Command Word.
6. What is the purpose of the Clock Command Word?
7. What is the difference between Encoded and Decoded keyboard scanning?
8. What is meant by "key debouncing," and how does 8279 handle it internally?

2-Key Lockout vs N-Key Rollover

9. What is 2-Key Lockout mode? Give a practical example of its use.
10. What is N-Key Rollover mode? How does it differ functionally from 2-key lockout?
11. Why would a typewriter/computer keyboard prefer N-key rollover over 2-key lockout?
12. What happens internally in the FIFO when two keys are pressed simultaneously in 2-key lockout mode?
13. How does the 8279 determine the order of key codes stored in the FIFO in N-key rollover mode?

Display/Rolling Message

14. How does the 8279 refresh a multiplexed display without continuous CPU intervention?
15. What is meant by "left entry" and "right entry" display modes?
16. How is the scrolling/rolling effect achieved in the message display program?
17. What is the role of the delay subroutine in the rolling message program?
18. What would happen if the delay between shifts were too small or too large?

Interfacing & Programming

19. What are the Data Port and Command/Status Port addresses used in your program, and how are they decoded?
20. How do you check whether the FIFO has valid key data ready to be read?
21. Why must the Mode Set command be issued before any data transfer, similar to 8255?
22. How would you modify the program to display the rolling message in the reverse direction?
23. What scan codes are generated for the keyboard matrix, and how are they converted to actual key values?

Analytical

24. What are real-life applications of the 8279 (e.g., calculators, embedded control panels, industrial keypads)?
25. Compare 8279-based keyboard scanning with software-pollled keyboard scanning in terms of CPU overhead.
26. Why is the 8279 called a "programmable" keyboard/display interface?

RESULT

The 8279 Programmable Keyboard/Display Interface was successfully interfaced with the 8086 microprocessor kit. A rolling message display was implemented on the seven-segment display, and keyboard scanning was successfully verified in both 2-Key Lockout mode and N-Key Rollover mode.

EXP.NO.11**INTERFACING WITH DIGITAL-TO-ANALOG CONVERTER (DAC)****AIM**

To interface a Digital-to-Analog Converter (DAC 0800/DAC 0808) with the 8086 microprocessor kit and write assembly language programs to generate different waveforms (square wave, triangular wave, sawtooth wave, and staircase waveform) at the DAC output.

LEARNING OUTCOMES

By the end of this experiment, the student will be able to:

1. Understand the basic principle of digital-to-analog conversion and the working of an R-2R ladder / weighted resistor type DAC.
2. Understand the interfacing of a DAC with the 8086 microprocessor through a peripheral interface (typically 8255).
3. Write 8086 assembly language programs to generate various analog waveforms by sending appropriate digital data sequences to the DAC.
4. Understand the relationship between digital input codes, the reference voltage, and the resulting analog output voltage.
5. Analyze the effect of software delay on the frequency of the generated waveform.
6. Observe and interpret waveforms generated on a CRO (Cathode Ray Oscilloscope) and correlate them with the program logic.
7. Debug hardware-software interfacing issues related to port addressing, data range, and timing in DAC-based waveform generation.

DESCRIPTION

A Digital-to-Analog Converter (DAC) converts a digital (binary) input into a proportional analog voltage or current output. A commonly used IC is the DAC 0800 (8-bit DAC), which produces an analog output current/voltage proportional to the 8-bit digital input applied to it, given by:

$$V_{\text{out}} = (D / 256) \times V_{\text{ref}}$$

where D is the decimal equivalent of the 8-bit digital input (0–255) and V_{ref} is the reference voltage.

Since the 8086 microprocessor cannot directly drive the DAC's parallel input lines (due to loading and timing constraints), the DAC is interfaced through a parallel peripheral interface such as the 8255, configured in Mode 0 (Simple output). Port A (or any assigned port) of the 8255 is connected to the 8-bit digital input of the DAC, and an operational amplifier (op-amp) stage is typically used at the DAC output to convert the current output into a usable voltage and buffer it.

By sending different digital data sequences (successive numbers) to the DAC at controlled time intervals, various standard waveforms can be generated:

- Square Wave — alternating between two fixed digital values (e.g., 00H and FFH) at regular intervals.

- Triangular Wave — digital value incremented from 00H to FFH, then decremented back to 00H, repeatedly.
- Sawtooth Wave — digital value incremented from 00H to FFH, then reset abruptly back to 00H, and repeated.
- Staircase Waveform — digital value incremented in fixed steps (not by 1) with a delay at each step, producing a stepped rising waveform.

ALGORITHM

(a) Square Wave Generation

1. Initialize the 8255 in Mode 0 with the port connected to DAC as output.
2. Send digital value 00H to the DAC port.
3. Call delay subroutine.
4. Send digital value FFH to the DAC port.
5. Call delay subroutine.
6. Repeat steps 2–5 continuously.

(b) Triangular Wave Generation

1. Initialize the 8255 in Mode 0 with output port connected to DAC.
2. Initialize a register with 00H.
3. Send the register value to the DAC port.
4. Increment the register value.
5. If the value has not reached FFH, repeat from step 3.
6. Once FFH is reached, decrement the register value.
7. Send the value to the DAC port.
8. If the value has not reached 00H, repeat from step 6.
9. Once 00H is reached, repeat from step 2 (continuous loop).

(c) Sawtooth Wave Generation

1. Initialize the 8255 in Mode 0 with output port connected to DAC.
2. Initialize a register with 00H.
3. Send the register value to the DAC port.
4. Increment the register value.
5. If the value has not reached FFH (or overflowed), repeat from step 3.
6. Once maximum value is reached, reset register to 00H and repeat from step 2.

(d) Staircase Waveform Generation

1. Initialize the 8255 in Mode 0 with output port connected to DAC.
2. Initialize a register with 00H and a fixed step size (e.g., 10H).
3. Send the register value to the DAC port.
4. Call delay subroutine (to hold each step visibly).
5. Add the step size to the register value.
6. If the value has not exceeded FFH, repeat from step 3.
7. Once maximum is reached, reset the register to 00H and repeat from step 2.

PROGRAM**Assumed Port Addresses (via 8255 interface):**

- **Port A (connected to DAC input) = C0H**
- **Control Register = C6H**

(a) Square Wave

Address	Mnemonic	Comments
1000	MOV AL, 80H	Control word: Port A = Output, Mode 0
1002	OUT C6, AL	Initialize 8255
1004	MOV AL, 00H	Load minimum value
1006	OUT C0, AL	Send to DAC
1008	CALL DELAY	Delay for half period
100B	MOV AL, FFH	Load maximum value
100D	OUT C0, AL	Send to DAC
100F	CALL DELAY	Delay for half period
1012	JMP 1004	Repeat

(b) Triangular Wave

Address	Mnemonic	Comments
1000	MOV AL, 80H	Control word
1002	OUT C6, AL	Initialize 8255
1004	MOV BL, 00H	Initialize counter

1006	MOV AL, BL	Load count into AL
1008	OUT C0, AL	Send to DAC (rising edge)
100A	INC BL	Increment counter
100C	JNZ 1006	Repeat until BL overflows (FF→00)
100E	MOV BL, FFH	Start descending
1010	MOV AL, BL	Load count into AL
1012	OUT C0, AL	Send to DAC (falling edge)
1014	DEC BL	Decrement counter
1016	JNZ 1010	Repeat until BL = 0
1018	JMP 1004	Repeat entire cycle

(c) Sawtooth Wave

Address	Mnemonic	Comments
1000	MOV AL, 80H	Control word
1002	OUT C6, AL	Initialize 8255
1004	MOV BL, 00H	Initialize counter
1006	MOV AL, BL	Load count
1008	OUT C0, AL	Send to DAC
100A	INC BL	Increment

100C	JNZ 1006	Repeat until overflow (rising ramp)
100E	JMP 1004	Reset and repeat (sharp fall)

(d) Staircase Waveform

Address	Mnemonic	Comments
1000	MOV AL, 80H	Control word
1002	OUT C6, AL	Initialize 8255
1004	MOV BL, 00H	Initialize counter
1006	MOV AL, BL	Load count
1008	OUT C0, AL	Send to DAC
100A	CALL DELAY	Hold at this step
100D	ADD BL, 10H	Increment by step size
1010	JNZ 1006	Repeat until overflow
1012	JMP 1004	Reset and repeat

(A simple DELAY subroutine using a register loop, similar to previous experiments, can be added at the end of each program.)

PROCEDURE

1. Connect the DAC 0800/0808 module to the 8255 interfacing card, which is in turn connected to the 8086 microprocessor kit.
2. Connect the output of the DAC (through the op-amp buffer stage, if provided) to a CRO.
3. Identify the port addresses for Port A and the Control Register from the kit manual.
4. Enter and assemble the desired waveform generation program (square/triangular/sawtooth/staircase).
5. Execute the program in continuous run mode.
6. Observe the resulting waveform on the CRO screen.
7. Vary the delay value in the program and observe the corresponding change in waveform

frequency.

8. Measure the amplitude and time period of the generated waveform using the CRO and verify against expected theoretical values.
9. Repeat for each waveform type and record observations.

OUTPUT

- Square Wave: A waveform alternating between minimum (0V) and maximum (V_{ref}) voltage levels at a period determined by the delay value.
- Triangular Wave: A waveform that rises linearly to maximum and falls linearly back to minimum, repeating continuously.
- Sawtooth Wave: A waveform that rises linearly to maximum, then drops sharply to minimum, repeating continuously.
- Staircase Waveform: A step-like rising waveform, incrementing in fixed voltage steps with a hold at each level, then resetting.

MODEL VIVA QUESTIONS

DAC Theory

1. What is a DAC? Explain its basic principle of operation.
2. What is the resolution of an 8-bit DAC? Calculate the step size for a reference voltage of 5V.
3. Explain the working of an R-2R ladder network based DAC.
4. What is the formula relating digital input, reference voltage, and analog output voltage of a DAC?
5. What is settling time in a DAC? Why is it important?
6. Differentiate between a DAC and an ADC.
7. What is the purpose of the op-amp stage typically used at the DAC's current output?

Interfacing

8. Why is the DAC not connected directly to the microprocessor's data bus? What interfacing device is used, and why?
9. In what mode is the 8255 configured for DAC interfacing, and why?
10. What is the control word used to configure Port A as output in Mode 0? How is it computed?
11. What is the significance of the reference voltage (V_{ref}) in DAC operation?

Waveform Generation / Program-Specific

12. How is a square wave generated using a DAC? What determines its frequency?
13. Explain how the triangular waveform program produces a linear rise and fall.
14. What is the difference in program logic between a triangular wave and a sawtooth wave?
15. How does the staircase waveform program differ from the ramp (sawtooth) waveform program?
16. What happens to the waveform frequency if the delay subroutine's count is increased?
17. What happens if the increment step size in the staircase program is changed from 10H to 20H? How does it affect the waveform?
18. Why does the digital value "wrap around" (overflow) in the increment loop, and how is this used to detect a full cycle?
19. How would you generate a variable duty cycle square wave? What program modification would be needed?

Analytical

20. What are the real-world applications of a DAC (e.g., audio systems, motor speed control, signal generation, sensor actuation)?
21. How would you measure the actual output voltage step size on the CRO and verify it matches the theoretical calculation?
22. What is the effect of noise or glitches during the DAC's code transition, and how can it be minimized (deglitcher circuit)?
23. If the CRO shows a distorted or non-linear triangular wave, what could be the possible hardware/software causes?

RESULT

The Digital-to-Analog Converter (DAC 0800/0808) was successfully interfaced with the 8086 microprocessor kit through the 8255 PPI, and various waveforms — square, triangular, sawtooth, and staircase — were generated and verified on the CRO.

EXP.NO.12**FAMILIARIZATION OF 8051 TRAINER KIT BY EXECUTING SIMPLE ASSEMBLY LANGUAGE PROGRAMS****(DECIMAL ARITHMETIC AND BIT MANIPULATION)****AIM**

To familiarize with the 8051 microcontroller trainer kit and to write, assemble, and execute simple Assembly Language Programs (ALP) for:

1. Decimal (BCD) arithmetic operations (addition/subtraction using DA A instruction)
2. Bit manipulation operations (set, clear, complement, test individual bits)

using the 8051 trainer kit / keyboard monitor commands.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

1. Identify and operate the various components of an 8051 trainer kit (keypad, display, ports, reset, single-step, execute keys).
2. Enter, edit, and execute machine code / assembly programs using the kit's monitor program.
3. Understand BCD (Binary Coded Decimal) number representation and the need for decimal adjustment after arithmetic operations.
4. Use the DA A (Decimal Adjust Accumulator) instruction correctly following ADD/ADDC operations.
5. Perform bit-level operations using bit-addressable memory and instructions such as SETB, CLR, CPL, JB, JNB, MOV (bit).
6. Verify program execution by observing register/memory contents on the kit's display and correlate results with manual calculations.

DESCRIPTION**8051 Trainer Kit**

The 8051 trainer kit consists of an 8051/8031 microcontroller, a hex keypad for data/address entry, a 6/7-segment LED (or LCD) display for viewing addresses and data, and a monitor program (firmware) that allows the user to:

- Enter a starting address and view/modify memory contents
- Enter opcodes byte-by-byte into RAM
- Execute programs (RUN) or execute step-by-step (SINGLE STEP) for debugging
- View the contents of the accumulator, registers, and memory after execution

Decimal Arithmetic (BCD)

The 8051 accumulator performs binary addition. When two BCD numbers are added using ADD/ADDC, the result may not be a valid BCD digit (if a nibble exceeds 9). The DA A instruction, executed immediately after ADD, corrects the result back into valid packed BCD form by adding 6 to a nibble if it is greater than 9 or if an auxiliary/carry occurred.

Bit Manipulation

The 8051 has a bit-addressable memory area (RAM bytes 20H–2FH, bit addresses 00H–7FH) and bit-addressable SFRs (e.g., P1, P2, PSW, ACC when bit-addressed). Instructions like SETB, CLR, CPL operate directly on a single bit without disturbing other bits in the byte — useful for controlling/testing individual I/O lines (e.g., LEDs, switches, relays) without affecting the rest of the port.

ALGORITHM

(A) Decimal (BCD) Addition

1. Start.
2. Load the first BCD number into the accumulator (A).
3. Add the second BCD number to A using ADD instruction.
4. Apply DA A to adjust the result into valid BCD form.
5. Store the result (and carry, if any, indicating a 3-digit result) into memory.
6. Stop.

(B) Bit Manipulation (Example: Toggle/Test a bit of Port 1 based on input switch)

1. Start.
2. Read the input bit (e.g., P1.0 connected to a switch).
3. Test the bit using JB/JNB instruction.
4. If the bit is set (1), set the output bit (e.g., P1.7) using SETB.
5. If the bit is clear (0), clear the output bit using CLR.
6. Repeat continuously (loop back) or stop after one check, as required.
7. Stop.

PROGRAM

(A) BCD Addition Program

Assume:

- First BCD number stored at memory location 40H
- Second BCD number stored at memory location 41H
- Result to be stored at memory location 42H (and carry flag noted separately)

Address	Label	Mnemonic	Opcode	Comments
4100		MOV A, 40H	E5 40	Load first BCD number into A
4102		ADD A, 41H	25 41	Add second BCD number
4104		DA A	D4	Decimal adjust accumulator
4105		MOV 42H, A	F5 42	Store BCD result
4107		MOV 43H, C	(via MOV C / MOV bit)	Store carry (if 3-digit result)
4109		SJMP \$	80 FE	Stop (halt / infinite loop)

Example:

- (40H) = 47H, (41H) = 25H
- $47 + 25 = 6C$ (binary add result) $\rightarrow$ after DA A $\rightarrow$ 72H (correct BCD sum of $47 + 25 = 72$)
- Result: (42H) = 72H, Carry = 0

(B) Bit Manipulation Program

Assume switch connected to P1.0, LED connected to P1.7.

Address	Label	Mnemonic	Opcode	Comments
4200	START	SETB P1.0	D2 90	Configure P1.0 as input
4202	LOOP	JB P1.0, ON	20 90 xx	Jump if switch pressed (bit=1)

4205		CLR P1.7	C2 97	Turn OFF LED
4207		SJMP LOOP	80 F9	Repeat
4209	ON	SETB P1.7	D2 97	Turn ON LED
420B		SJMP LOOP	80 F5	Repeat

Simple Bit-Toggle Program (alternative):

Address	Mnemonic	Comments
4300	MOV A, #55H	Load pattern
4302	MOV P1, A	Output pattern to Port 1
4304	CPL P1.3	Complement bit P1.3 only
4306	SETB P1.4	Set bit P1.4
4308	CLR P1.5	Clear bit P1.5
430A	SJMP \$	Halt

PROCEDURE

1. Switch ON the 8051 trainer kit and observe the display shows the monitor prompt (usually address–data format).
2. Press RESET to initialize the kit.
3. Using the keypad, select the "MEM" or address-entry mode and enter the starting address (e.g., 4100H) where the program is to be stored.
4. Enter the opcode bytes of the assembled program one by one, pressing the appropriate key (NEXT/INCREMENT) to move to the next memory location after each byte entry.
5. Before running the BCD program, enter the sample data bytes (e.g., 47H at 40H and 25H at 41H) using the same memory-entry method.
6. After entering the complete program, verify the entered opcodes by displaying the memory contents from the starting address again.
7. Enter the starting address of the program in the "GO"/"EXECUTE" register/prompt.
8. Press RUN (or SINGLE STEP for step-by-step verification) to execute the program.
9. For the bit manipulation program, connect switches/LEDs to the specified port pins (P1.0 and P1.7) as per the kit's I/O connector.
10. Observe the result:
 - For BCD addition: display the memory location (42H) to view the result.
 - For bit manipulation: toggle the switch and observe the LED response.
11. Cross-check the result obtained on the kit with the manually calculated result.
12. Note down the observations and switch OFF the kit after completing the experiment.

MODEL VIVA QUESTIONS

1. What is the function of the DA A instruction? Why is it required after BCD addition?
2. What is the difference between packed BCD and unpacked BCD?
3. Can DA A be used after subtraction? Why or why not?
4. Which flags are affected by the DA A instruction?
5. What is the bit-addressable memory range in the internal RAM of 8051?
6. Name any four bit-addressable SFRs of the 8051.
7. What is the difference between SETB, CLR, and CPL instructions?
8. How does JB differ from JNB? Give the opcode format.
9. Why can't you perform bit operations on Port 3 pins that are used for external memory access in certain modes?
10. What is the purpose of the "SINGLE STEP" feature on a trainer kit?
11. How is a byte address converted to its corresponding bit address (e.g., byte 20H → bit addresses)?
12. What are the components of an 8051 trainer kit and their functions?
13. What is the maximum value a single BCD digit can hold, and what happens if binary addition exceeds it?
14. Differentiate between MOVX and MOV instructions.
15. What is the significance of the carry flag in multi-byte BCD addition?

RESULT

The 8051 trainer kit was successfully familiarized, and the Assembly Language Programs for:

1. **Decimal (BCD) Arithmetic** — verified using the DA A instruction, correctly adjusting the binary addition result into valid BCD form (e.g., $47H + 25H \rightarrow 72H$).
2. **Bit Manipulation** — verified by testing and controlling individual bits of Port 1 using SETB, CLR, and JB/JNB instructions, confirming correct LED response to switch input.

were entered, assembled, executed, and verified successfully, and the outputs matched the expected/manually calculated results.

SYSTEM SOFTWARE

EXP NO :1**A) SIMULATION OF FCFS CPU SCHEDULING ALGORITHM****AIM**

To simulate the First Come First Served (FCFS) non-preemptive CPU scheduling algorithm using a C program, and to compute the Completion Time, Turnaround Time and Waiting Time for a given set of processes.

LEARNING OUTCOME

- Understand the concept of CPU scheduling and the role of the operating system scheduler.
- Learn the working principle of the FCFS non-preemptive scheduling algorithm.
- Develop the ability to compute Completion Time (CT), Turnaround Time (TAT) and Waiting Time (WT) for a set of processes.
- Gain skill in implementing scheduling algorithms using the C programming language.
- Analyse the performance of FCFS scheduling in terms of average waiting time and turnaround time.

DESCRIPTION

First Come First Served (FCFS) is the simplest CPU scheduling algorithm. In this scheme, the process that arrives first in the ready queue is allocated the CPU first, and processing continues in the strict order of arrival, exactly like a queue at a ticket counter. FCFS is a non-preemptive algorithm, which means that once a process is allocated the CPU, it holds the CPU until it completes its execution; it cannot be interrupted by any other process, regardless of the burst time or priority of the newly arrived process.

FCFS is easy to understand and implement using a simple FIFO (First In First Out) queue. However, its average waiting time is often quite long, and it suffers from the convoy effect, where a short process gets stuck behind a long process, causing all processes behind it to wait longer than necessary.

ALGORITHM

1. Start the program.
2. Read the number of processes n.
3. For each process, read its Arrival Time (AT) and Burst Time (BT).

4. Sort the processes in ascending order of Arrival Time.
5. Calculate the Completion Time (CT) of the first process as $AT[0] + BT[0]$.
6. For each subsequent process i , calculate $CT[i]$ as: if $CT[i-1] > AT[i]$, then $CT[i] = CT[i-1] + BT[i]$; else $CT[i] = AT[i] + BT[i]$.
7. Calculate Turnaround Time: $TAT[i] = CT[i] - AT[i]$.
8. Calculate Waiting Time: $WT[i] = TAT[i] - BT[i]$.
9. Calculate the average Turnaround Time and average Waiting Time.
10. Display the Process ID, AT, BT, CT, TAT and WT of each process along with the averages.
11. Stop the program.

PROGRAM

```
#include <stdio.h>

int main()
{
    int n, i, j, temp;
    int at[20], bt[20], ct[20], tat[20], wt[20], p[20];
    float total_tat = 0, total_wt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        p[i] = i + 1;
        printf("Enter Arrival Time and Burst Time for P%d: ", p[i]);
        scanf("%d %d", &at[i], &bt[i]);
    }

    /* Sort processes by arrival time (FCFS order) */
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (at[j] > at[j + 1])
            {
                temp = at[j]; at[j] = at[j + 1]; at[j + 1] = temp;
                temp = bt[j]; bt[j] = bt[j + 1]; bt[j + 1] = temp;
                temp = p[j]; p[j] = p[j + 1]; p[j + 1] = temp;
            }
        }
    }
}
```

```
}

ct[0] = at[0] + bt[0];
for (i = 1; i < n; i++)
{
    if (ct[i - 1] > at[i])
        ct[i] = ct[i - 1] + bt[i];
    else
        ct[i] = at[i] + bt[i];
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for (i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_tat += tat[i];
    total_wt += wt[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("\nAverage Turnaround Time = %.2f", total_tat / n);
printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

return 0;
}
```

PROCEDURE

1. Open a text editor / IDE (e.g. Turbo C, Code::Blocks, or GCC on a terminal).
2. Type the C program shown above and save it with the extension .c (for example, fcfs.c).
3. Compile the program using the command: gcc fcfs.c -o fcfs
4. Run the executable using the command: ./fcfs (or fcfs.exe on Windows).
5. Enter the number of processes when prompted.
6. Enter the Arrival Time and Burst Time for each process.
7. Observe the output, which displays the Completion Time, Turnaround Time and Waiting Time for each process, along with the average Turnaround Time and average Waiting Time.
8. Verify the output manually using a Gantt chart for the same input values.

OUTPUT

Sample input: $n = 4$ processes with the following Arrival Time and Burst Time:

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

Average Turnaround Time = $(5 + 7 + 14 + 19) / 4 = 11.25$

Average Waiting Time = $(0 + 4 + 6 + 13) / 4 = 5.75$

MODEL VIVA QUESTIONS

1. What is CPU scheduling and why is it needed?
2. What is the difference between preemptive and non-preemptive scheduling?
3. Explain the FCFS scheduling algorithm with an example.
4. Is FCFS a preemptive or non-preemptive algorithm? Justify your answer.
5. Define Arrival Time, Burst Time, Completion Time, Turnaround Time and Waiting Time.

RESULT

The C program to simulate the First Come First Served (FCFS) non-preemptive CPU scheduling algorithm was written, executed, and verified successfully. The Completion Time, Turnaround Time and Waiting Time for the given set of processes were computed correctly, and the output was verified using a Gantt chart.

B) SIMULATION OF SJF CPU SCHEDULING ALGORITHM

AIM

To simulate the Shortest Job First (SJF) non-preemptive CPU scheduling algorithm using a C program, and to compute the Completion Time, Turnaround Time and Waiting Time for a given set of processes.

LEARNING OUTCOME

- Understand the concept of CPU scheduling and the criteria used to compare scheduling algorithms.
- Learn the working principle of the SJF non-preemptive scheduling algorithm.
- Develop the ability to compute Completion Time (CT), Turnaround Time (TAT) and Waiting Time (WT) for a set of processes.
- Gain skill in implementing scheduling algorithms that require dynamic selection of the next process using the C programming language.
- Compare the performance of SJF with FCFS in terms of average waiting time.
- Understand the concept of starvation in scheduling algorithms.

DESCRIPTION

Shortest Job First (SJF) is a CPU scheduling algorithm in which the process with the smallest burst time is selected for execution next from among the processes that have already arrived in the ready queue. In its non-preemptive form, once the CPU is allocated to a process, it continues to run until it completes, even if a process with a shorter burst time arrives in the meantime.

SJF is known to give the minimum average waiting time among all non-preemptive scheduling algorithms for a given set of processes, since shorter jobs are cleared out of the queue quickly. However, it requires advance knowledge of the burst time of each process, which is often not available in practice and must be estimated. SJF can also lead to starvation, where a process with a large burst time waits indefinitely if shorter processes keep arriving ahead of it.

ALGORITHM

1. Start the program.
2. Read the number of processes n.
3. For each process, read its Arrival Time (AT) and Burst Time (BT).

4. Set the current time to the arrival time of the first process to arrive, and mark all processes as not completed.
5. Among all the processes that have arrived ($\text{Arrival Time} \leq \text{current time}$) and are not yet completed, select the one with the smallest Burst Time.
6. If no process has arrived at the current time, advance the current time and repeat step 5.
7. Execute the selected process to completion (non-preemptive) and set its Completion Time (CT) = current time + Burst Time.
8. Update the current time to the Completion Time of the executed process, and mark it as completed.
9. Repeat steps 5 to 8 until all processes are completed.
10. Calculate Turnaround Time: $\text{TAT}[i] = \text{CT}[i] - \text{AT}[i]$.
11. Calculate Waiting Time: $\text{WT}[i] = \text{TAT}[i] - \text{BT}[i]$.
12. Calculate the average Turnaround Time and average Waiting Time.
13. Display the Process ID, AT, BT, CT, TAT and WT of each process along with the averages.
14. Stop the program.

PROGRAM

```
#include <stdio.h>

int main()
{
    int n, i, j, temp, done[20] = {0};
    int at[20], bt[20], ct[20], tat[20], wt[20], p[20];
    int current_time = 0, completed = 0, idx;
    float total_tat = 0, total_wt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        p[i] = i + 1;
        printf("Enter Arrival Time and Burst Time for P%d: ", p[i]);
        scanf("%d %d", &at[i], &bt[i]);
    }

    while (completed < n)
    {
        idx = -1;
        int min_bt = 9999;
```

```
/* Among processes that have arrived and are not done,
   pick the one with the smallest burst time */
for (i = 0; i < n; i++)
{
    if (!done[i] && at[i] <= current_time && bt[i] < min_bt)
    {
        min_bt = bt[i];
        idx = i;
    }
}

if (idx == -1)
{
    /* No process has arrived yet; advance the clock */
    current_time++;
    continue;
}

ct[idx] = current_time + bt[idx];
current_time = ct[idx];
done[idx] = 1;
completed++;
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for (i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_tat += tat[i];
    total_wt += wt[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
           p[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("\nAverage Turnaround Time = %.2f", total_tat / n);
printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

return 0;
}
```

PROCEDURE

1. Open a text editor / IDE (e.g. Turbo C, Code::Blocks, or GCC on a terminal).
2. Type the C program shown above and save it with the extension .c (for example, sjf.c).
3. Compile the program using the command: `gcc sjf.c -o sjf`
4. Run the executable using the command: `./sjf` (or `sjf.exe` on Windows).
5. Enter the number of processes when prompted.
6. Enter the Arrival Time and Burst Time for each process.
7. Observe the output, which displays the Completion Time, Turnaround Time and Waiting Time for each process, along with the average Turnaround Time and average Waiting Time.
8. Verify the output manually by selecting, at each decision point, the shortest job among the processes that have arrived, and draw the corresponding Gantt chart.

OUTPUT

Sample input: $n = 4$ processes with the following Arrival Time and Burst Time. Execution order obtained: $P1 \rightarrow P3 \rightarrow P2 \rightarrow P4$.

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time
P1	0	7	7	7	0
P2	1	4	12	11	7
P3	2	1	8	6	5
P4	3	4	16	13	9

Average Turnaround Time = $(7 + 11 + 6 + 13) / 4 = 9.25$

Average Waiting Time = $(0 + 7 + 5 + 9) / 4 = 5.25$

MODEL VIVA QUESTIONS

1. What is the SJF scheduling algorithm? Explain with an example.
2. Differentiate between preemptive and non-preemptive SJF.
3. Why is SJF also called Shortest Job Next (SJN)?
4. Why does SJF give the minimum average waiting time among non-preemptive algorithms?
5. What is the main practical limitation of implementing SJF in a real operating system?

RESULT

The C program to simulate the Shortest Job First (SJF) non-preemptive CPU scheduling algorithm was written, executed, and verified successfully. The Completion Time, Turnaround Time and Waiting Time for the given set of processes were computed correctly, and the output was verified using a Gantt chart.

C) SIMULATION OF ROUND ROBIN CPU SCHEDULING ALGORITHM

AIM

To simulate the Round Robin (RR) preemptive CPU scheduling algorithm using a C program, and to compute the Completion Time, Turnaround Time and Waiting Time for a given set of processes with a fixed time quantum.

LEARNING OUTCOME

- Understand the concept of preemptive CPU scheduling and time-sliced execution.
- Learn the working principle of the Round Robin scheduling algorithm and the role of the time quantum.
- Develop the ability to compute Completion Time (CT), Turnaround Time (TAT) and Waiting Time (WT) using a circular ready queue.
- Gain skill in implementing queue-based preemptive scheduling using the C programming language.
- Analyse the effect of time quantum size on average waiting time, turnaround time, and context-switch overhead.
- Compare Round Robin with non-preemptive algorithms such as FCFS and SJF.

DESCRIPTION

Round Robin (RR) is a preemptive CPU scheduling algorithm designed specifically for time-sharing systems. Each process in the ready queue is assigned a fixed unit of CPU time called a time quantum (or time slice). Processes are held in a circular queue; the CPU picks the process at the front of the queue and runs it for at most one time quantum. If the process does not finish within that quantum, it is preempted and placed at the back of the queue, and the CPU moves on to the next process in line. If a process completes or blocks before its quantum expires, the CPU immediately moves to the next process.

The performance of Round Robin depends heavily on the size of the time quantum. A very large quantum makes RR behave like FCFS, while a very small quantum increases fairness but also increases context-switch overhead. Round Robin ensures that no process waits more than $(n - 1) \times \text{quantum time}$ before getting the CPU again, making it well suited for interactive and time-sharing environments.

ALGORITHM

1. Start the program.
2. Read the number of processes n.
3. For each process, read its Arrival Time (AT) and Burst Time (BT); initialise its Remaining Burst Time as equal to BT.
4. Read the Time Quantum (TQ).
5. Insert the first process to arrive into the ready queue and set the current time to its Arrival Time.
6. Remove the process at the front of the ready queue.
7. If its Remaining Burst Time is greater than TQ, execute it for TQ units, reduce its Remaining Burst Time by TQ, and advance the current time by TQ.
8. Otherwise, execute it for its Remaining Burst Time, set its Remaining Burst Time to 0, advance the current time accordingly, and record its Completion Time (CT).
9. Add to the ready queue any new processes that have arrived by the updated current time.
10. If the process executed in step 6 has not finished, add it to the back of the ready queue.
11. Repeat steps 6 to 10 until all processes are completed and the ready queue is empty.
12. Calculate Turnaround Time: $TAT[i] = CT[i] - AT[i]$.
13. Calculate Waiting Time: $WT[i] = TAT[i] - BT[i]$.
14. Calculate the average Turnaround Time and average Waiting Time.
15. Display the Process ID, AT, BT, CT, TAT and WT of each process along with the averages.
16. Stop the program.

PROGRAM

```
#include <stdio.h>

int main()
{
    int n, i, tq, current_time = 0, completed = 0;
    int at[20], bt[20], rem_bt[20], ct[20], tat[20], wt[20], p[20];
    float total_tat = 0, total_wt = 0;
    int queue[100], front = 0, rear = 0;
    int in_queue[20] = {0};

    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
for (i = 0; i < n; i++)
{
    p[i] = i + 1;
    printf("Enter Arrival Time and Burst Time for P%d: ", p[i]);
    scanf("%d %d", &at[i], &bt[i]);
    rem_bt[i] = bt[i];
}

printf("Enter Time Quantum: ");
scanf("%d", &tq);

/* Start with the process that arrives first */
int start = 0;
for (i = 1; i < n; i++)
    if (at[i] < at[start]) start = i;

current_time = at[start];
queue[rear++] = start;
in_queue[start] = 1;

while (completed < n)
{
    int idx = queue[front++];

    if (rem_bt[idx] > tq)
    {
        current_time += tq;
        rem_bt[idx] -= tq;
    }
    else
    {
        current_time += rem_bt[idx];
        rem_bt[idx] = 0;
    }

    /* Enqueue any process that has arrived by current_time
       and is not yet in the queue and not finished */
    for (i = 0; i < n; i++)
    {
        if (at[i] <= current_time && rem_bt[i] > 0 && !in_queue[i] && i != idx)
        {
            queue[rear++] = i;
            in_queue[i] = 1;
        }
    }
}
```

```

    }

    if (rem_bt[idx] == 0)
    {
        ct[idx] = current_time;
        completed++;
    }
    else
    {
        queue[rear++] = idx; /* re-queue the process at the tail */
    }
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for (i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_tat += tat[i];
    total_wt += wt[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
           p[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("\nAverage Turnaround Time = %.2f", total_tat / n);
printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

return 0;
}

```

PROCEDURE

1. Open a text editor / IDE (e.g. Turbo C, Code::Blocks, or GCC on a terminal).
2. Type the C program shown above and save it with the extension .c (for example, rr.c).
3. Compile the program using the command: gcc rr.c -o rr
4. Run the executable using the command: ./rr (or rr.exe on Windows).
5. Enter the number of processes when prompted.
6. Enter the Arrival Time and Burst Time for each process.
7. Enter the Time Quantum when prompted.
8. Observe the output, which displays the Completion Time, Turnaround Time and Waiting Time for each process, along with the average Turnaround Time and average Waiting Time.

9. Verify the output manually by drawing the Gantt chart, moving each running process to the back of the queue whenever it is preempted after using one full time quantum.

OUTPUT

Sample input: $n = 4$ processes with the following Arrival Time and Burst Time, and Time Quantum = 2.

Gantt chart obtained: P1 (0–2) → P2 (2–4) → P3 (4–6) → P4 (6–7) → P1 (7–10) → P2 (10–12) → P3 (12–13).

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time
P1	0	5	10	10	5
P2	1	4	12	11	7
P3	2	2	13	11	9
P4	3	1	7	4	3

Average Turnaround Time = $(10 + 11 + 11 + 4) / 4 = 9.00$

Average Waiting Time = $(5 + 7 + 9 + 3) / 4 = 6.00$

MODEL VIVA QUESTIONS

1. What is Round Robin scheduling? Why is it called a preemptive algorithm?
2. What is a time quantum? How does its size affect scheduling performance?
3. What happens to Round Robin if the time quantum is very large? Very small?
4. What data structure is used to implement the ready queue in Round Robin?
5. What is a context switch? Why does Round Robin incur more context-switch overhead than FCFS?

RESULT

The C program to simulate the Round Robin (RR) preemptive CPU scheduling algorithm was written, executed, and verified successfully. The Completion Time, Turnaround Time and Waiting Time for the given set of processes were computed correctly for the chosen time quantum, and the output was verified using a Gantt chart.

D) SIMULATION OF PRIORITY CPU SCHEDULING ALGORITHM

AIM

To simulate the **Non-Preemptive Priority CPU Scheduling** algorithm and calculate the **Waiting Time (WT)** and **Turnaround Time (TAT)** for each process.

LEARNING OUTCOME

After completing this experiment, students will be able to:

- Understand the concept of Priority CPU Scheduling.
- Implement the Non-Preemptive Priority Scheduling algorithm.
- Calculate Waiting Time and Turnaround Time.
- Compare execution order based on process priority.
- Analyze CPU scheduling performance.

DESCRIPTION

Priority Scheduling is a CPU scheduling algorithm in which each process is assigned a priority. The CPU is allocated to the process having the highest priority (or the lowest priority number, depending on convention). In **Non-Preemptive Priority Scheduling**, once a process starts execution, it continues until completion without interruption.

Formula

Turnaround Time (TAT) = Completion Time – Arrival Time

Waiting Time (WT) = Turnaround Time – Burst Time

ALGORITHM

1. Start.
2. Read the number of processes.
3. Enter Process ID, Burst Time, and Priority for each process.
4. Sort the processes according to priority (lowest number indicates highest priority).
5. Set Waiting Time of the first process to 0.
6. Calculate Waiting Time for each remaining process.
7. Calculate Turnaround Time using:
 - $TAT = WT + BT$
8. Compute Average Waiting Time and Average Turnaround Time.
9. Display the scheduling table.
10. Stop.

PROGRAM

```
#include <stdio.h>

int main()
{
    int n, i, j;
    int bt[20], wt[20], tat[20], p[20], temp;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Burst Time and Priority\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d\n", i + 1);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Priority: ");
        scanf("%d", &p[i]);
    }

    // Sorting according to priority
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(p[i] > p[j])
            {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    wt[0] = 0;

    for(i = 1; i < n; i++)
    {
```

```
    wt[i] = wt[i - 1] + bt[i - 1];
}

for(i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nProcess\tPriority\tBT\tWT\tTAT\n");
for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, p[i], bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}
```

PROCEDURE

1. Open Turbo C/CodeBlocks/Visual Studio Code.
2. Create a new C program.
3. Enter the above source code.
4. Save the file with a .c extension.
5. Compile the program.
6. Execute the program.
7. Enter the number of processes, burst times, and priorities.
8. Observe the Waiting Time and Turnaround Time of each process.
9. Note the average Waiting Time and Turnaround Time.

SAMPLE OUTPUT

Enter number of processes: 4

Enter Burst Time and Priority

Process 1

Burst Time: 5

Priority: 2

Process 2

Burst Time: 3

Priority: 1

Process 3

Burst Time: 8

Priority: 4

Process 4

Burst Time: 6

Priority: 3

Process	Priority	BT	WT	TAT
---------	----------	----	----	-----

P1	1	3	0	3
----	---	---	---	---

P2	2	5	3	8
----	---	---	---	---

P3	3	6	8	14
----	---	---	---	----

P4	4	8	14	22
----	---	---	----	----

Average Waiting Time = 6.25

Average Turnaround Time = 11.75

MODEL VIVA QUESTIONS

1. What is CPU Scheduling?
2. What is Priority Scheduling?
3. What is the difference between Preemptive and Non-Preemptive Scheduling?
4. Which process executes first in Priority Scheduling?
5. What is Burst Time?

RESULT

Thus, the Non-Preemptive Priority CPU Scheduling algorithm was successfully simulated, and the Waiting Time, Turnaround Time, Average Waiting Time, and Average Turnaround Time were calculated correctly.

EXP NO : 2**A) SIMULATION OF SEQUENTIAL FILE ALLOCATION STRATEGY****AIM**

To simulate the **Sequential File Allocation Strategy** and demonstrate how files are allocated in contiguous memory blocks.

LEARNING OUTCOME

After completing this experiment, students will be able to:

- Understand the concept of Sequential File Allocation.
- Implement the Sequential File Allocation strategy using C.
- Allocate files in contiguous memory blocks.
- Identify whether sufficient contiguous space is available.
- Analyze the advantages and disadvantages of Sequential Allocation.

DESCRIPTION

Sequential (Contiguous) File Allocation is one of the simplest file allocation methods used in operating systems. In this method, each file occupies a set of **contiguous (adjacent) disk blocks**. The operating system stores the **starting block number** and the **length (number of blocks)** of each file.

When a new file is created, the operating system searches for a sequence of free contiguous blocks large enough to store the file.

Advantages

- Simple to implement.
- Fast file access.
- Efficient sequential and direct access.
- Low storage overhead.

Disadvantages

- External fragmentation occurs.
- File size must be known before allocation.
- Difficult to extend files after creation.

ALGORITHM

1. Start.
2. Read the total number of memory blocks.
3. Initialize all blocks as free.

4. Read the number of files.
5. For each file:
 - Read the starting block.
 - Read the file size (number of blocks).
 - Check whether all required contiguous blocks are free.
 - If available, allocate the blocks.
 - Otherwise, display "Allocation Not Possible."
6. Display the allocated blocks for each file.
7. Stop.

PROGRAM

```
#include <stdio.h>

int main()
{
    int memory[100] = {0};
    int n, files;
    int start, size;
    int i, j, flag;

    printf("Enter total number of memory blocks: ");
    scanf("%d", &n);

    printf("Enter number of files: ");
    scanf("%d", &files);

    for(i = 1; i <= files; i++)
    {
        printf("\nFile %d\n", i);

        printf("Enter starting block: ");
        scanf("%d", &start);

        printf("Enter file size (number of blocks): ");
        scanf("%d", &size);

        flag = 1;

        if(start + size > n)
            flag = 0;
        else
        {
            for(j = start; j < start + size; j++)
            {
```

```
        if(memory[j] == 1)
        {
            flag = 0;
            break;
        }
    }

    if(flag == 1)
    {
        printf("File Allocated Successfully.\n");
        printf("Allocated Blocks: ");

        for(j = start; j < start + size; j++)
        {
            memory[j] = 1;
            printf("%d ", j);
        }
        printf("\n");
    }
    else
    {
        printf("Allocation Not Possible.\n");
    }
}

return 0;
}
```

PROCEDURE

1. Open Turbo C, Code::Blocks, or Visual Studio Code.
2. Create a new C source file.
3. Type the given program.
4. Save the file with a .c extension.
5. Compile the program.
6. Execute the program.
7. Enter the total number of memory blocks.
8. Enter the number of files.
9. Provide the starting block and file size for each file.
10. Observe whether allocation is successful or not.

OUTPUT

Enter total number of memory blocks: 20

Enter number of files: 3

File 1

Enter starting block: 2

Enter file size (number of blocks): 5

File Allocated Successfully.

Allocated Blocks: 2 3 4 5 6

File 2

Enter starting block: 8

Enter file size (number of blocks): 4

File Allocated Successfully.

Allocated Blocks: 8 9 10 11

File 3

Enter starting block: 5

Enter file size (number of blocks): 6

Allocation Not Possible.

MODEL VIVA QUESTIONS

1. What is file allocation?
2. What is Sequential (Contiguous) File Allocation?
3. Why is it called contiguous allocation?
4. What information is stored for each file in Sequential Allocation?
5. What are memory blocks?
6. What is external fragmentation?
7. Why does external fragmentation occur?
8. What are the advantages of Sequential Allocation?
9. What are the disadvantages of Sequential Allocation?
10. Can a file grow after allocation in Sequential Allocation? Why?
11. What happens if contiguous free space is unavailable?
12. Name other file allocation methods.
13. Differentiate Sequential Allocation and Linked Allocation.
14. Differentiate Sequential Allocation and Indexed Allocation.
15. Which file allocation method provides the fastest file access?

RESULT

Thus, the **Sequential File Allocation Strategy** was successfully simulated using a C program.

B) SIMULATION OF INDEXED FILE ALLOCATION STRATEGY

AIM

To write a C program to simulate the Indexed File Allocation strategy used by an operating system for allocating disk blocks to files, and to study how an index block is used to keep track of the data blocks allocated to a file.

LEARNING OUTCOME

- Understand the concept of file allocation methods used by an operating system to manage disk space.
- Understand how indexed allocation uses a separate index block to store pointers to all data blocks of a file.
- Able to implement allocation and free-block management using arrays in C.
- Able to compare indexed allocation with contiguous and linked allocation in terms of direct access, overhead and fragmentation.
- Able to trace and verify allocation of blocks to multiple files without conflict.

DESCRIPTION

Indexed allocation is a disk space allocation strategy in which each file has its own index block. The index block is an array of disk block addresses (pointers) that point to the actual data blocks belonging to the file. When a file is created, an index block is allocated for it, and every data block used by the file has its address recorded as an entry in this index block.

Unlike contiguous allocation, indexed allocation does not require the blocks of a file to be located next to each other on the disk, so it does not suffer from external fragmentation and does not require the maximum file size to be known in advance. Unlike linked allocation, it supports direct (random) access to any block of the file, because the address of the desired block can be obtained directly from the index block instead of traversing a chain of pointers.

The main overhead of this method is the extra disk block(s) needed to store the index itself. For very large files, multiple levels of indexing (linked index blocks, multilevel index, or combined schemes as used in UNIX inodes) may be required. In this simulation, a simplified single-level index block is used: the user specifies the total number of disk blocks, the number of files, and for every file the block numbers to be allocated; the program allocates the blocks if they are free and prints the index block content of each file along with the final free/allocated status of the disk.

ALGORITHM

1. Start the program.
2. Read the total number of disk blocks available and initialize a free-block status array with all blocks marked as free (0).
3. Read the number of files to be created.
4. For each file, read the number of blocks required.
5. For each required block of the file, read the block number requested by the user.
6. Check whether the requested block number is valid and free. If it is free, mark it as allocated and store its address as an entry in the index block of that file.
7. If the block is invalid or already allocated, prompt the user to enter a different block number.
8. Repeat steps 4–6 until all files have been allocated their required number of blocks.
9. Display the index block (list of allocated data block numbers) for every file.
10. Display the final status (Free / Allocated) of every disk block.
11. Stop the program.

PROGRAM

```
#include <stdio.h>

#define MAX 20

int main()
{
    int files, blocks, i, j, k, req[MAX], idx[MAX][MAX], count[MAX];
    int free_block[MAX];

    printf("Enter number of blocks in the disk: ");
    scanf("%d", &blocks);

    for (i = 0; i < blocks; i++)
        free_block[i] = 0; /* 0 = free, 1 = allocated */

    printf("Enter number of files: ");
    scanf("%d", &files);

    for (i = 0; i < files; i++)
    {
        printf("\nEnter number of blocks required for file %d: ", i + 1);
        scanf("%d", &req[i]);
        count[i] = 0;
    }
}
```

```

printf("Enter block numbers (index) for file %d:\n", i + 1);
for (j = 0; j < req[i]; j++)
{
    printf("Block %d: ", j + 1);
    scanf("%d", &k);

    if (k < 0 || k >= blocks)
    {
        printf("Invalid block number!\n");
        j--;
        continue;
    }
    if (free_block[k] == 1)
    {
        printf("Block %d already allocated. Choose another.\n", k);
        j--;
        continue;
    }

    idx[i][count[i]] = k;
    free_block[k] = 1;
    count[i]++;
}
}

printf("\nFile\tIndex Block --> Allocated Data Blocks\n");
for (i = 0; i < files; i++)
{
    printf("%d\t[ ", i + 1);
    for (j = 0; j < count[i]; j++)
        printf("%d ", idx[i][j]);
    printf("]\n");
}

printf("\nBlock\tStatus\n");
for (i = 0; i < blocks; i++)
    printf("%d\t%s\n", i, free_block[i] ? "Allocated" : "Free");

return 0;
}

```

PROCEDURE

1. Open a terminal / Linux lab system and create a new file, e.g. indexed.c, using a text editor.
2. Type or copy the C program given above into the file and save it.

3. Compile the program using the GCC compiler: `gcc indexed.c -o indexed`
4. Run the compiled program: `./indexed`
5. When prompted, enter the total number of disk blocks in the system.
6. Enter the number of files to be simulated.
7. For each file, enter the number of blocks it requires and the specific block numbers to allocate to it.
8. Observe the index block printed for each file and the free/allocated status of every disk block.
9. Repeat the run with different inputs (including invalid or already-allocated block numbers) to verify that the program correctly rejects them and re-prompts.

OUTPUT

Sample Run:

Enter number of blocks in the disk: 10

Enter number of files: 2

Enter number of blocks required for file 1: 3

Enter block numbers (index) for file 1:

Block 1: 2

Block 2: 5

Block 3: 7

Enter number of blocks required for file 2: 2

Enter block numbers (index) for file 2:

Block 1: 1

Block 2: 9

File Index Block --> Allocated Data Blocks

1 [2 5 7]

2 [1 9]

Block Status

0 Free

1 Allocated

2 Allocated

3 Free

4 Free

5 Allocated

6 Free

7 Allocated

8 Free

9 Allocated

MODEL VIVA QUESTIONS

1. What is file allocation and why is it needed in an operating system?
2. What are the different file allocation methods supported by an OS?
3. What is an index block in indexed allocation?
4. How does indexed allocation support direct access to a file's blocks?
5. What is the main disadvantage / overhead of indexed allocation?

RESULT

Thus, a C program to simulate the Indexed File Allocation strategy was written, executed and verified successfully. The program correctly allocates the requested disk blocks to each file, maintains a separate index block containing the addresses of the data blocks belonging to that file, prevents allocation of blocks already in use, and displays the final allocation status of the disk.

C) SIMULATION OF LINKED FILE ALLOCATION STRATEGY

AIM

To write a C program to simulate the Linked File Allocation strategy used by an operating system for allocating disk blocks to files, and to study how each data block of a file holds a pointer to the next block in the chain.

LEARNING OUTCOME

- Understand the concept of file allocation methods used by an operating system to manage disk space.
- Understand how linked allocation uses a pointer stored in every block to link it to the next block of the same file.
- Able to implement a linked-list based allocation and free-block management scheme using arrays in C.
- Able to compare linked allocation with contiguous and indexed allocation in terms of direct access, overhead and fragmentation.
- Able to trace the block chain of a file from its starting block to the terminating NULL pointer.

DESCRIPTION

Linked allocation is a disk space allocation strategy in which each file is represented as a linked list of disk blocks. The disk blocks of a file need not be contiguous; they may be scattered anywhere on the disk. The directory entry for the file contains a pointer to the first block of the file (the start block). Every block, in addition to holding file data, reserves a small area to store the address of the next block belonging to the file, and the last block's pointer is set to NULL (or -1) to mark the end of the chain.

Because blocks are linked rather than stored together, linked allocation completely eliminates external fragmentation, and a file can grow simply by linking a new free block to the end of the chain, so its maximum size need not be known in advance. Its main drawback is that it supports only sequential access efficiently — to reach the *n*th block of a file, the operating system must traverse the chain from the start block, following *n* pointers. It is also less reliable, since a single corrupted pointer can break the chain and make the remaining blocks of the file unreachable, and each block wastes some space for storing the pointer.

In this simulation, a simplified scheme is used: the user specifies the total number of disk blocks, the number of files, and for every file the sequence of block numbers to be allocated. The program allocates each block if it is free, links it to the previous block of the same file, marks the last block's pointer as NULL, and finally prints the start block and the complete chain of blocks for every file along with the free/allocated status of the entire disk.

ALGORITHM

1. Start the program.
2. Read the total number of disk blocks available and initialize a free-block status array with all blocks marked as free (0), and the next-pointer of every block as -1.
3. Read the number of files to be created.
4. For each file, read the number of blocks required.
5. For each required block of the file, read the block number requested by the user.
6. Check whether the requested block number is valid and free. If it is free, mark it as allocated.
7. If this is the first block of the file, store it as the start (head) pointer of the file; otherwise, set the next-pointer of the previously allocated block of this file to the current block, thereby linking the two blocks.
8. After the last required block of the file has been allocated, set its next-pointer to -1 (NULL) to terminate the chain.
9. If a requested block is invalid or already allocated, prompt the user to enter a different block number.
10. Repeat steps 4–8 until all files have been allocated their required number of blocks.
11. Display the start block and the complete linked chain of data blocks for every file.
12. Display the final status (Free / Allocated) of every disk block.
13. Stop the program.

PROGRAM

```
#include <stdio.h>

#define MAX 20

int main()
{
    int files, blocks, i, j, k, req[MAX], start[MAX];
    int link[MAX][MAX]; /* link[i][j] = jth block of file i */
    int next[MAX];      /* next[k] = block linked after block k */
    int free_block[MAX]; /* 0 = free, 1 = allocated */

    printf("Enter number of blocks in the disk: ");
    scanf("%d", &blocks);

    for (i = 0; i < blocks; i++)
    {
        free_block[i] = 0;
        next[i] = -1;
    }
}
```

```
printf("Enter number of files: ");
scanf("%d", &files);

for (i = 0; i < files; i++)
{
    printf("\nEnter number of blocks required for file %d: ", i + 1);
    scanf("%d", &req[i]);

    for (j = 0; j < req[i]; j++)
    {
        printf("Enter block %d for file %d: ", j + 1, i + 1);
        scanf("%d", &k);

        if (k < 0 || k >= blocks || free_block[k] == 1)
        {
            printf("Block invalid or already allocated. Try again.\n");
            j--;
            continue;
        }

        free_block[k] = 1;
        link[i][j] = k;

        if (j == 0)
            start[i] = k;          /* first block = start pointer */
        else
            next[link[i][j - 1]] = k; /* link previous block to this */
    }
    next[link[i][req[i] - 1]] = -1; /* last block points to NULL */
}

printf("\nFile\tStart Block\tBlock Chain\n");
for (i = 0; i < files; i++)
{
    printf("%d\t%d\t\t", i + 1, start[i]);
    k = start[i];
    while (k != -1)
    {
        printf("%d", k);
        k = next[k];
        if (k != -1)
            printf(" -> ");
    }
    printf(" -> NULL\n");
}
```

```
}

printf("\nBlock\tStatus\n");
for (i = 0; i < blocks; i++)
    printf("%d\t%s\n", i, free_block[i] ? "Allocated" : "Free");

return 0;
}
```

PROCEDURE

1. Open a terminal / Linux lab system and create a new file, e.g. linked.c, using a text editor.
2. Type or copy the C program given above into the file and save it.
3. Compile the program using the GCC compiler: gcc linked.c -o linked
4. Run the compiled program: ./linked
5. When prompted, enter the total number of disk blocks in the system.
6. Enter the number of files to be simulated.
7. For each file, enter the number of blocks it requires and the specific block numbers, in the order they should be linked, one after another.
8. Observe the start block and the block chain printed for each file, and the free/allocated status of every disk block.
9. Repeat the run with different inputs (including invalid or already-allocated block numbers) to verify that the program correctly rejects them and re-prompts.

OUTPUT

Sample Run:

Enter number of blocks in the disk: 10

Enter number of files: 2

Enter number of blocks required for file 1: 3

Enter block 1 for file 1: 2

Enter block 2 for file 1: 5

Enter block 3 for file 1: 8

Enter number of blocks required for file 2: 2

Enter block 1 for file 2: 1

Enter block 2 for file 2: 6

File	Start Block	Block Chain
1	2	2 -> 5 -> 8 -> NULL
2	1	1 -> 6 -> NULL

Block Status

0	Free
1	Allocated
2	Allocated
3	Free
4	Free
5	Allocated
6	Allocated
7	Free
8	Allocated
9	Free

MODEL VIVA QUESTIONS

1. What is file allocation and why is it needed in an operating system?
2. What are the different file allocation methods supported by an OS?
3. What is linked allocation and how does it store a file on disk?
4. How is the next block of a file located in linked allocation?
5. Does linked allocation suffer from external fragmentation? Justify your answer.
6. What is the major drawback of linked allocation with respect to direct access?
7. How is linked allocation different from indexed allocation?
8. How is linked allocation different from contiguous allocation?
9. What happens to the rest of a file if one pointer in the chain gets corrupted?
10. What is File Allocation Table (FAT) and how does it relate to linked allocation?
11. What overhead does every block carry in pure linked allocation?

RESULT

Thus, a C program to simulate the Linked File Allocation strategy was written, executed and verified successfully. The program correctly allocates the requested disk blocks to each file, links every block to the next block of the same file starting from the start block and ending in NULL, prevents allocation of blocks already in use, and displays the final allocation status of the disk.

EXP NO.3**IMPLEMENTATION OF PAGING TECHNIQUE OF MEMORY MANAGEMENT****AIM**

To write a C program to simulate the Paging technique of memory management used by an operating system, wherein the logical address space of a process is divided into fixed-size pages and the physical memory is divided into equal-size frames, and to study how a page table is used to translate a logical address into the corresponding physical address.

LEARNING OUTCOME

- Understand the concept of paging as a non-contiguous memory allocation technique.
- Understand how logical address space is divided into pages and physical address space into frames of the same size.
- Understand the role of the page table in mapping page numbers to frame numbers.
- Able to implement address translation by splitting a logical address into a page number and a page offset.
- Able to compute the corresponding physical address using the frame number and the offset.
- Able to compare paging with contiguous allocation in terms of fragmentation and flexibility.

DESCRIPTION

Paging is a memory management technique that permits the physical address space of a process to be non-contiguous. The logical address space of a process is divided into fixed-size blocks called pages, and the physical memory is divided into fixed-size blocks of the same size called frames. The size of a page is always equal to the size of a frame, and both are defined by the underlying hardware, typically as a power of two.

When a process is to be executed, its pages are loaded into any available frames in physical memory, not necessarily contiguous ones. A page table is maintained for every process to keep track of the frame in which each page of the process is stored. Every logical address generated by the CPU is divided into two parts: a page number (p), which is used as an index into the page table to obtain the corresponding frame number, and a page offset (d), which identifies the exact location within that page/frame. The physical address is then computed by combining the frame number with the offset, so that the same offset locates the corresponding byte within the frame.

Because any free frame can be assigned to any page of a process, paging completely eliminates external fragmentation, since there is no requirement for contiguous allocation. However, it may still suffer from internal fragmentation, which occurs when the size of a process is not an exact multiple of the page size, causing the last page to be only partially filled while the entire frame is reserved for it.

In this simulation, a simplified scheme is used: the user specifies the size of a page/frame, the total number of frames available in physical memory, and the number of pages required by the process. The program allocates a free frame to every page (building the page table), and then accepts a logical address, splits it into a page number and offset, looks up the frame number from the page table, and computes and displays the equivalent physical address.

ALGORITHM

1. Start the program.
2. Read the page size (which is the same as the frame size) and the total number of frames available in physical memory.
3. Read the number of pages required by the process.
4. Check whether the number of pages required is less than or equal to the number of free frames available. If not, report that the process cannot be accommodated in memory.
5. For every page of the process, allocate the next free frame and record the mapping of page number to frame number in the page table.
6. Display the constructed page table showing the page number and the corresponding frame number for every page.
7. Read a logical address to be translated.
8. Compute the page number as (logical address / page size) and the page offset as (logical address % page size).
9. Check whether the computed page number is valid (within the number of pages of the process). If not, report an invalid address.
10. Look up the frame number corresponding to the page number from the page table.
11. Compute the physical address as (frame number * page size) + page offset.
12. Display the page number, offset, frame number and the computed physical address.
13. Repeat steps 7–12 for further address translations as required.
14. Stop the program.

PROGRAM

```
#include <stdio.h>

#define MAX 50

int main()
{
    int page_size, frames, pages, i;
    int frame_table[MAX];    /* frame_table[p] = frame allotted to page p */
```

```
int frame_used[MAX];    /* 0 = free, 1 = allocated */
int logical_addr, page_no, offset, physical_addr;
char choice;

printf("Enter page size (in bytes): ");
scanf("%d", &page_size);

printf("Enter number of frames in physical memory: ");
scanf("%d", &frames);

for (i = 0; i < frames; i++)
    frame_used[i] = 0;

printf("Enter number of pages required by the process: ");
scanf("%d", &pages);

if (pages > frames)
{
    printf("Process cannot be accommodated. Not enough free frames.\n");
    return 0;
}

/* Build the page table: allocate the next free frame to every page */
for (i = 0; i < pages; i++)
{
    frame_table[i] = i;    /* simple simulation: frame i -> page i */
    frame_used[i] = 1;
}

printf("\nPage Table\n");
printf("Page No\tFrame No\n");
for (i = 0; i < pages; i++)
    printf("%d\t%d\n", i, frame_table[i]);

do
{
    printf("\nEnter logical address to be translated: ");
    scanf("%d", &logical_addr);

    page_no = logical_addr / page_size;
    offset = logical_addr % page_size;

    if (page_no < 0 || page_no >= pages)
    {
        printf("Invalid logical address (page number out of range).\n");
    }
}
```

```
}
else
{
    physical_addr = (frame_table[page_no] * page_size) + offset;

    printf("Page Number  : %d\n", page_no);
    printf("Offset      : %d\n", offset);
    printf("Frame Number : %d\n", frame_table[page_no]);
    printf("Physical Addr : %d\n", physical_addr);
}

printf("\nTranslate another address? (y/n): ");
scanf(" %c", &choice);

} while (choice == 'y' || choice == 'Y');

return 0;
}
```

PROCEDURE

1. Open a terminal / Linux lab system and create a new file, e.g. paging.c, using a text editor.
2. Type or copy the C program given above into the file and save it.
3. Compile the program using the GCC compiler: gcc paging.c -o paging
4. Run the compiled program: ./paging
5. When prompted, enter the page size (in bytes) and the total number of frames in physical memory.
6. Enter the number of pages required by the process.
7. Observe the page table displayed, showing the frame allotted to every page.
8. Enter a logical address and observe the page number, offset, frame number and the computed physical address.
9. Repeat the translation for different logical addresses, including addresses that lie beyond the page range, to verify that the program correctly reports an invalid address.

OUTPUT

Sample Run:

```
Enter page size (in bytes): 512
Enter number of frames in physical memory: 8
Enter number of pages required by the process: 4
```

Page Table

Page No	Frame No
0	0
1	1
2	2
3	3

Enter logical address to be translated: 1050

Page Number : 2

Offset : 26

Frame Number : 2

Physical Addr : 1050

Translate another address? (y/n): y

Enter logical address to be translated: 2200

Invalid logical address (page number out of range).

Translate another address? (y/n): n

MODEL VIVA QUESTIONS

1. What is paging in memory management and why is it used?
2. What is the difference between a page and a frame?
3. How is a logical address translated into a physical address in paging?
4. What is a page table and what information does it store?
5. Does paging suffer from external fragmentation? Justify your answer.

RESULT

Thus, a C program to simulate the Paging technique of memory management was written, executed and verified successfully. The program correctly constructs the page table by allocating a frame to every page of the process, translates a given logical address into its page number and offset, and computes the corresponding physical address using the page table, while correctly reporting invalid addresses that fall outside the page range of the process.

EXPNO.4**A) SIMULATION OF SINGLE LEVEL DIRECTORY FILE ORGANIZATION TECHNIQUE****AIM**

To study, simulate, and implement a single level directory file organization technique using the C programming language, and to perform basic file operations such as create, delete, search, and display within it.

LEARNING OUTCOME

- Understand the concept of directory structures used in operating systems for file organization.
- Understand the working, advantages, and limitations of a single level directory structure.
- Gain the ability to simulate basic file system operations such as creation, deletion, and searching of files.
- Develop the ability to represent an operating system concept using an appropriate data structure and program logic.

DESCRIPTION

A directory is a container that holds information about the files present in a file system, such as file names, locations, sizes, and access permissions. Among the various directory structures used in operating systems, the single level directory is the simplest form, where all the files created by all users are placed in one common directory. There is no concept of sub-directories, and hence every file in the system must have a unique name so that it can be identified without ambiguity.

Although a single level directory is easy to implement and manage due to its simplicity, it suffers from major drawbacks when the number of files or users increases. Since only one directory exists, naming conflicts frequently occur if two files are given the same name, and there is no way to logically group related files together. This structure is therefore best suited for very small or single-user systems.

In this experiment, a single level directory is simulated using an array of structures in C, where each structure element represents a file entry consisting of the file name and a status flag indicating whether the entry is in use. Basic directory operations such as creating a file, deleting a file, searching for a file, and displaying the contents of the directory are implemented using simple linear search logic.

ALGORITHM

1. Start the program and initialize the directory (an array of file structures) with all entries marked as empty/unused.
2. Display a menu with the options: Create File, Delete File, Search File, Display Directory, and Exit.

3. Read the user's choice.
4. If the choice is Create File: read the file name, check whether a file with the same name already exists in the directory; if it exists, display an error message, otherwise store the new file name in the first available empty slot and increment the file count.
5. If the choice is Delete File: read the file name, search for it in the directory; if found, mark that slot as empty and decrement the file count, otherwise display a 'file not found' message.
6. If the choice is Search File: read the file name, perform a linear search through the directory, and display whether the file exists or not.
7. If the choice is Display Directory: traverse the entire array and print the names of all files that are currently marked as used, along with the total file count.
8. If the choice is Exit: terminate the program.
9. Repeat steps 2 to 8 until the user chooses to exit.
10. Stop the program.

PROGRAM

```
#include <stdio.h>
#include <string.h>

#define MAX 20

struct file {
    char name[30];
    int used;
};

struct file directory[MAX];
int fileCount = 0;

void createFile(char *name) {
    int i;
    for (i = 0; i < MAX; i++) {
        if (directory[i].used == 1 && strcmp(directory[i].name, name) == 0) {
            printf("Error: File '%s' already exists.\n", name);
            return;
        }
    }
    for (i = 0; i < MAX; i++) {
        if (directory[i].used == 0) {
            strcpy(directory[i].name, name);
            directory[i].used = 1;
            fileCount++;
        }
    }
}
```

```
        printf("File '%s' created successfully.\n", name);
        return;
    }
}
printf("Error: Directory is full.\n");
}

void deleteFile(char *name) {
    int i;
    for (i = 0; i < MAX; i++) {
        if (directory[i].used == 1 && strcmp(directory[i].name, name) == 0) {
            directory[i].used = 0;
            fileCount--;
            printf("File '%s' deleted successfully.\n", name);
            return;
        }
    }
    printf("Error: File '%s' not found.\n", name);
}

void searchFile(char *name) {
    int i;
    for (i = 0; i < MAX; i++) {
        if (directory[i].used == 1 && strcmp(directory[i].name, name) == 0) {
            printf("File '%s' found in the directory.\n", name);
            return;
        }
    }
    printf("File '%s' not found in the directory.\n", name);
}

void displayDirectory() {
    int i;
    printf("\n----- Single Level Directory ----- \n");
    printf("Total Files: %d\n", fileCount);
    for (i = 0; i < MAX; i++) {
        if (directory[i].used == 1)
            printf("%s\n", directory[i].name);
    }
    printf("----- \n");
}

int main() {
    int choice;
    char fname[30];
```

```
for (int i = 0; i < MAX; i++)
    directory[i].used = 0;

while (1) {
    printf("\n----- Single Level Directory Simulation ----- \n");
    printf("1. Create File\n");
    printf("2. Delete File\n");
    printf("3. Search File\n");
    printf("4. Display Directory\n");
    printf("5. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter file name to create: ");
            scanf("%s", fname);
            createFile(fname);
            break;
        case 2:
            printf("Enter file name to delete: ");
            scanf("%s", fname);
            deleteFile(fname);
            break;
        case 3:
            printf("Enter file name to search: ");
            scanf("%s", fname);
            searchFile(fname);
            break;
        case 4:
            displayDirectory();
            break;
        case 5:
            printf("Exiting program.\n");
            return 0;
        default:
            printf("Invalid choice. Try again.\n");
    }
}
return 0;
}
```

PROCEDURE

1. Open a text editor or IDE (such as Turbo C, Code::Blocks, or GCC on Linux) and create a new source file, for example sld.c.
2. Type in the C program that simulates the single level directory structure as shown above.
3. Compile the program using a C compiler, for example: gcc sld.c -o sld
4. Execute the compiled program, for example: ./sld (on Linux) or sld.exe (on Windows).
5. From the menu displayed, choose option 1 to create a few sample files by entering unique file names.
6. Choose option 4 to display the current contents of the directory and verify that the created files are listed correctly.
7. Choose option 3 to search for a specific file and observe whether it is found or not.
8. Choose option 2 to delete one of the created files and then choose option 4 again to confirm that the file has been removed from the directory.
9. Try creating a file with a name that already exists to observe how the program handles the naming conflict.
10. Choose option 5 to exit the program, and record the outputs obtained for the lab report.

OUTPUT

----- Single Level Directory Simulation -----

1. Create File
2. Delete File
3. Search File
4. Display Directory
5. Exit

Enter your choice: 1

Enter file name to create: sample.txt

File 'sample.txt' created successfully.

----- Single Level Directory Simulation -----

1. Create File
2. Delete File
3. Search File
4. Display Directory
5. Exit

Enter your choice: 1

Enter file name to create: report.doc

File 'report.doc' created successfully.

----- Single Level Directory Simulation -----

1. Create File
2. Delete File
3. Search File
4. Display Directory
5. Exit

Enter your choice: 4

----- Single Level Directory -----

Total Files: 2

sample.txt

report.doc

----- Single Level Directory Simulation -----

1. Create File
2. Delete File
3. Search File
4. Display Directory
5. Exit

Enter your choice: 3

Enter file name to search: sample.txt

File 'sample.txt' found in the directory.

----- Single Level Directory Simulation -----

1. Create File
2. Delete File
3. Search File
4. Display Directory
5. Exit

Enter your choice: 2

Enter file name to delete: report.doc

File 'report.doc' deleted successfully.

----- Single Level Directory Simulation -----

1. Create File
2. Delete File
3. Search File
4. Display Directory
5. Exit

Enter your choice: 5

Exiting program.

MODEL VIVA QUESTIONS

1. What is a directory in an operating system?
2. What is a single level directory?
3. What is the main advantage of a single level directory?
4. What are the disadvantages of a single level directory?
5. How is a single level directory different from a two level directory?

RESULT

The single level directory file organization technique was successfully studied and simulated using a C program. Basic file operations, namely creation, deletion, searching, and display of files, were implemented and executed successfully, and the corresponding outputs were verified.

B) SIMULATION OF TWO-LEVEL DIRECTORY FILE ORGANIZATION TECHNIQUE

AIM

To write a program to simulate the two-level directory structure used for file organization in operating systems, and to perform basic file operations such as create, delete, search, and display within this structure.

LEARNING OUTCOME

- Understand the concept of directory structures used in file management systems.
- Understand the limitations of a single-level directory and how a two-level directory overcomes them.
- Learn how each user is given a separate User File Directory (UFD) under a Master File Directory (MFD).
- Gain the ability to implement basic directory operations such as create, delete, and search programmatically.
- Develop an understanding of how the operating system resolves path names in a two-level directory system.

DESCRIPTION

In a single-level directory, all files of all users are stored in the same directory, which leads to naming conflicts when two users choose the same file name, and also lacks grouping of files by user. The two-level directory structure overcomes this limitation by giving each user their own User File Directory (UFD).

At the top of the structure is a single Master File Directory (MFD), which is indexed by user name or account number. Each entry in the MFD points to the UFD of that particular user. When a user refers to a file, only their own UFD is searched, so different users may have files with the same name without any conflict. However, users cannot directly access another user's files unless explicit permission or a path is specified, since there is no direct cooperation or sharing between directories at this level.

This structure solves the name-collision problem of single-level directories but still has limitations, such as the inability of users to group their own files into sub-categories, which is later solved by tree-structured (hierarchical) directories.

ALGORITHM

1. Start the program.
2. Create a Master File Directory (MFD) to hold the list of registered users.
3. For each user, create a separate User File Directory (UFD) linked to the MFD.

4. Display a menu with options: Create User, Create File, Delete File, Search File, Display Directory, and Exit.
5. If 'Create User' is selected, add a new user name to the MFD and initialize an empty UFD for that user.
6. If 'Create File' is selected, accept the user name and file name, check whether the user exists in the MFD, and if so, add the file name to that user's UFD (only if it does not already exist).
7. If 'Delete File' is selected, accept the user name and file name, locate the file in the corresponding UFD, and remove it if found.
8. If 'Search File' is selected, accept the user name and file name, and search only within that user's UFD; display whether the file exists.
9. If 'Display Directory' is selected, list all users in the MFD along with the files present in each of their UFDs.
10. Repeat steps 4 to 9 until the user selects 'Exit'.
11. Stop the program.

PROGRAM

```
#include<stdio.h>

struct st
{
char dname[10];
char sname[10][10]; char fname[10][10][10];
int ds,sds[10];
}dir[10];

void main()
{
int i,j,k,n;
clrscr();
printf("enter number of directories:"); scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("enter directory %d names:",i+1);
scanf("%s",&dir[i].dname); printf("enter size of directories:");
scanf("%d",&dir[i].ds); for(j=0;j<dir[i].ds;j++)
{
printf("enter subdirectory name and size:"); scanf("%s",&dir[i].sname[j]);
```

```

scanf("%d",&dir[i].sds[j]);
for(k=0;k<dir[i].sds[j];k++)
{
printf("enter file name:");
scanf("%s",&dir[i].fname[j][k]);
}
Dept. Of CSE, VASTAdvanced Research Lab
}
}
printf("\ndirname\t\tsize\t\tsubdirname\t\tsize\t\tfiles");
printf("\n*****\n");
for(i=0;i<n;i++)
{
printf("%s\t\t%d",dir[i].dname,dir[i].ds);
for(j=0;j<dir[i].ds;j++)
{
printf("\t\t%s\t\t%d\t\t",dir[i].sdname[j],dir[i].sds[j]);
for(k=0;k<dir[i].sds[j];k++)
printf("%s\t",dir[i].fname[j][k]);
printf("\n\t\t");
}
printf("\n");
}}

```

PROCEDURE

1. Open a text editor or IDE and create a new source file for the program (e.g., using C, C++, Java, or Python).
2. Define data structures to represent the MFD (a list/array of users) and, for each user, a UFD (a list/array of file names).
3. Implement functions/modules for each menu operation: create user, create file, delete file, search file, and display directory structure.
4. Write the main function to display the menu in a loop and call the appropriate function based on user choice.
5. Compile the program and correct any syntax errors.
6. Execute the program and create a few sample users (e.g., User1, User2).

7. Under each user, create a few files, including files with the same name under different users, to demonstrate that no naming conflict occurs.
8. Test the delete and search operations for files within a specific user's directory.
9. Use the 'Display Directory' option to view the complete two-level directory structure.
10. Note down the outputs at each stage and verify that the results match the expected behaviour of a two-level directory system.

OUTPUT

The program successfully simulates a two-level directory structure. Sample output is illustrated below:

```
Enter number of users: 2 Enter user name: User1 Enter user name: User2 --- MENU --- 1.Create File 2.Delete File 3.Search File 4.Display Directory 5.Exit Enter choice: 1 Enter user name: User1 Enter file name: report.txt File 'report.txt' created under User1 Enter choice: 1 Enter user name: User2 Enter file name: report.txt File 'report.txt' created under User2 (No conflict — same file name exists independently under different users) Enter choice: 4 MFD: User1 -> report.txt User2 -> report.txt Enter choice: 3 Enter user name: User1 Enter file name: report.txt File found under User1 Enter choice: 5 Exiting program.
```

MODEL VIVA QUESTIONS

1. What is a two-level directory structure, and how does it differ from a single-level directory structure?
2. What is the role of the Master File Directory (MFD) and the User File Directory (UFD) in this structure?
3. How does the two-level directory structure solve the file naming conflict problem faced in single-level directories?
4. Can a user in a two-level directory system directly access files belonging to another user? Justify your answer.
5. What are the limitations of the two-level directory structure, and how are they addressed by the tree-structured directory system?

RESULT

The program to simulate the two-level directory file organization technique was successfully written, executed, and verified. All operations — creating users, creating files, deleting files, searching files, and displaying the directory structure — were performed correctly, and the output confirmed that files with identical names under different users are handled without conflict.

C) SIMULATION OF HIERARCHICAL (TREE-STRUCTURED) DIRECTORY FILE ORGANIZATION TECHNIQUE

AIM

To write a program to simulate the hierarchical (tree-structured) directory structure used for file organization in operating systems, and to perform basic operations such as creating directories and sub-directories, creating files, deleting files, searching files using path names, and displaying the directory tree.

LEARNING OUTCOME

- Understand the concept of a hierarchical or tree-structured directory in file management systems.
- Understand how the hierarchical structure overcomes the grouping limitation of the two-level directory.
- Learn the concepts of root directory, sub-directories, parent directory, and current working directory.
- Understand the use of absolute and relative path names to locate files within the tree.
- Gain the ability to implement directory and file operations such as create, delete, search, and display programmatically using a tree data structure.

DESCRIPTION

The two-level directory structure solves the file-naming conflict between users but does not allow a user to group their own files according to different purposes or projects. The hierarchical (tree-structured) directory structure removes this limitation by allowing users to create their own sub-directories and organize their files within them, just as in modern operating systems such as Windows, Linux, and UNIX.

In this structure, there is a single root directory at the top of the tree. Every directory can contain files as well as further sub-directories, and every file or directory (except the root) has exactly one parent directory. This forms a tree, where directories are the internal nodes and files are typically the leaf nodes, although a directory itself may also be treated as a special type of file.

Each user has a current working directory, and files can be referred to either by an absolute path name, which starts from the root and lists every directory down to the file, or by a relative path name, which starts from the current working directory. This structure allows efficient searching, grouping of related files, and permits different users to have directories and files with the same name as long as they lie in different parts of the tree.

ALGORITHM

1. Start the program.

2. Create a root directory as the topmost node of the tree.
3. Represent each node of the tree as a directory or a file, where a directory node can further contain child directories and files.
4. Display a menu with options: Create Directory, Create File, Delete File/Directory, Search File, Change Directory, Display Tree, and Exit.
5. If 'Create Directory' is selected, accept the name of the new sub-directory and add it as a child of the current directory.
6. If 'Create File' is selected, accept the file name and add it as a child (leaf node) of the current directory.
7. If 'Delete File/Directory' is selected, accept the name, locate it under the current directory, and remove it (and all its children, if it is a directory) from the tree.
8. If 'Search File' is selected, accept a path name (absolute or relative) and traverse the tree accordingly to check whether the file exists.
9. If 'Change Directory' is selected, accept the target directory name and update the current working directory pointer to that node, if it exists.
10. If 'Display Tree' is selected, recursively traverse the tree from the root and display the complete directory structure with proper indentation to show hierarchy.
11. Repeat steps 4 to 10 until the user selects 'Exit'.
12. Stop the program.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_CHILDREN 20
#define MAX_NAME 30

typedef struct Node {
    char name[MAX_NAME];
    int isFile;          /* 1 = file, 0 = directory */
    struct Node *parent;
    struct Node *children[MAX_CHILDREN];
    int childCount;
} Node;

Node *root;
Node *current;

Node* createNode(char *name, int isFile, Node *parent) {
```

```
Node *n = (Node*) malloc(sizeof(Node));
strcpy(n->name, name);
n->isFile = isFile;
n->parent = parent;
n->childCount = 0;
return n;
}

Node* findChild(Node *dir, char *name) {
    int i;
    for (i = 0; i < dir->childCount; i++) {
        if (strcmp(dir->children[i]->name, name) == 0)
            return dir->children[i];
    }
    return NULL;
}

void createDirectory(char *name) {
    if (findChild(current, name) != NULL) {
        printf("'%s' already exists here.\n", name);
        return;
    }
    if (current->childCount >= MAX_CHILDREN) {
        printf("Cannot create more entries here.\n");
        return;
    }
    Node *n = createNode(name, 0, current);
    current->children[current->childCount++] = n;
    printf("Directory '%s' created under %s\n", name, current->name);
}

void createFile(char *name) {
    if (findChild(current, name) != NULL) {
        printf("'%s' already exists here.\n", name);
        return;
    }
    if (current->childCount >= MAX_CHILDREN) {
        printf("Cannot create more entries here.\n");
        return;
    }
    Node *n = createNode(name, 1, current);
    current->children[current->childCount++] = n;
    printf("File '%s' created under %s\n", name, current->name);
}
```

```
void deleteEntry(char *name) {
    int i, pos = -1;
    for (i = 0; i < current->childCount; i++) {
        if (strcmp(current->children[i]->name, name) == 0) {
            pos = i;
            break;
        }
    }
    if (pos == -1) {
        printf("'%s' not found under %s\n", name, current->name);
        return;
    }
    free(current->children[pos]);
    for (i = pos; i < current->childCount - 1; i++)
        current->children[i] = current->children[i + 1];
    current->childCount--;
    printf("'%s' deleted from %s\n", name, current->name);
}
```

```
Node* searchPath(char *path) {
    Node *start = (path[0] == '/') ? root : current;
    char temp[200];
    strcpy(temp, path);
    char *token = strtok(temp, "/");
    Node *ptr = start;
    while (token != NULL) {
        if (strcmp(token, root->name) == 0 && ptr == root) {
            token = strtok(NULL, "/");
            continue;
        }
        Node *next = findChild(ptr, token);
        if (next == NULL)
            return NULL;
        ptr = next;
        token = strtok(NULL, "/");
    }
    return ptr;
}
```

```
void changeDirectory(char *name) {
    Node *target = findChild(current, name);
    if (target == NULL || target->isFile) {
        printf("Directory '%s' not found.\n", name);
        return;
    }
}
```

```
    current = target;
    printf("Current directory changed to: %s\n", current->name);
}

void displayTree(Node *node, int depth) {
    int i;
    for (i = 0; i < depth; i++)
        printf(" ");
    printf("%s%s\n", node->name, node->isFile ? "" : "/");
    if (!node->isFile) {
        for (i = 0; i < node->childCount; i++)
            displayTree(node->children[i], depth + 1);
    }
}

int main() {
    int choice;
    char name[MAX_NAME], path[200];

    root = createNode("root", 0, NULL);
    current = root;

    do {
        printf("\nCurrent directory: %s\n", current->name);
        printf("--- MENU ---\n");
        printf("1.Create Directory 2.Create File 3.Delete 4.Search\n");
        printf("5.Change Directory 6.Display Tree 7.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter directory name: ");
                scanf("%s", name);
                createDirectory(name);
                break;
            case 2:
                printf("Enter file name: ");
                scanf("%s", name);
                createFile(name);
                break;
            case 3:
                printf("Enter name to delete: ");
                scanf("%s", name);
                deleteEntry(name);
```

```
        break;
    case 4:
        printf("Enter path: ");
        scanf("%s", path);
        if (searchPath(path) != NULL)
            printf("Found: %s\n", path);
        else
            printf("Not found: %s\n", path);
        break;
    case 5:
        printf("Enter directory name: ");
        scanf("%s", name);
        changeDirectory(name);
        break;
    case 6:
        printf("Display Tree:\n");
        displayTree(root, 0);
        break;
    case 7:
        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice.\n");
    }
} while (choice != 7);

return 0;
}
```

PROCEDURE

1. Open a text editor or IDE and create a new source file for the program (e.g., using C, C++, Java, or Python).
2. Define a tree node structure containing a name, a type field (file or directory), a pointer/reference to its parent, and a list of children.
3. Initialize the root node as the starting point of the tree and set it as the current working directory.
4. Implement functions/modules for each menu operation: create directory, create file, delete, search (using path names), change directory, and display tree.
5. Write the main function to display the menu in a loop and call the appropriate function based on the user's choice.
6. Compile the program and correct any syntax errors.

7. Execute the program and create a small directory tree, for example a root with sub-directories such as 'Documents' and 'Photos', and further sub-directories or files inside them.
8. Test the search operation using both absolute path names (from the root) and relative path names (from the current directory).
9. Test the delete operation on a file and on a sub-directory containing files, and verify that deletion behaves correctly.
10. Use the 'Display Tree' option to view the complete hierarchical structure and verify it against the operations performed, and note down the results.

OUTPUT

The program successfully simulates a hierarchical directory structure. Sample output is illustrated below:

```
Current directory: root --- MENU --- 1.Create Directory 2.Create File 3.Delete 4.Search
5.Change Directory 6.Display Tree 7.Exit Enter choice: 1 Enter directory name: Documents
Directory 'Documents' created under root Enter choice: 1 Enter directory name: Photos
Directory 'Photos' created under root Enter choice: 5 Enter directory name: Documents Current
directory changed to: root/Documents Enter choice: 2 Enter file name: resume.docx File
'resume.docx' created under root/Documents Enter choice: 4 Enter path:
/root/Documents/resume.docx File found at root/Documents/resume.docx Enter choice: 6
Display Tree: root Documents resume.docx Photos Enter choice: 7 Exiting program.
```

MODEL VIVA QUESTIONS

1. What is a hierarchical (tree-structured) directory, and how does it differ from a two-level directory structure?
2. What is the difference between an absolute path name and a relative path name? Give an example of each.
3. What is meant by the 'current working directory', and why is it useful in a hierarchical structure?
4. How does the operating system delete a directory that contains sub-directories and files within a tree-structured directory system?
5. What are the advantages of a hierarchical directory structure over single-level and two-level directory structures?

RESULT

The program to simulate the hierarchical (tree-structured) directory file organization technique was successfully written, executed, and verified. All operations — creating directories and files, deleting them, searching using absolute and relative path names, changing the current directory, and displaying the directory tree — were performed correctly, confirming proper grouping and organization of files within nested sub-directories.

EXPNO.5**IMPLEMENTATION OF THE BANKER'S ALGORITHM FOR DEADLOCK AVOIDANCE****AIM**

To write a program to implement the Banker's Algorithm for deadlock avoidance, which checks whether a given resource allocation state of the system is in a safe state, and if so, determines a safe sequence in which processes can be executed without leading to deadlock.

LEARNING OUTCOME

- Understand the concept of deadlock and the difference between deadlock prevention, avoidance, and detection.
- Understand the notion of a 'safe state' and an 'unsafe state' in resource allocation.
- Learn the data structures used by the Banker's Algorithm: Allocation, Max, Available, and Need matrices.
- Understand how the Safety Algorithm determines whether a safe sequence of process execution exists.
- Learn how the Resource-Request Algorithm decides whether a process's request for resources can be granted immediately without risking deadlock.

DESCRIPTION

The Banker's Algorithm, proposed by Edsger Dijkstra, is a deadlock-avoidance algorithm used in operating systems that allocate multiple instances of several resource types among multiple processes. Its name comes from the analogy of a banker who never allocates cash in a way that would leave the bank unable to satisfy the needs of all its customers.

The algorithm works with four data structures: the Available vector, which indicates the number of available instances of each resource type; the Max matrix, which indicates the maximum demand of each process for each resource type; the Allocation matrix, which indicates the number of resources of each type currently allocated to each process; and the Need matrix, which indicates the remaining resource need of each process, computed as $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$.

Before granting a resource request, the operating system uses the Banker's Algorithm to simulate the allocation and then runs the Safety Algorithm to check whether the resulting state is safe, that is, whether there exists at least one sequence in which all processes can complete execution using the available resources without waiting indefinitely. If such a sequence exists, the request is granted; otherwise, the process must wait, even if the resources requested are currently available, since granting the request could otherwise lead the system into an unsafe state and eventually into deadlock.

ALGORITHM

1. Start the program.
2. Input the number of processes (n) and the number of resource types (m).
3. Input the Allocation matrix, representing resources currently allocated to each process, and the Max matrix, representing the maximum demand of each process.
4. Input the Available vector, representing the number of available instances of each resource type.
5. Compute the Need matrix using the formula $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$ for every process i and resource type j.
6. Initialize a Work vector equal to Available, and a Boolean Finish array of size n, with all values set to false.
7. Find an index i such that Finish[i] is false and Need[i] is less than or equal to Work (element-wise, for all resource types).
8. If such a process i is found, add its Allocation to Work ($\text{Work} = \text{Work} + \text{Allocation}[i]$), set Finish[i] to true, add i to the safe sequence, and repeat step 7 for the remaining processes.
9. If no such process can be found and all processes have Finish[i] = true, the system is in a safe state; print the safe sequence obtained.
10. If no such process can be found and some Finish[i] remains false, the system is in an unsafe state, and deadlock avoidance is not possible for the current state.
11. Stop the program.

PROGRAM

```
#include<stdio.h>

int alloc[10][10],max[10][10],avl[10],need[10][10],s,i,j,res[10],avlr[10];

void read(int a[10][10],int m ,int n)
{
for (i=0;i<m;i++)
{
for(j=0;j<n;j++)
scanf("%d",&a[i][j]);
}
}

void write (int a[10][10],int m,int n)
{
for(i=0;i<m;i++)
```

```
{
printf(“%d”,i+1);
for (j=0;j<n;j++)
{
printf(“%d”,a[i][j]);
}
}
}
}
void check1(int m,int n)
Dept. Of CSE, VASTAdvanced Research Lab
{
int f1=0,f2=0,f3=0,count;
do
{
count=0;
for(i=0;i<m;i++)
{
f1=f2=f3=0;
for(j=0;j<n;j++)
{
if(need[i][j]>avl[r[j])
{
break;
}
else if (need[i][j]==0)
f2++;
f3++;
if(f2==n)
break;
}
if(f3==n &&f2!=n)
{
for(j=0;j<n;j++)
{
```

```
need[i][j]=0;
avl[j]+=alloc[i][j];
}
printf("\n Process %d",i);
count ++;
}
}
}
while(count >0)
}
int check(int m,int n)
{
for(i=0;i<m;i++)
for(j=0;j<n;j++)
if (need[i][j]!=0)
return 0;
return 1;
}
int main()
{
int m,n,c,ch=1,f1=0,f2,count=0,f3,s1;
printf("Enter the number of Process & Resource:");
scanf("%d%d",&m,&n);
printf("Enter maximum allocation matrix:");
read(max,m,n);
printf("Enter allocated values:");
read(alloc,m,n);
f1=count=0;
printf("Enter maximum available resources:");
for(i=0;i<n;i++)
scanf("%d",&avl[i]);
for(i=0;i<m;i++)
for(j=0;j<n;j++)
need[i][j]=max[i][j]-alloc[i][j];
```

```
printf("maximum allocation matrix:");
write(max,m,n);
printf("Allocated matrix:");
write(alloc,m,n);
printf("need matrix:");
write(need,m,n);
for(i=0;i<n;i++)
for(j=0;j<m;j++)
res[i]+=alloc[j][i];
printf("Total resource already allocated");
for(i=0;i<n;i++)
printf("%d",res[i]);
for(i=0;i<n;i++)
avl[i]=avl[i]-res[i];
printf("Available resource count:");
for(i=0;i<n;i++)
printf("%d",avl[i]);
check1(m,n);
s=check(m,n);
if(s==1)
printf("Safe state");
else
printf("unsafe state");
}
```

PROCEDURE

1. Open a text editor or IDE and create a new source file for the program (e.g., using C, C++, Java, or Python).
2. Declare and initialize the Allocation, Max, Need, and Available data structures as two-dimensional arrays (matrices) and a one-dimensional array respectively.
3. Implement a function to compute the Need matrix from the Allocation and Max matrices.
4. Implement the Safety Algorithm as a function that returns whether the current state is safe, and if so, outputs the safe sequence.

5. Implement the Resource-Request Algorithm as a function that, given a request from a process, tentatively updates Available, Allocation, and Need, checks safety using the Safety Algorithm, and either grants or rolls back the request accordingly.
6. Write the main function to accept input values for the number of processes, number of resource types, Allocation, Max, and Available, and to call the safety-checking function.
7. Compile the program and correct any syntax errors.
8. Execute the program using a standard test case (e.g., 5 processes and 3 resource types) and verify that the safe sequence produced matches the expected result.
9. Test the program with a resource-request scenario, both for a request that should be granted and one that should be denied, to confirm correct behaviour.
10. Modify the input values to create an unsafe state and verify that the program correctly reports that no safe sequence exists.

OUTPUT

The program successfully implements the Banker's Algorithm. Sample output is illustrated below:

```
Enter number of processes: 5 Enter number of resource types: 3 Enter Allocation matrix: P0: 0 1
0 P1: 2 0 0 P2: 3 0 2 P3: 2 1 1 P4: 0 0 2 Enter Max matrix: P0: 7 5 3 P1: 3 2 2 P2: 9 0 2 P3: 2 2
2 P4: 4 3 3 Enter Available vector: 3 3 2 Need Matrix: P0: 7 4 3 P1: 1 2 2 P2: 6 0 0 P3: 0 1 1 P4:
4 3 1 Running Safety Algorithm... System is in a SAFE state. Safe sequence: P1 -> P3 -> P4 ->
P0 -> P2
```

MODEL VIVA QUESTIONS

1. What is deadlock, and what are the four necessary conditions for deadlock to occur?
2. What is the difference between a safe state and an unsafe state? Does an unsafe state always lead to deadlock?
3. What are the Allocation, Max, Need, and Available data structures used in the Banker's Algorithm, and what does each represent?
4. How does the Banker's Algorithm differ from deadlock detection and deadlock prevention approaches?
5. What is the time complexity of the Safety Algorithm, and why is the Banker's Algorithm considered impractical for some real systems?

RESULT

The program to implement the Banker's Algorithm for deadlock avoidance was successfully written, executed, and verified. The Need matrix was correctly computed from the Allocation and Max matrices, the Safety Algorithm correctly identified a safe sequence for the given resource allocation state, and resource requests were correctly granted or denied based on whether the resulting state remained safe.

EXPNO.6**A)SIMULATE THE FOLLOWING DISK SCHEDULING ALGORITHMS.-FCFS****AIM**

To write a program to simulate the First-Come-First-Served (FCFS) disk scheduling algorithm, and to calculate the total head movement (seek time) required to service a given sequence of disk I/O requests.

LEARNING OUTCOME

- Understand the need for disk scheduling algorithms in operating systems.
- Understand how the FCFS disk scheduling algorithm services requests in the order in which they arrive.
- Learn how to compute the total head movement for a given request sequence and initial head position.
- Understand the advantages and drawbacks of FCFS compared to other disk scheduling algorithms such as SSTF, SCAN, and C-SCAN.
- Gain the ability to represent and trace disk head movement programmatically and graphically.

DESCRIPTION

A disk drive services I/O requests to read or write data at different cylinders (tracks), and since the disk head must physically move across the disk surface, the order in which requests are serviced greatly affects performance. Disk scheduling algorithms decide the order of servicing pending requests so as to minimize total head movement, which in turn reduces average seek time and improves throughput.

The First-Come-First-Served (FCFS) algorithm is the simplest disk scheduling algorithm. It services disk requests strictly in the order in which they arrive in the request queue, without any reordering based on cylinder position. While this makes FCFS fair and easy to implement, it does not attempt to minimize head movement, so if the requests are scattered across the disk, the head may move back and forth erratically, resulting in a large total seek time compared to algorithms such as SSTF, SCAN, or C-SCAN.

The total head movement is calculated by taking the absolute difference between the current head position and the next request in the queue, moving the head to that position, and repeating this process for every request in the given order, finally summing all the individual movements.

ALGORITHM

1. Start the program.
2. Input the number of disk requests (n) and the sequence of cylinder numbers to be accessed, in the order they arrive.

3. Input the initial position of the disk head.
4. Initialize a variable totalHeadMovement to 0, and set currentPosition to the initial head position.
5. For each request in the queue, in the given order, compute the absolute difference between currentPosition and the request's cylinder number.
6. Add this difference to totalHeadMovement, and update currentPosition to the cylinder number of the request just serviced.
7. Repeat step 5 and 6 for all remaining requests in the queue in their original order.
8. After all requests have been serviced, display the order in which the requests were serviced (same as input order) and the total head movement.
9. Optionally, plot the sequence of head positions against time/request order to visualize the seek pattern.
10. Stop the program.

PROGRAM

```
#include<stdio.h>

main()
{
int t[20], n, l, j, tohm[20], tot=0; float avhm;
clrscr();
printf("enter the no.of tracks"); scanf("%d",&n);
printf("enter the tracks to be traversed"); for(i=2;i<n+2;i++)
scanf("%d",&t*i+); for(i=1;i<n+1;i++)
{
tohm[i]=t[i+1]-t[i]; if(tohm[i]<0)
tohm[i]=tohm[i]*(-1);
}
for(i=1;i<n+1;i++) tot+=tohm[i];
avhm=(float)tot/n;
printf("Tracks traversed\tDifference
between tracks\n"); for(i=1;i<n+1;i++)
printf("%d\t\t%d\n",t*i+,tohm*i+); printf("\nAverage header movements:
%f",avhm);
}
```

PROCEDURE

1. Open a text editor or IDE and create a new source file for the program (e.g., using C, C++, Java, or Python).
2. Declare an array to store the sequence of disk requests and variables for the initial head position and total head movement.
3. Implement a function/module that iterates through the request array in order and accumulates the absolute difference between consecutive positions.
4. Write the main function to accept the number of requests, the request sequence, and the initial head position as input.
5. Call the FCFS servicing function and store the total head movement computed.
6. Compile the program and correct any syntax errors.
7. Execute the program using a standard test case (e.g., request queue = 98, 183, 37, 122, 14, 124, 65, 67 with initial head position 53) and note the servicing order and total head movement.
8. Verify the result manually by computing the absolute differences step by step and cross-checking with the program's output.
9. Re-run the program with a different initial head position and request sequence to observe how the total head movement changes.
10. Record and tabulate the results for comparison with other disk scheduling algorithms, if required.

OUTPUT

The program successfully simulates the FCFS disk scheduling algorithm. Sample output is illustrated below:

Enter number of requests: 8 Enter request sequence: 98 183 37 122 14 124 65 67 Enter initial head position: 53 Order of servicing (FCFS): 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67 Head movement calculation: $|53-98| = 45$ $|98-183| = 85$ $|183-37| = 146$ $|37-122| = 85$ $|122-14| = 108$ $|14-124| = 110$ $|124-65| = 59$ $|65-67| = 2$ Total Head Movement = 640

MODEL VIVA QUESTIONS

1. What is disk scheduling, and why is it required in operating systems?
2. How does the FCFS disk scheduling algorithm work, and how is the total head movement calculated?
3. What are the advantages and disadvantages of the FCFS disk scheduling algorithm?
4. How does FCFS compare with SSTF (Shortest Seek Time First) in terms of total head movement and fairness?
5. Does the order of arrival of requests affect the performance of FCFS? Justify your answer with an example.

RESULT

The program to simulate the First-Come-First-Served (FCFS) disk scheduling algorithm was successfully written, executed, and verified. The disk requests were serviced strictly in their order of arrival, and the total head movement was correctly computed and matched the manually verified calculation.

B) SIMULATE THE FOLLOWING DISK SCHEDULING ALGORITHMS.-SCAN**AIM**

To write a program to simulate the SCAN (Elevator) disk scheduling algorithm, and to calculate the total head movement (seek time) required to service a given sequence of disk I/O requests.

LEARNING OUTCOME

- Understand the limitations of FCFS and SSTF disk scheduling algorithms that motivate the SCAN algorithm.
- Understand how the SCAN algorithm moves the disk head in one direction, servicing requests along the way, before reversing direction at the end of the disk.
- Learn how to compute the total head movement for a given request sequence, initial head position, and direction of movement.
- Understand why SCAN is often called the 'elevator algorithm' due to its similarity to the working of a building elevator.
- Gain the ability to implement and trace the SCAN algorithm programmatically and compare its performance with FCFS and SSTF.

DESCRIPTION

The SCAN disk scheduling algorithm, also known as the Elevator algorithm, moves the disk head in one direction across the disk, servicing all pending requests that lie in that direction, until it reaches the end (or the last request in that direction), and then reverses direction and services the remaining requests on the way back, much like an elevator that services all requested floors in one direction before reversing.

Unlike FCFS, which services requests strictly in arrival order, or SSTF, which always jumps to the nearest request and can cause starvation of far-away requests, SCAN guarantees that every pending request is eventually serviced within one full sweep of the disk, giving it more predictable and generally lower total head movement, since the head does not move back and forth erratically.

To simulate SCAN, the requests are first sorted, and the head moves toward one end of the disk (for example, toward the higher cylinder numbers), servicing every request encountered along the way, until it reaches the last cylinder (or the highest requested cylinder, depending on the variant used), then reverses direction and services the remaining requests while moving toward the other end. The total head movement is the sum of the distances covered during this sweep in both directions.

ALGORITHM

1. Start the program.
2. Input the number of disk requests (n) and the sequence of cylinder numbers to be accessed.

3. Input the initial position of the disk head, the direction of movement (towards higher or lower cylinder numbers), and the maximum disk size (number of cylinders, e.g., 0 to 199).
4. Sort the request array in ascending order of cylinder numbers.
5. Based on the initial head position, divide the requests into two groups: those with cylinder numbers greater than the head position, and those with cylinder numbers less than the head position.
6. If the direction is towards higher cylinder numbers, service all requests greater than the head position in increasing order first, moving the head up to the maximum end of the disk (if required by the SCAN variant), then reverse direction and service all requests less than the head position in decreasing order.
7. If the direction is towards lower cylinder numbers, service all requests less than the head position in decreasing order first, moving the head down to 0, then reverse direction and service all requests greater than the head position in increasing order.
8. At each step, compute the absolute difference between the current head position and the next cylinder to be serviced, and add it to totalHeadMovement, updating the current head position after each move.
9. After all requests have been serviced, display the order of servicing and the total head movement.
10. Stop the program.

PROGRAM

```
#include<stdio.h>

main()
{
int t[20], d[20], h, i, j, n, temp, k, atr[20], tot, p, sum=0; clrscr();
printf("enter the no of tracks to be traversed"); scanf("%d",&n);
printf("enter the position of head"); scanf("%d",&h);
t[0]=0;
t[1]=h;
printf("enter the tracks"); for(i=2;i<n+2;i++)
scanf("%d",&t[i]); for(i=0;i<n+2;i++)
{
for(j=0;j<(n+2)-i-1;j++)
{
if(t[j]>t[j+1])
{
```

```
temp=t[j]; t[j]=t[j+1]; t[j+1]=temp;
}
}
}
for(i=0;i<n+2;i++)
if(t[i]==h)
j=i;
k=i;
p=0;
while(t[j]!=0)
{
atr[p]=t[j];
j--;
p++;
}
atr[p]=t[j];
for(p=k+1;p<n+2;p++,k++)
atr[p]=t[k+1];
for(j=0;j<n+1;j++)
{
if(atr[j]>atr[j+1])
d[j]=atr[j]-atr[j+1];
else
d[j]=atr[j+1]-atr[j];
sum+=d[j];
}
printf("\nAverage header movements:%f", (float)sum/n);
getch();
}
```

PROCEDURE

1. Open a text editor or IDE and create a new source file for the program (e.g., using C, C++, Java, or Python).

2. Declare an array to store the disk request sequence, and variables for the initial head position, direction, disk size, and total head movement.
3. Implement a sorting function (or use a built-in sort) to arrange the requests in ascending order of cylinder number.
4. Implement a function that splits the sorted requests into two sub-lists based on the head position, and services them in the correct order according to the chosen direction.
5. Write the main function to accept the number of requests, the request sequence, the initial head position, the direction, and the disk size as input.
6. Call the SCAN servicing function and store the total head movement and servicing order computed.
7. Compile the program and correct any syntax errors.
8. Execute the program using a standard test case (e.g., request queue = 98, 183, 37, 122, 14, 124, 65, 67 with initial head position 53, moving towards higher cylinder numbers, disk size 0–199) and note the servicing order and total head movement.
9. Verify the result manually by tracing the head movement step by step and cross-checking with the program's output.
10. Re-run the program with a different initial head position and direction to observe how the total head movement changes, and compare the results with FCFS and SSTF for the same request sequence.

OUTPUT

The program successfully simulates the SCAN disk scheduling algorithm. Sample output is illustrated below:

```
Enter number of requests: 8 Enter request sequence: 98 183 37 122 14 124 65 67 Enter initial
head position: 53 Enter direction (towards higher cylinder numbers): Yes Disk size: 0 to 199
Sorted requests: 14 37 65 67 98 122 124 183 Order of servicing (SCAN): 53 -> 65 -> 67 -> 98 -
> 122 -> 124 -> 183 -> 199 -> 37 -> 14 Head movement calculation: |53-65| = 12 |65-67| = 2
|67-98| = 31 |98-122| = 24 |122-124| = 2 |124-183| = 59 |183-199| = 16 |199-37| = 162 |37-14| =
23 Total Head Movement = 331
```

MODEL VIVA QUESTIONS

1. What is the SCAN disk scheduling algorithm, and why is it also called the 'elevator algorithm'?
2. How does SCAN differ from FCFS and SSTF in terms of head movement pattern and fairness to requests?
3. Does the SCAN algorithm always move the head all the way to the end of the disk, even if there are no requests near that end? Explain.
4. What is the difference between the SCAN algorithm and the C-SCAN (Circular SCAN) algorithm?

5. What is 'starvation' in the context of disk scheduling, and how does SCAN help reduce it compared to SSTF?

RESULT

The program to simulate the SCAN (Elevator) disk scheduling algorithm was successfully written, executed, and verified. The disk requests were sorted and serviced in the order dictated by the sweep of the disk head in the chosen direction, and the total head movement was correctly computed and matched the manually verified calculation.

C) SIMULATE THE FOLLOWING DISK SCHEDULING ALGORITHMS.C-SCAN**AIM**

To write a program to simulate the C-SCAN (Circular SCAN) disk scheduling algorithm, and to calculate the total head movement (seek time) required to service a given sequence of disk I/O requests.

LEARNING OUTCOME

- Understand the limitation of the SCAN algorithm that motivates the C-SCAN algorithm, namely the uneven waiting time for requests near the two ends of the disk.
- Understand how the C-SCAN algorithm treats the disk as a circular list, servicing requests only in one direction and then jumping back to the beginning without servicing requests on the return trip.
- Learn how to compute the total head movement for a given request sequence, initial head position, and disk size, including the return jump.
- Understand how C-SCAN provides more uniform wait times compared to SCAN.
- Gain the ability to implement and trace the C-SCAN algorithm programmatically and compare its performance with FCFS, SSTF, and SCAN.

DESCRIPTION

The C-SCAN (Circular SCAN) disk scheduling algorithm is a variation of the SCAN algorithm designed to provide a more uniform wait time for all disk requests. In the plain SCAN algorithm, requests near the middle of the disk are serviced more frequently than requests near either end, since the head reverses direction at the ends and sweeps back over the middle again almost immediately, while requests just serviced near an end must wait for almost two full sweeps before being serviced again.

C-SCAN addresses this by treating the disk as a circular list where the last cylinder is considered adjacent to the first cylinder. The disk head moves in a single direction only, for example from the current position towards the highest cylinder number, servicing all requests along the way. Upon reaching the last cylinder, instead of reversing direction and servicing requests on the way back as in SCAN, the head jumps immediately back to the first cylinder (cylinder 0) without servicing any requests during this return trip, and then continues moving in the same original direction, servicing the remaining requests.

This circular movement ensures that the time between successive visits to any given cylinder is roughly the same for all cylinders, making C-SCAN fairer than SCAN, although the return jump from the last cylinder to the first does add to the total head movement, since it is usually counted as part of the total seek distance.

ALGORITHM

1. Start the program.
2. Input the number of disk requests (n) and the sequence of cylinder numbers to be accessed.
3. Input the initial position of the disk head and the disk size (number of cylinders, e.g., 0 to 199), assuming the head always moves towards higher cylinder numbers.
4. Sort the request array in ascending order of cylinder numbers.
5. Divide the sorted requests into two groups: those with cylinder numbers greater than or equal to the head position, and those with cylinder numbers less than the head position.
6. Service all requests greater than or equal to the head position in increasing order, moving the head towards the highest cylinder number (disk size - 1).
7. Add the head movement from the last requested cylinder to the highest cylinder to totalHeadMovement, then add the jump distance from the highest cylinder back to cylinder 0 to totalHeadMovement (this return trip services no requests).
8. From cylinder 0, service all the remaining requests (those originally less than the head position) in increasing order.
9. At each step, compute the absolute difference between the current head position and the next cylinder to be serviced, and add it to totalHeadMovement, updating the current head position after each move.
10. After all requests have been serviced, display the order of servicing and the total head movement, and stop the program.

PROGRAM

```
#include<stdio.h> main()
{
int t[20], d[20], h, i, j, n, temp, k, atr[20], tot, p, sum=0; clrscr();
printf("enter the no of tracks to be traversed"); scanf("%d",&n);
printf("enter the position of head"); scanf("%d",&h);
t[0]=0;
t[1]=h;
printf("enter total tracks"); scanf("%d",&tot);
t[2]=tot-1;
printf("enter the tracks"); for(i=3;i<=n+2;i++)
scanf("%d",&t[i]); for(i=0;i<=n+2;i++)
for(j=0;j<=(n+2)-i-1;j++)
```

```
if(t[j]>t[j+1])
{
temp=t[j];
t[j]=t[j+1];
t[j+1]=temp;
}
for(i=0;i<=n+2;i++)
if(t[i]==h)
j=i;
break;
p=0;
while(t[j]!=tot-1)
{
atr[p]=t[j];
j++;
p++;
}
atr[p]=t[j];
p++;
i=0;
while(p!=(n+3) && t[i]!=t[h])
{
atr[p]=t[i];
i++;
p++;
}
for(j=0;j<n+2;j++)
{
if(atr[j]>atr[j+1])
d[j]=atr[j]-atr[j+1];
else
d[j]=atr[j+1]-atr[j];
sum+=d[j];
}
```

```
printf("total header movements%d",sum);
printf("avg is %f",(float)sum/n); }
```

PROCEDURE

1. Open a text editor or IDE and create a new source file for the program (e.g., using C, C++, Java, or Python).
2. Declare an array to store the disk request sequence, and variables for the initial head position, disk size, and total head movement.
3. Implement a sorting function (or use a built-in sort) to arrange the requests in ascending order of cylinder number.
4. Implement a function that splits the sorted requests into two sub-lists based on the head position, services the first sub-list up to the maximum cylinder, adds the circular jump back to cylinder 0, and then services the second sub-list.
5. Write the main function to accept the number of requests, the request sequence, the initial head position, and the disk size as input.
6. Call the C-SCAN servicing function and store the total head movement and servicing order computed.
7. Compile the program and correct any syntax errors.
8. Execute the program using a standard test case (e.g., request queue = 98, 183, 37, 122, 14, 124, 65, 67 with initial head position 53, disk size 0–199) and note the servicing order and total head movement.
9. Verify the result manually by tracing the head movement step by step, including the circular jump, and cross-checking with the program's output.
10. Re-run the program with a different initial head position to observe how the total head movement changes, and compare the results with SCAN for the same request sequence to note the difference caused by the circular jump.

OUTPUT

The program successfully simulates the C-SCAN disk scheduling algorithm. Sample output is illustrated below:

Enter number of requests: 8 Enter request sequence: 98 183 37 122 14 124 65 67 Enter initial head position: 53 Disk size: 0 to 199 Sorted requests: 14 37 65 67 98 122 124 183 Order of servicing (C-SCAN): 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> 37 Head movement calculation: $|53-65| = 12$ $|65-67| = 2$ $|67-98| = 31$ $|98-122| = 24$ $|122-124| = 2$ $|124-183| = 59$ $|183-199| = 16$ $|199-0| = 199$ (circular jump) $|0-14| = 14$ $|14-37| = 23$ Total Head Movement = 382

MODEL VIVA QUESTIONS

1. What is the C-SCAN disk scheduling algorithm, and how does it treat the disk as a circular structure?
2. How does C-SCAN differ from SCAN in terms of servicing requests during the return trip?
3. Why does C-SCAN provide more uniform wait times for all cylinders compared to SCAN?
4. Is the jump from the last cylinder back to cylinder 0 counted in the total head movement? Why or why not?
5. In what type of workload or system would C-SCAN be preferred over SCAN, FCFS, or SSTF?

RESULT

The program to simulate the C-SCAN (Circular SCAN) disk scheduling algorithm was successfully written, executed, and verified. The disk requests were serviced in one direction only, followed by a circular jump back to the beginning of the disk, and the total head movement, including the return jump, was correctly computed and matched the manually verified calculation.

EXPNO.7**A) SIMULATE THE FOLLOWING PAGE REPLACEMENT ALGORITHMS:-FIFO****AIM**

To write a program to simulate the First-In-First-Out (FIFO) page replacement algorithm, and to calculate the total number of page faults that occur for a given reference string and a given number of page frames.

LEARNING OUTCOME

- Understand the concept of virtual memory, paging, and the need for page replacement algorithms.
- Understand the meaning of a page fault and how it is counted for a given reference string.
- Learn how the FIFO page replacement algorithm selects the oldest page in memory for replacement when a page fault occurs and no free frame is available.
- Understand the concept of Belady's Anomaly and how it relates specifically to the FIFO algorithm.
- Gain the ability to trace and implement the FIFO algorithm programmatically for varying numbers of page frames.

DESCRIPTION

In a demand-paging operating system, when a process references a page that is not currently present in the main memory, a page fault occurs, and the operating system must bring the required page into memory, replacing an existing page if all available frames are already occupied. The strategy used to choose the page to be replaced is called a page replacement algorithm, and different algorithms aim to minimize the total number of page faults for a given sequence of memory references.

The First-In-First-Out (FIFO) page replacement algorithm is one of the simplest such algorithms. It maintains the pages currently in memory in a queue ordered by the time they were brought into memory. Whenever a page fault occurs and a frame must be freed, the page that has been in memory the longest, that is, the page at the front of the queue, is replaced, regardless of how frequently or recently it has been used. The newly loaded page is then added to the back of the queue.

While FIFO is easy to implement using a simple queue data structure, it does not consider the actual usage pattern of the pages, so it can sometimes replace a page that is about to be referenced again. FIFO is also notable for exhibiting Belady's Anomaly, an unusual situation in which increasing the number of available page frames can, for certain reference strings, actually increase the total number of page faults instead of decreasing them.

ALGORITHM

1. Start the program.
2. Input the length of the reference string and the sequence of page numbers to be referenced.
3. Input the number of available page frames (m).
4. Initialize an empty queue to represent the frames currently holding pages in memory, and set pageFaults to 0.
5. For each page in the reference string, check whether it is already present in the frames (queue).
6. If the page is already present, it is a page hit; move to the next page in the reference string without any change to the queue.
7. If the page is not present, it is a page fault; increment pageFaults by 1.
8. If the queue is not yet full (fewer than m pages are currently loaded), simply add the new page to the back of the queue.
9. If the queue is full, remove the page at the front of the queue (the oldest page) and add the new page to the back of the queue.
10. Repeat steps 5 to 9 for every page in the reference string, and after processing the entire string, display the total number of page faults.
11. Stop the program.

PROGRAM

```
#include <stdio.h>

#define MAX 50

int main() {
    int refString[MAX], frames[MAX], n, m, i, j, k;
    int pageFaults = 0, front = 0, filled = 0;
    int found;

    printf("Enter length of reference string: ");
    scanf("%d", &n);

    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &refString[i]);

    printf("Enter number of page frames: ");
```

```
scanf("%d", &m);

for (i = 0; i < m; i++)
    frames[i] = -1; /* -1 indicates an empty frame */

printf("\nRef\tFrames\t\t\tStatus\n");

for (i = 0; i < n; i++) {
    found = 0;

    /* check if the page is already present in the frames */
    for (j = 0; j < filled; j++) {
        if (frames[j] == refString[i]) {
            found = 1;
            break;
        }
    }

    if (found) {
        printf("%d\t", refString[i]);
        printf("[ ");
        for (k = 0; k < filled; k++)
            printf("%d ", frames[k]);
        printf("]\t\tHit\n");
    } else {
        pageFaults++;

        if (filled < m) {
            /* frames not yet full, insert at the end */
            frames[filled] = refString[i];
            filled++;
        } else {
            /* frames full, replace the oldest page (FIFO order) */
            frames[front] = refString[i];
            front = (front + 1) % m;
        }

        printf("%d\t", refString[i]);
        printf("[ ");
        for (k = 0; k < filled; k++)
            printf("%d ", frames[k]);
        printf("]\t\tFault\n");
    }
}
```

```
printf("\nTotal Page Faults = %d\n", pageFaults);

return 0;
}
```

PROCEDURE

1. Open a text editor or IDE and create a new source file for the program (e.g., using C, C++, Java, or Python).
2. Declare an array to store the reference string, and a queue (which can be implemented using an array or a linked list) to represent the current contents of the page frames.
3. Implement a function to check whether a given page is currently present in the queue of frames.
4. Implement the FIFO replacement logic: if the queue is full and a page fault occurs, remove the element at the front of the queue and insert the new page at the back.
5. Write the main function to accept the reference string, its length, and the number of page frames as input, and to call the FIFO simulation function while counting page faults.
6. Compile the program and correct any syntax errors.
7. Execute the program using a standard test case (e.g., reference string = 1 3 0 3 5 6 3 with 3 page frames) and note the frame contents at each step and the total number of page faults.
8. Verify the result manually by tracing the queue contents step by step for the same reference string and cross-checking with the program's output.
9. Re-run the program with a larger number of page frames for the same reference string to check whether Belady's Anomaly is exhibited (that is, whether the number of page faults increases instead of decreasing).
10. Record and tabulate the results for different frame sizes for comparison with other page replacement algorithms, if required.

OUTPUT

The program successfully simulates the FIFO page replacement algorithm. Sample output is illustrated below:

```
Enter length of reference string: 12 Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 Enter number
of page frames: 3 Ref Frames      Status 7  [7]      Fault 0  [7,0]      Fault 1  [7,0,1]
Fault 2  [0,1,2]      Fault 0  [0,1,2]      Hit 3   [1,2,3]      Fault 0  [2,3,0]      Fault 4  [3,0,4]
Fault 2  [0,4,2]      Fault 3   [4,2,3]      Fault 0  [2,3,0]      Fault 3  [2,3,0]      Hit Total
Page Faults = 10
```

MODEL VIVA QUESTIONS

1. What is a page fault, and under what circumstances does a page replacement algorithm come into play?
2. How does the FIFO page replacement algorithm decide which page to replace when a page fault occurs?
3. What is Belady's Anomaly, and why does it specifically affect the FIFO algorithm?
4. What is the time and space complexity of the FIFO page replacement algorithm?
5. How does FIFO compare with the Optimal (OPT) and LRU (Least Recently Used) page replacement algorithms in terms of performance?

RESULT

The program to simulate the First-In-First-Out (FIFO) page replacement algorithm was successfully written, executed, and verified. The page frames were correctly updated on each reference, the oldest page was replaced whenever a page fault occurred with full frames, and the total number of page faults was correctly computed and matched the manually verified calculation.

B) SIMULATE THE FOLLOWING PAGE REPLACEMENT ALGORITHMS:-LRU**AIM**

To write a program to simulate the Least Recently Used (LRU) page replacement algorithm, and to calculate the total number of page faults that occur for a given reference string and a given number of page frames.

LEARNING OUTCOME

- Understand the concept of virtual memory, paging, and the need for page replacement algorithms.
- Understand the meaning of a page fault and how it is counted for a given reference string.
- Learn how the LRU page replacement algorithm selects, for replacement, the page that has not been used for the longest period of time.
- Understand why LRU generally performs better than FIFO and does not suffer from Belady's Anomaly, since it approximates the optimal algorithm by using past reference behaviour.
- Gain the ability to trace and implement the LRU algorithm programmatically for varying numbers of page frames.

DESCRIPTION

In a demand-paging operating system, when a process references a page that is not currently present in the main memory, a page fault occurs, and the operating system must bring the required page into memory, replacing an existing page if all available frames are already occupied. The strategy used to choose the page to be replaced is called a page replacement algorithm, and different algorithms aim to minimize the total number of page faults for a given sequence of memory references.

The Least Recently Used (LRU) page replacement algorithm is based on the principle of locality of reference, which assumes that a page that has not been used for a long time is less likely to be used again in the near future. Whenever a page fault occurs and a frame must be freed, LRU replaces the page in memory that was least recently accessed, that is, the page whose most recent reference lies furthest back in time compared to the other pages currently in memory.

To implement LRU, the system needs to keep track of the time (or order) at which each page currently in memory was last referenced. This can be done using a counter/timestamp associated with each frame, a stack that is reordered on every reference, or other similar bookkeeping structures. Whenever a page already in memory is referenced again (a page hit), its last-used time is updated. When a page fault occurs and a replacement is required, the page with the oldest (smallest) last-used time is evicted. LRU generally performs closer to the optimal page replacement algorithm than FIFO and does not exhibit Belady's Anomaly, although it is more expensive to implement in hardware or software due to the need to continuously track usage recency.

ALGORITHM

1. Start the program.
2. Input the length of the reference string and the sequence of page numbers to be referenced.
3. Input the number of available page frames (m).
4. Initialize the frames to empty (e.g., -1) and a recentUse array, of the same size as the frames, to store the last-used time of each frame; set pageFaults to 0 and a time counter to 0.
5. For each page in the reference string, check whether it is already present in the frames.
6. If the page is already present, it is a page hit; update the recentUse value of that frame to the current time, and move to the next page without any replacement.
7. If the page is not present, it is a page fault; increment pageFaults by 1.
8. If the frames are not yet full (fewer than m pages are currently loaded), simply place the new page in the next free frame and set its recentUse value to the current time.
9. If the frames are full, scan the recentUse array to find the frame with the smallest (oldest) last-used time; replace the page in that frame with the new page and update its recentUse value to the current time.
10. Increment the time counter after processing each reference, and repeat steps 5 to 9 for every page in the reference string.
11. After processing the entire reference string, display the total number of page faults, and stop the program.

PROGRAM

```
#include <stdio.h>

#define MAX 50

int main() {
    int refString[MAX], frames[MAX], recentUse[MAX];
    int n, m, i, j, k, time = 0;
    int pageFaults = 0, filled = 0;
    int found, lruIndex, lruTime;

    printf("Enter length of reference string: ");
    scanf("%d", &n);

    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &refString[i]);

    printf("Enter number of page frames: ");
```

```
scanf("%d", &m);

for (i = 0; i < m; i++) {
    frames[i] = -1;    /* -1 indicates an empty frame */
    recentUse[i] = -1; /* time of last use of this frame */
}

printf("\nRef\tFrames\t\t\tStatus\n");

for (i = 0; i < n; i++) {
    found = 0;

    /* check if the page is already present in the frames */
    for (j = 0; j < filled; j++) {
        if (frames[j] == refString[i]) {
            found = 1;
            recentUse[j] = time; /* update last used time */
            break;
        }
    }

    if (found) {
        printf("%d\t", refString[i]);
        printf("[ ");
        for (k = 0; k < filled; k++)
            printf("%d ", frames[k]);
        printf("]\t\tHit\n");
    } else {
        pageFaults++;

        if (filled < m) {
            /* frames not yet full, insert at the end */
            frames[filled] = refString[i];
            recentUse[filled] = time;
            filled++;
        } else {
            /* frames full, find the Least Recently Used page */
            lruIndex = 0;
            lruTime = recentUse[0];
            for (j = 1; j < m; j++) {
                if (recentUse[j] < lruTime) {
                    lruTime = recentUse[j];
                    lruIndex = j;
                }
            }
        }
    }
}
```

```
        frames[lruIndex] = refString[i];
        recentUse[lruIndex] = time;
    }

    printf("%d\t", refString[i]);
    printf("[ ");
    for (k = 0; k < filled; k++)
        printf("%d ", frames[k]);
    printf("]\t\tFault\n");
}

time++;
}

printf("\nTotal Page Faults = %d\n", pageFaults);

return 0;
}
```

PROCEDURE

1. Open a text editor or IDE and create a new source file for the program (e.g., using C, C++, Java, or Python).
2. Declare an array to store the reference string, an array to represent the current contents of the page frames, and a parallel array to store the last-used time of each frame.
3. Implement a function to check whether a given page is currently present in the frames, and to update its last-used time on a hit.
4. Implement the LRU replacement logic: on a page fault with full frames, scan the last-used times to find the frame holding the least recently used page, and replace it.
5. Write the main function to accept the reference string, its length, and the number of page frames as input, and to call the LRU simulation function while counting page faults.
6. Compile the program and correct any syntax errors.
7. Execute the program using a standard test case (e.g., reference string = 7 0 1 2 0 3 0 4 2 3 0 3 with 3 page frames) and note the frame contents at each step and the total number of page faults.
8. Verify the result manually by tracing the frame contents and last-used times step by step for the same reference string and cross-checking with the program's output.
9. Re-run the program with a different number of page frames for the same reference string and observe how the total number of page faults changes.
10. Record and tabulate the results for different frame sizes for comparison with FIFO and other page replacement algorithms, if required.

OUTPUT

The program successfully simulates the LRU page replacement algorithm. Sample output is illustrated below:

```
Enter length of reference string: 12 Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 Enter number
of page frames: 3 Ref Frames      Status 7 [7]      Fault 0 [7,0]      Fault 1 [7,0,1]
Fault 2 [2,0,1]      Fault 0 [2,0,1]      Hit 3 [2,0,3]      Fault 0 [2,0,3]      Hit 4 [4,0,3]
Fault 2 [4,0,2]      Fault 3 [4,3,2]      Fault 0 [0,3,2]      Fault 3 [0,3,2]      Hit Total Page
Faults = 9
```

MODEL VIVA QUESTIONS

1. What is the principle of locality of reference, and how does the LRU page replacement algorithm make use of it?
2. How does the LRU algorithm decide which page to replace when a page fault occurs, and what bookkeeping does it require?
3. Why does the LRU algorithm not suffer from Belady's Anomaly, unlike FIFO?
4. What are the practical difficulties in implementing LRU exactly in hardware, and what approximation techniques (e.g., clock/second-chance algorithm) are used instead?
5. How does LRU compare with the Optimal (OPT) page replacement algorithm in terms of performance and practicality?

RESULT

The program to simulate the Least Recently Used (LRU) page replacement algorithm was successfully written, executed, and verified. The frame contents and last-used times were correctly updated on each reference, the least recently used page was replaced whenever a page fault occurred with full frames, and the total number of page faults was correctly computed and matched the manually verified calculation.

C) SIMULATE THE FOLLOWING PAGE REPLACEMENT ALGORITHMS:LFU**AIM**

To write a C program to simulate the Least Frequently Used (LFU) page replacement algorithm and to compute the number of page hits and page faults for a given reference string and number of frames.

LEARNING OUTCOME

After completing this experiment, the student will be able to:

- Understand the concept of virtual memory and paging in operating systems.
- Understand why page replacement algorithms are needed when a page fault occurs and no free frame is available.
- Understand the working principle of the LFU page replacement algorithm.
- Implement and simulate the LFU algorithm using a C program.
- Compare the efficiency of LFU with other page replacement algorithms (FIFO, LRU, Optimal) based on the number of page faults.

DESCRIPTION

In a paging system, when a page is referenced and it is not present in main memory, a page fault occurs. If all frames are already occupied, the operating system must select a page to remove (a victim) to make room for the new page. The strategy used to select the victim page is called a page replacement algorithm.

Least Frequently Used (LFU) is a page replacement algorithm based on the idea that a page which has been used least often in the past is least likely to be used in the near future. Each page in memory is associated with a counter/frequency count that records how many times it has been referenced. When a page fault occurs and memory is full, the page with the smallest frequency count is replaced.

Tie-breaking rule: If two or more pages have the same (lowest) frequency, the Least Recently Used among them (or the one that entered memory first, depending on convention) is chosen for replacement.

Drawback: A page that was heavily used in the initial phase of execution but is no longer needed may retain a high frequency count and stay in memory unnecessarily — this is a known limitation of pure LFU.

ALGORITHM

1. Start.
2. Read the number of page frames available in memory.
3. Read the reference string (sequence of page requests).
4. Initialize all frames as empty and set frequency count of all pages to 0.
5. For each page in the reference string:

- a. If the page is already present in a frame → it is a hit. Increment its frequency count.
- b. If the page is not present in a frame → it is a fault.
 - i. If there is a free frame available, load the page into it and set its frequency count to 1.
 - ii. If no free frame is available, find the page in memory with the minimum frequency count (break ties using least recently used), remove it, and load the new page in its place with frequency count 1.
6. Repeat step 5 until all pages in the reference string are processed.
7. Count and display the total number of page hits and page faults.
8. Stop.

PROGRAM

```
#include <stdio.h>
#define MAX 50
int main() {
    int ref[MAX], freq[MAX], frames_arr[MAX], loadTime[MAX];
    int n, frames, i, j, k, pos, minFreq, minIndex, found;
    int faults = 0, hits = 0, time = 0;

    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);

    printf("Enter the reference string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    for (i = 0; i < frames; i++) {
        frames_arr[i] = -1; // empty frame
        freq[i] = 0;
        loadTime[i] = 0;
    }

    printf("\n%-12s%-20s%-10s\n", "Page", "Frame Status", "Result");

    for (i = 0; i < n; i++) {
        found = 0;
        time++;

```

```
// Check if page already in memory
for (j = 0; j < frames; j++) {
    if (frames_arr[j] == ref[i]) {
        found = 1;
        pos = j;
        break;
    }
}

if (found) {
    freq[pos]++;
    loadTime[pos] = time;
    hits++;
    printf("%-12d", ref[i]);
    for (k = 0; k < frames; k++)
        printf("%3d", frames_arr[k]);
    printf("    HIT\n");
} else {
    faults++;
    // Look for a free frame
    int freeSlot = -1;
    for (j = 0; j < frames; j++) {
        if (frames_arr[j] == -1) {
            freeSlot = j;
            break;
        }
    }

    if (freeSlot != -1) {
        frames_arr[freeSlot] = ref[i];
        freq[freeSlot] = 1;
        loadTime[freeSlot] = time;
    } else {
        // Find least frequently used; tie -> least recently loaded
        minFreq = freq[0];
        minIndex = 0;
        for (j = 1; j < frames; j++) {
            if (freq[j] < minFreq ||
                (freq[j] == minFreq && loadTime[j] < loadTime[minIndex])) {
                minFreq = freq[j];
                minIndex = j;
            }
        }
        frames_arr[minIndex] = ref[i];
        freq[minIndex] = 1;
    }
}
```

```

        loadTime[minIndex] = time;
    }

    printf("%-12d", ref[i]);
    for (k = 0; k < frames; k++) {
        if (frames_arr[k] == -1)
            printf(" -");
        else
            printf("%3d", frames_arr[k]);
    }
    printf("    FAULT\n");
}
}

printf("\nTotal Page Hits  = %d", hits);
printf("\nTotal Page Faults = %d\n", faults);

return 0;
}

```

PROCEDURE

1. Open a text editor / IDE (Turbo C, Code::Blocks, gcc on Linux, etc.).
2. Type the above C program and save it with a .c extension (e.g., lfu.c).
3. Compile the program: gcc lfu.c -o lfu
4. Run the executable: ./lfu
5. Enter the number of pages in the reference string.
6. Enter the reference string values one by one.
7. Enter the number of available frames.
8. Observe the frame status displayed for each reference (HIT/FAULT).
9. Note down the total number of page hits and page faults displayed at the end.
10. Repeat with different reference strings/frame sizes for verification.

OUTPUT

Sample Input / Output:

Enter number of pages in reference string: 12
Enter the reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

Page	Frame Status	Result
7	7 - -	FAULT

0	7 0 -	FAULT
1	7 0 1	FAULT
2	2 0 1	FAULT
0	2 0 1	HIT
3	2 0 3	FAULT
0	2 0 3	HIT
4	4 0 3	FAULT
2	4 0 2	FAULT
3	4 3 2	FAULT
0	0 3 2	FAULT
3	0 3 2	HIT

Total Page Hits = 3

Total Page Faults = 9

(Actual frame contents/order may vary slightly depending on the tie-breaking convention used, but the total fault/hit count for this reference string with 3 frames remains consistent with LFU logic.)

MODEL VIVA QUESTIONS

1. What is a page fault?
2. What is the purpose of a page replacement algorithm?
3. Explain the working principle of the LFU algorithm.
4. What is the main drawback of the LFU algorithm?
5. How is tie-breaking handled in LFU when two pages have the same frequency?

RESULT

The C program to simulate the Least Frequently Used (LFU) page replacement algorithm was successfully written, executed, and verified. The number of page hits and page faults for the given reference string was computed correctly.

EXERCISES/EXPERIMENTS FROM ASSEMBLERS, LOADERS AND MACROPROCESSOR

EXP.NO.1**IMPLEMENT PASS ONE OF A TWO PASS ASSEMBLER.****AIM**

To design, develop and execute a C program to implement Pass I of a two pass assembler, which constructs the Symbol Table (SYMTAB), Literal Table (LITTAB) and an intermediate code file from a given assembly language source program.

LEARNING OUTCOME

Understand the internal working and design of a two pass assembler.

- Learn how the Location Counter (LOCCTR) is used to assign addresses to instructions and data.
- Understand the construction and role of the Symbol Table and Literal Table.
- Learn the purpose of intermediate code and how it supports Pass II processing.
- Gain practical experience in handling assembler directives such as START, END, LTORG, RESW and RESB.

DESCRIPTION

An assembler is a translator that converts an assembly language program into machine code. Since some information, such as the final address of a forward-referenced symbol or the address of a literal, may not be available while a statement is first scanned, the translation process is split into two passes.

Pass I of the assembler is mainly concerned with the definition of symbols and literals and the assignment of addresses, rather than with generating the actual machine instructions. Its major tasks are:

- Keep track of the Location Counter (LOCCTR) to assign an address to every instruction and data item in the program.
- Build the Symbol Table (SYMTAB), storing the name and address of every label defined in the program.
- Build the Literal Table (LITTAB), storing every literal used and, at a subsequent LTORG or END statement, assigning it an address.
- Process pseudo-ops (declaratives) such as START, END, RESW, RESB and LTORG, updating LOCCTR appropriately for each.
- Write an intermediate code file containing the LOCCTR value, label, opcode and operand for every source statement, for use as input to Pass II.

The output of Pass I - the Symbol Table, the Literal Table and the intermediate code - together provide Pass II with everything it needs to resolve addresses and generate the final machine code.

ALGORITHM

1. Start.
2. Initialize LOCCTR (Location Counter) to 0.
3. Read the first input line from the source program.
4. If OPCODE is START, save the operand (starting address) in LOCCTR, write the line to the intermediate file, and read the next line.
5. If OPCODE is not START, set LOCCTR to 0.
6. Repeat steps 6 to 12 until OPCODE is END.
7. If a symbol (label) is present in the label field, search SYMTAB for the symbol.
8. If the symbol is already present in SYMTAB, signal an error (duplicate symbol definition); otherwise enter the symbol, along with the current value of LOCCTR, into SYMTAB.
9. Search the operation code in the Machine Opcode Table (OPTAB).
10. If found, write the LOCCTR value, OPCODE and OPERAND to the intermediate file, and increment LOCCTR by the instruction length given in OPTAB.
11. If OPCODE is a pseudo instruction such as WORD, RESW, RESB or BYTE, process it separately: update LOCCTR based on the operand and data type, and enter/verify the symbol in SYMTAB.
12. If OPCODE is LTORG or END, dump the literal table entries that do not yet have addresses assigned, assign them the current LOCCTR value, and increment LOCCTR accordingly.
13. If OPCODE is not found in OPTAB and is not a valid pseudo-op, signal an error (invalid opcode).
14. When OPCODE is END, write the last intermediate file line, record the total program length (LOCCTR - starting address) and pass the SYMTAB, LITTAB and intermediate file to Pass II.
15. Stop.

PROGRAM

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
#define MAX 50
```

```
struct Symbol {
    char label[10];
    int address;
} symtab[MAX];
```

```
struct Literal {
    char lit[10];
    int address;
} littab[MAX];

struct Opcode {
    char mnemonic[10];
    int length;
} optab[] = {
    {"STOP", 1}, {"ADD", 1}, {"SUB", 1}, {"MULT", 1},
    {"MOVER", 1}, {"MOVEM", 1}, {"COMP", 1},
    {"BC", 1}, {"DIV", 1}, {"READ", 1}, {"PRINT", 1}
};

int nOpt = 11, symCount = 0, litCount = 0, locctr = 0;

int searchOptab(char *op) {
    int i;
    for (i = 0; i < nOpt; i++)
        if (strcmp(optab[i].mnemonic, op) == 0)
            return i;
    return -1;
}

int searchSymtab(char *sym) {
    int i;
    for (i = 0; i < symCount; i++)
        if (strcmp(symtab[i].label, sym) == 0)
            return i;
    return -1;
}

void insertSymtab(char *sym, int addr) {
    strcpy(symtab[symCount].label, sym);
    symtab[symCount].address = addr;
    symCount++;
}

void insertLittab(char *lit) {
    strcpy(littab[litCount].lit, lit);
    littab[litCount].address = -1;
    litCount++;
}

void dumpLiterals() {
```

```
int i;
for (i = 0; i < litCount; i++) {
    if (littab[i].address == -1) {
        littab[i].address = locctr;
        locctr++;
    }
}
}

int main() {
    FILE *fp, *ic, *st, *lt;
    char label[10], opcode[10], operand[15];
    int opIndex, i;

    fp = fopen("input.txt", "r");
    ic = fopen("intermediate.txt", "w");

    if (fp == NULL) {
        printf("Cannot open input.txt\n");
        return 1;
    }

    while (fscanf(fp, "%s %s %s", label, opcode, operand) != EOF) {

        if (strcmp(opcode, "START") == 0) {
            locctr = atoi(operand);
            fprintf(ic, "\t\t%s\t%s\t%s\n", label, opcode, operand);
            continue;
        }

        if (strcmp(label, "-") != 0) {
            if (searchSymtab(label) == -1)
                insertSymtab(label, locctr);
            else
                printf("Error: Duplicate symbol %s\n", label);
        }

        if (operand[0] == '=') {
            if (searchSymtab(operand) == -1)
                insertLittab(operand);
        }

        opIndex = searchOptab(opcode);

        if (opIndex != -1) {
```

```

        fprintf(ic, "%d\t%s\t%s\t%s\n", locctr, label, opcode, operand);
        locctr += optab[opIndex].length;
    }
    else if (strcmp(opcode, "LTORG") == 0) {
        fprintf(ic, "%d\t%s\t%s\t%s\n", locctr, label, opcode, operand);
        dumpLiterals();
    }
    else if (strcmp(opcode, "RESW") == 0) {
        fprintf(ic, "%d\t%s\t%s\t%s\n", locctr, label, opcode, operand);
        locctr += atoi(operand);
    }
    else if (strcmp(opcode, "RESB") == 0) {
        fprintf(ic, "%d\t%s\t%s\t%s\n", locctr, label, opcode, operand);
        locctr += atoi(operand);
    }
    else if (strcmp(opcode, "END") == 0) {
        dumpLiterals();
        fprintf(ic, "%d\t%s\t%s\t%s\n", locctr, label, opcode, operand);
        break;
    }
    else {
        printf("Error: Invalid opcode %s\n", opcode);
    }
}

fclose(fp);
fclose(ic);

st = fopen("symtab.txt", "w");
fprintf(st, "SYMBOL\tADDRESS\n");
for (i = 0; i < symCount; i++)
    fprintf(st, "%s\t%d\n", symtab[i].label, symtab[i].address);
fclose(st);

lt = fopen("littab.txt", "w");
fprintf(lt, "LITERAL\tADDRESS\n");
for (i = 0; i < litCount; i++)
    fprintf(lt, "%s\t%d\n", littab[i].lit, littab[i].address);
fclose(lt);

printf("Pass I completed. Intermediate code, SYMTAB and LITTAB generated.\n");
return 0;
}

```

PROCEDURE

1. Open a text editor / IDE (e.g. gcc, Turbo C, or an online C compiler).
2. Type the Pass I source program shown above and save it as pass1.c.
3. Prepare an assembly source program (e.g. input.txt) containing START, machine instruction, declarative and END statements, and place it in the same directory.
4. Compile the program using the command: gcc pass1.c -o pass1
5. Execute the program using the command: ./pass1
6. Verify the generated files - intermediate.txt (intermediate code), symtab.txt (symbol table) and littab.txt (literal table).
7. Compare the generated Symbol Table and Literal Table with values traced manually on paper to verify correctness.

OUTPUT

Sample input program (input.txt):

```
-   START 101
L1  MOVER AREG  A
-   ADD  AREG  B
-   MOVEM AREG  C
-   PRINT C    -
A   RESW  1
B   RESW  1
C   RESW  1
-   LTORG -    -
-   END  -    -
```

Generated Symbol Table (SYMTAB):

SYMBOL	ADDRESS
L1	101
A	102
B	103
C	104

Generated Literal Table (LITTAB):

LITERAL	ADDRESS
=	105

(Exact values will vary depending on the mnemonics, operands and literals used in the actual input program supplied.)

MODEL VIVA QUESTIONS

1. What is the main function of Pass I of a two-pass assembler?
2. What is the Location Counter (LOCCTR) and why is it needed?
3. What is the difference between Pass I and Pass II of an assembler?
4. What is LORG used for in Pass I processing?
5. Why can't literals be assigned addresses immediately when they are encountered?

RESULT

Thus, a C program was written and executed successfully to implement Pass I of a two pass assembler, which reads the assembly source program, constructs the Symbol Table and Literal Table, assigns addresses using the Location Counter, and generates the intermediate code required for Pass II.

EXP.NO.2**IMPLEMENT PASS TWO OF A TWO PASS ASSEMBLER****AIM**

To design, develop and execute a C program to implement Pass II of a two pass assembler, which uses the intermediate code, Symbol Table (SYMTAB) and Literal Table (LITTAB) generated by Pass I to produce the final machine code / object program.

LEARNING OUTCOME

- Understand how Pass II completes the translation process started by Pass I.
- Learn how forward references to symbols are resolved using the Symbol Table.
- Understand how literals are converted into actual machine code using the Literal Table.
- Learn the structure of an object program - Header, Text and End records.
- Gain practical experience in generating machine code from intermediate code using OPTAB.

DESCRIPTION

Pass II of the assembler is responsible for generating the actual machine code for every instruction in the source program. It takes as input the intermediate code, Symbol Table and Literal Table that were produced by Pass I, and does not need to re-scan the original source program.

Since Pass I has already resolved every label's address (including forward references) and assigned addresses to all literals, Pass II can process the program in a single, straightforward left-to-right scan. Its major tasks are:

- Read SYMTAB and LITTAB into memory so that symbol and literal addresses can be looked up quickly.
- Read each record of the intermediate code, one instruction at a time.
- For each machine instruction, search OPTAB to obtain the machine opcode corresponding to the mnemonic.
- Resolve the operand: if it is a symbol, fetch its address from SYMTAB; if it is a literal, fetch its address from LITTAB; if it is a register, fetch its register code.
- Combine the opcode, register code and resolved address to form the complete machine instruction, and write it to the output object file.
- At LORG or END records, generate machine code for the actual values of literals and place them at their assigned addresses.
- Assemble the Header, Text and End records to form the final object program.

The output of Pass II is the object program (or a simple machine code listing), which is ready to be loaded and executed, along with an error listing for any undefined symbols or invalid opcodes encountered.

ALGORITHM

1. Start.
2. Read the Symbol Table (SYMTAB) and Literal Table (LITTAB) produced by Pass I into memory.
3. Read the Machine Opcode Table (OPTAB), which stores the mnemonic, machine opcode and instruction length/format for each instruction.
4. Read the first record of the intermediate code file generated by Pass I.
5. Repeat steps 6 to 12 until the END record is encountered.
6. If the opcode field indicates a declarative such as RESW, RESB or START, no machine code is generated; simply move to the next record.
7. If the opcode field indicates LTORG or END, generate machine code for every literal in LITTAB that has not yet been assigned code, by fetching its value and placing it at the address stored in LITTAB.
8. Otherwise, search OPTAB for the mnemonic to obtain the corresponding machine opcode.
9. If the operand is a symbol, search SYMTAB to obtain its address; if the symbol is undefined, report an error (undefined symbol).
10. If the operand is a literal, search LITTAB to obtain its assigned address.
11. Combine the machine opcode, register code (if any) and the resolved address/operand to form the complete machine instruction.
12. Write the generated machine code, along with the LOCCTR value, to the output/object file (or Text record of the object program).
13. Read the next intermediate code record.
14. When the END record is reached, write the Header record (program name, starting address, program length), the accumulated Text records, and the End record (starting execution address) to complete the object program.
15. Stop.

PROGRAM

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX 50
```

```

struct Symbol {
    char label[10];
    int address;
} symtab[MAX];
int symCount = 0;

struct Literal {
    char lit[10];
    int address;
} littab[MAX];
int litCount = 0;

struct Opcode {
    char mnemonic[10];
    char machineCode[5];
} optab[] = {
    {"MOVER", "04"}, {"MOVEM", "05"}, {"ADD", "01"}, {"SUB", "02"},
    {"MULT", "03"}, {"COMP", "06"}, {"BC", "07"},
    {"DIV", "08"}, {"READ", "09"}, {"PRINT", "10"}, {"STOP", "00"}
};
int nOpt = 11;

/* register codes */
struct Reg {
    char name[5];
    char code[3];
} regtab[] = { {"AREG", "1"}, {"BREG", "2"}, {"CREG", "3"}, {"DREG", "4"} };
int nReg = 4;

void loadSymtab() {
    FILE *fp = fopen("symtab.txt", "r");
    char line[50], label[10];
    int addr;
    fgets(line, sizeof(line), fp);    /* skip header */
    while (fscanf(fp, "%s %d", label, &addr) != EOF) {
        strcpy(symtab[symCount].label, label);
        symtab[symCount].address = addr;
        symCount++;
    }
    fclose(fp);
}

void loadLittab() {
    FILE *fp = fopen("littab.txt", "r");
    char line[50], lit[10];

```

```
int addr;
fgets(line, sizeof(line), fp);
while (fscanf(fp, "%s %d", lit, &addr) != EOF) {
    strcpy(littab[litCount].lit, lit);
    littab[litCount].address = addr;
    litCount++;
}
fclose(fp);
}

int searchSymtab(char *sym) {
    int i;
    for (i = 0; i < symCount; i++)
        if (strcmp(symtab[i].label, sym) == 0) return i;
    return -1;
}

int searchLittab(char *lit) {
    int i;
    for (i = 0; i < litCount; i++)
        if (strcmp(littab[i].lit, lit) == 0) return i;
    return -1;
}

int searchOptab(char *op) {
    int i;
    for (i = 0; i < nOpt; i++)
        if (strcmp(optab[i].mnemonic, op) == 0) return i;
    return -1;
}

int searchRegtab(char *reg) {
    int i;
    for (i = 0; i < nReg; i++)
        if (strcmp(regtab[i].name, reg) == 0) return i;
    return -1;
}

int main() {
    FILE *ic, *out;
    char loc[10], label[10], opcode[10], operand[15];
    int opIndex, regIndex, symIndex, litIndex;

    loadSymtab();
    loadLittab();
}
```

```
ic = fopen("intermediate.txt", "r");
out = fopen("machinecode.txt", "w");

if (ic == NULL) {
    printf("Cannot open intermediate.txt\n");
    return 1;
}

while (fscanf(ic, "%s %s %s %s", loc, label, opcode, operand) != EOF) {

    if (strcmp(opcode, "START") == 0 ||
        strcmp(opcode, "RESW") == 0 ||
        strcmp(opcode, "RESB") == 0) {
        continue;          /* no machine code generated */
    }

    if (strcmp(opcode, "LTORG") == 0) {
        int i;
        for (i = 0; i < litCount; i++) {
            fprintf(out, "%d\t(%s)\t%02d\n",
                    littab[i].address, littab[i].lit, 0);
        }
        continue;
    }

    if (strcmp(opcode, "END") == 0) {
        int i;
        for (i = 0; i < litCount; i++) {
            fprintf(out, "%d\t(%s)\t%02d\n",
                    littab[i].address, littab[i].lit, 0);
        }
        break;
    }

    opIndex = searchOptab(opcode);
    if (opIndex == -1) {
        printf("Error: Invalid opcode %s at location %s\n", opcode, loc);
        continue;
    }

    regIndex = searchRegtab(operand);
    if (regIndex != -1) {
        fprintf(out, "%s\t%s %s\n", loc, optab[opIndex].machineCode,
                regtab[regIndex].code);
    }
}
```

```

        continue;
    }

    if (operand[0] == '=') {
        litIndex = searchLittab(operand);
        if (litIndex != -1)
            fprintf(out, "%s\t%s %d\n", loc, optab[opIndex].machineCode,
                    littab[litIndex].address);
        else
            printf("Error: Undefined literal %s\n", operand);
        continue;
    }

    symIndex = searchSymtab(operand);
    if (symIndex != -1) {
        fprintf(out, "%s\t%s %d\n", loc, optab[opIndex].machineCode,
                symtab[symIndex].address);
    } else if (strcmp(operand, "-") == 0) {
        fprintf(out, "%s\t%s\n", loc, optab[opIndex].machineCode);
    } else {
        printf("Error: Undefined symbol %s at location %s\n", operand, loc);
    }
}

fclose(ic);
fclose(out);

printf("Pass II completed. Machine code generated in machinecode.txt\n");
return 0;
}

```

PROCEDURE

1. Ensure that Pass I has already been executed and that intermediate.txt, symtab.txt and littab.txt are present in the working directory.
2. Open a text editor / IDE and type the Pass II source program shown above; save it as pass2.c.
3. Compile the program using the command: gcc pass2.c -o pass2
4. Execute the program using the command: ./pass2
5. Verify the generated file machinecode.txt, which lists the address and machine code for every instruction and literal.
6. Cross-check the machine code manually using OPTAB, SYMTAB and LITTAB to confirm each instruction has been translated correctly.

OUTPUT

Using the intermediate code, SYMTAB and LITTAB generated by Pass I for the sample program:

```

101 L1 MOVER AREG A
102 - ADD AREG B
103 - MOVEM AREG C
104 - PRINT C
105 A RESW 1
106 B RESW 1
107 C RESW 1
108 - LTORG
108 - END

```

Generated Machine Code (machinecode.txt):

ADDRESS	MNEMONIC OPCODE	MACHINE CODE
101	MOVER AREG, A	04 1 102
102	ADD AREG, B	01 1 103
103	MOVEM AREG, C	05 1 104
104	PRINT C	10 104
108	(=1)	00 01

(Exact opcode and register codes depend on the OPTAB and register codes defined in the program; RESW locations do not generate machine code.)

MODEL VIVA QUESTIONS

1. What is the primary purpose of Pass II of a two-pass assembler?
2. Why does Pass II need the Symbol Table built by Pass I?
3. How are literals converted to machine code in Pass II?
4. What are the three main sections of the object program produced at the end of Pass II?
5. What error can occur in Pass II if a symbol is used but never defined?

RESULT

Thus, a C program was written and executed successfully to implement Pass II of a two pass assembler, which reads the intermediate code, Symbol Table and Literal Table generated by Pass I, resolves all symbol and literal references, and generates the final machine code / object program.

EXP.NO.3**IMPLEMENT A SINGLE PASS ASSEMBLER.****AIM**

To write, execute, and study a C program that implements a Single Pass Assembler which converts a given assembly language program into its equivalent machine/object code (intermediate code) in a single scan of the source program, while simultaneously constructing the Symbol Table (SYMTAB).

LEARNING OUTCOME

- Understand the design and working of language processors, specifically assemblers, and the difference between single pass and two pass assembly.
- Learn to construct and use data structures such as the Symbol Table (SYMTAB), Mnemonics/Opcode Table (OPTAB), and Pseudo-Opcode Table (POTAB).
- Understand the concept of forward references and how a single pass assembler resolves them using a Table of Incomplete Instructions (TIP) / back-patching technique.
- Gain the ability to translate symbolic instructions into intermediate code and generate the corresponding machine code.
- Develop skills in implementing systems programming concepts using the C programming language.

DESCRIPTION

An assembler is a language translator that converts an assembly language program into its equivalent machine language (object) program. A Two Pass Assembler scans the source program twice: the first pass builds the symbol table, and the second pass generates the target/object code using the completed symbol table. A Single Pass Assembler, on the other hand, performs both symbol table construction and code generation in a single scan of the source program. The main challenge in a single pass assembler is handling forward references — that is, references to symbols that are used before they are defined (for example, a forward jump to a label that appears later in the program). Since the symbol's address is not yet known when it is first encountered, the assembler cannot immediately generate the correct operand address. This problem is solved by maintaining a Table of Incomplete Instructions (TIP). Whenever a forward reference is detected, an entry is made in the TIP along with the address of the instruction that needs to be fixed. When the symbol is finally defined (i.e., it appears as a label), the assembler searches the TIP for all pending references to that symbol and back-patches (fixes up) those instructions with the correct address. This is why the technique is also called the "back-patching" method. The assembler uses three key tables: SYMTAB (Symbol Table): stores symbol name, its address, and length. OPTAB (Mnemonic/Opcode Table): stores the mnemonic opcode and its corresponding machine opcode along with instruction format/length. POTAB (Pseudo-Opcode

Table): stores assembler directives such as START, END, DS, DC, etc., which do not generate machine code directly but guide the assembly process.

ALGORITHM

1. Start.

2. Initialize the location counter (LC) to 0, and initialize SYMTAB, OPTAB, and the Table of Incomplete Instructions (TIP) as empty.

3. Read the first line of the source program.

4. If the opcode is START, read the operand as the starting address, set LC to that value, and read the next line.

5. Repeat steps 6 to 11 until the opcode read is END.

6. If a label is present in the current instruction:

If the label is not already present in SYMTAB, insert it along with the current value of LC.

If the label is already present in SYMTAB (with a null/undefined address), update its address to the current LC and search the TIP for pending references to this symbol; back-patch each such instruction with the resolved address, then delete those entries from the TIP.

7. Search the mnemonic opcode in OPTAB.

8. If the opcode is a machine instruction (found in OPTAB):

If the operand symbol is already defined in SYMTAB, generate the machine code using its address.

If the operand symbol is not yet defined in SYMTAB, insert the symbol with an undefined (null) address, generate the machine code with a blank/placeholder operand address, and make an entry in the TIP recording this instruction's address for later back-patching.

9. If the opcode is a pseudo-opcode (found in POTAB) such as DS or DC, process it accordingly (reserve/define storage) and update LC by the appropriate length.

10. Increment LC by the length of the current instruction/data.

11. Write the generated intermediate/machine code line to the output (object code) file, and read the next input line.

12. On reaching END, ensure the TIP is empty (i.e., no unresolved forward references remain); if entries remain, report an undefined symbol error.

13. Print the final SYMTAB and the generated object/machine code.

14. Stop.

PROGRAM

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#define MAX 50

struct symtab {
    char name[10];
    int addr;
    int defined; /* 1 = defined, 0 = forward-referenced */
} symtab[MAX];
int symcount = 0;

struct optab {
    char mnemonic[10];
    char mcode[5];
    int length;
} optab[] = {
    {"MOVER", "04", 3}, {"MOVEM", "05", 3}, {"ADD", "01", 3},
    {"SUB", "02", 3}, {"COMP", "06", 3}, {"BC", "07", 3},
    {"DIV", "03", 3}, {"MULT", "08", 3}, {"STOP", "00", 1}
};
int optab_size = 9;

struct tip {
    char symbol[10];
    int instr_addr; /* address of instruction needing back-patch */
} tip[MAX];
int tipcount = 0;

/* returns index in symtab, or -1 if not found */
int findSymbol(char *sym) {
    int i;
    for (i = 0; i < symcount; i++)
        if (strcmp(symtab[i].name, sym) == 0)
            return i;
    return -1;
}

int findOpcode(char *mne, int *len) {
```

```

int i;
for (i = 0; i < optab_size; i++)
    if (strcmp(optab[i].mnemonic, mne) == 0) {
        *len = optab[i].length;
        return i;
    }
return -1;
}

void backpatch(char *sym, int addr) {
    int i, j;
    for (i = 0; i < tipcount; i++) {
        if (strcmp(tip[i].symbol, sym) == 0) {
            printf("    [Back-patch] Instruction at %d resolved -> operand %s = %d\n",
                tip[i].instr_addr, sym, addr);
            /* remove entry from TIP by shifting */
            for (j = i; j < tipcount - 1; j++)
                tip[j] = tip[j + 1];
            tipcount--; i--;
        }
    }
}

int main() {
    char label[10], mnemonic[10], operand[10];
    int LC = 0, len, opidx, sidx;
    FILE *fp;
    char line[100];

    fp = fopen("input.asm", "r");
    if (fp == NULL) {
        printf("Cannot open input.asm\n");
        return 1;
    }

    printf("LC\tLABEL\tOPCODE\tOPERAND\tMACHINE CODE / ACTION\n");
    printf("-----\n");

    while (fgets(line, sizeof(line), fp)) {
        label[0] = mnemonic[0] = operand[0] = '\0';
        sscanf(line, "%s %s %s", label, mnemonic, operand);

        if (strcmp(mnemonic, "START") == 0) {
            LC = atoi(operand);
            printf("\t\tSTART\t%s\t\tLC initialised to %d\n", operand, LC);

```

```

        continue;
    }
    if (strcmp(mnemonic, "END") == 0) {
        printf("%d\t\tEND\t\t\tAssembly complete\n", LC);
        break;
    }

    /* Step: handle label definition */
    if (strcmp(label, "-") != 0 && strlen(label) != 0 && strcmp(mnemonic, "") != 0
        && strcmp(label, mnemonic) != 0 && operand[0] != '\0'
        && (sscanf(line, "%s %s %s", label, mnemonic, operand) == 3)) {
        /* label field really present only if 3 tokens read and first isn't opcode */
    }

    /* Re-parse properly: assume format LABEL OPCODE OPERAND (LABEL may be "-")
*/
    {
        char l2[10], m2[10], o2[10];
        int n = sscanf(line, "%s %s %s", l2, m2, o2);
        if (n == 3 && strcmp(l2, "-") != 0) {
            strcpy(label, l2); strcpy(mnemonic, m2); strcpy(operand, o2);
        } else if (n >= 2) {
            strcpy(label, "-");
            strcpy(mnemonic, l2);
            strcpy(operand, (n == 3) ? m2 : "");
        }
    }

    if (strcmp(label, "-") != 0) {
        idx = findSymbol(label);
        if (idx == -1) {
            strcpy(symtab[symcount].name, label);
            symtab[symcount].addr = LC;
            symtab[symcount].defined = 1;
            symcount++;
        } else {
            symtab[idx].addr = LC;
            symtab[idx].defined = 1;
            backpatch(label, LC);
        }
    }

    opidx = findOpcode(mnemonic, &len);
    if (opidx != -1) {
        if (strcmp(mnemonic, "STOP") == 0) {

```

```

        printf("%d\t%s\t%s\t\tMachine code: %s\n", LC, label, mnemonic,
optab[opidx].mcode);
    } else {
        sidx = findSymbol(operand);
        if (sidx == -1) {
            strcpy(symtab[symcount].name, operand);
            symtab[symcount].addr = -1;
            symtab[symcount].defined = 0;
            symcount++;
            tip[tipcount].instr_addr = LC;
            strcpy(tip[tipcount].symbol, operand);
            tipcount++;
            printf("%d\t%s\t%s\t\tMachine code: %s 01 (?) [forward ref -> TIP]\n",
                LC, label, mnemonic, operand, optab[opidx].mcode);
        } else if (symtab[sidx].defined) {
            printf("%d\t%s\t%s\t\tMachine code: %s 01 %d\n",
                LC, label, mnemonic, operand, optab[opidx].mcode, symtab[sidx].addr);
        } else {
            tip[tipcount].instr_addr = LC;
            strcpy(tip[tipcount].symbol, operand);
            tipcount++;
            printf("%d\t%s\t%s\t\tMachine code: %s 01 (?) [forward ref -> TIP]\n",
                LC, label, mnemonic, operand, optab[opidx].mcode);
        }
    }
    LC += len;
} else if (strcmp(mnemonic, "DS") == 0) {
    printf("%d\t%s\tDS\t\tReserved %s word(s)\n", LC, label, operand, operand);
    LC += atoi(operand);
} else if (strcmp(mnemonic, "DC") == 0) {
    printf("%d\t%s\tDC\t\tConstant stored, LC+1\n", LC, label, operand);
    LC += 1;
}
}

fclose(fp);

printf("\n----- SYMBOL TABLE (SYMTAB) ----- \n");
printf("SYMBOL\tADDRESS\n");
{ int i;
    for (i = 0; i < symcount; i++)
        printf("%-10s\t%d\n", symtab[i].name, symtab[i].addr);
}

if (tipcount != 0)

```

```

    printf("\nWarning: %d symbol(s) remain undefined at END.\n", tipcount);
else
    printf("\nAll forward references successfully resolved.\n");

return 0;
}

```

PROCEDURE

1. Open a text editor / IDE (e.g., Turbo C++, GCC, Code::Blocks) and create a new C source file, for example `single_pass_assembler.c`.
2. Type the program given above and save the file.
3. In the same directory, create an input file named `input.asm` containing a sample assembly program, for example:


```

START 100      MOVER AREG, A      MOVEM AREG, B
BC ANY, L1     ...      L1 ADD AREG, ONE      STOP  A DS 1  B DS 1  ONE
DC 1          END

```
4. Compile the program using the command: `gcc single_pass_assembler.c -o assembler`
5. Run the compiled executable: `./assembler` (or `assembler.exe` on Windows)
6. Observe the location-counter-wise trace of the assembly process printed on the screen, including forward references marked for the Table of Incomplete Instructions (TIP).
7. Verify the final Symbol Table (SYMTAB) printed at the end and confirm that all forward references have been resolved through back-patching.
8. Note down the output and compare it with the expected object code for verification.

OUTPUT

Sample input.asm:

```

START 100
MOVER AREG A
MOVEM AREG B
BC ANY L1
MOVER BREG A
L1  ADD AREG ONE
STOP
A   DS 1
B   DS 1
ONE DC 1
END

```

Sample (representative) console output:

LC	LABEL	OPCODE	OPERAND	MACHINE CODE / ACTION

	START	100		LC initialised to 100
100	-	MOVER	A	Machine code: 04 01 (?) [forward ref -> TIP]

```

103 -   MOVEM B           Machine code: 05 01 (?) [forward ref -> TIP]
106 -   BC   L1          Machine code: 07 01 (?) [forward ref -> TIP]
109 -   MOVER A          Machine code: 04 01 108
      [Back-patch] Instruction at 106 resolved -> operand L1 = 112
112   L1   ADD   ONE      Machine code: 01 01 (?) [forward ref -> TIP]
115 -   STOP              Machine code: 00
116   A    DS    1        Reserved 1 word(s)
117   B    DS    1        Reserved 1 word(s)
118   ONE  DC    1        Constant stored, LC+1
119   END                  Assembly complete

```

----- SYMBOL TABLE (SYMTAB) -----

SYMBOL	ADDRESS
A	116
B	117
L1	112
ONE	118

All forward references successfully resolved.

Note: The exact addresses/values depend on the sample program used; the trace above illustrates how forward references (e.g., B, L1, ONE) are entered into the Table of Incomplete Instructions and later back-patched once the corresponding label is defined.

MODEL VIVA QUESTIONS

- Q1. What is an assembler?
- Q2. What is the basic difference between a single pass and a two pass assembler?
- Q3. What is a forward reference?
- Q4. How does a single pass assembler handle forward references?
- Q5. What is back-patching?

RESULT

Thus, a C program to implement a Single Pass Assembler — which constructs the Symbol Table (SYMTAB) and generates the intermediate/machine code in a single scan of the source program, resolving forward references using the Table of Incomplete Instructions (TIP) and the back-patching technique — was successfully written, executed, and verified, and the output was found to be correct.

EXP.NO.4**IMPLEMENT A TWO PASS MACRO PROCESSOR****AIM**

To write, execute, and study a C program that implements a Two Pass Macro Processor which expands macro calls present in a given assembly language program into their equivalent macro definitions, using a two pass strategy: Pass I processes macro definitions, and Pass II expands macro calls.

LEARNING OUTCOME

- Understand the concept of macros, macro definition, macro call, and macro expansion in assembly language programming.
- Learn to construct and use the data structures required by a macro processor: Macro Name Table (MNT), Macro Definition Table (MDT), and Argument List Array (ALA).
- Understand the difference between a single pass and a two pass macro processor, and why two passes are needed when a program contains nested macro calls or macro calls that appear before the corresponding definition in processing order.
- Learn how positional/keyword parameters in a macro body are replaced by index markers (e.g., #1, #2) during definition processing, and substituted back with actual arguments during expansion.
- Gain the ability to translate a source program containing macro calls into an expanded, macro-free program suitable for further assembly.
- Develop skills in implementing systems programming concepts using the C programming language.

DESCRIPTION

A macro is a unit of a program that is defined once, given a name, and can be repeatedly invoked (called) by that name at different points in a program. Each time the macro is called, it is replaced by the sequence of statements that form its body — this process is called macro expansion. Macros allow programmers to avoid repeating the same block of code and to parameterize that code using formal parameters that are replaced by actual arguments at each call.

A macro processor is the system software responsible for detecting macro definitions and macro calls in a source program and performing macro expansion. A Two Pass Macro Processor performs this task in two scans of the source program:

- Pass I (Macro Definition Processing): Scans the source program, identifies each macro definition (from MACRO to MEND), records the macro name and its formal parameters, and stores the body of the macro (with formal parameters replaced by positional notation such as #1, #2, ...) in the Macro Definition Table (MDT). An entry is also made in the Macro Name Table (MNT) giving the macro's name, the index of its first statement in the MDT, and the number of parameters.

- **Pass II (Macro Expansion):** Scans the source program again. Every statement that is not a macro call is copied to the expanded output as it is. Whenever a macro call is encountered, the Macro Name Table is searched for that macro name; if found, the corresponding body is fetched from the MDT, and every positional parameter marker (#1, #2, ...) in the body is substituted with the actual argument supplied at the call, using the Argument List Array (ALA). The expanded statements are written to the output in place of the macro call.

The processor uses three key data structures:

- **MNT (Macro Name Table):** stores the macro name, the starting index (MDT pointer) of its definition, and the number of formal parameters.
- **MDT (Macro Definition Table):** stores the body of every macro definition, line by line, with formal parameters replaced by positional index markers (#1, #2, ...), terminated by a MEND entry.
- **ALA (Argument List Array):** temporarily holds the actual arguments supplied in a macro call, indexed by parameter position, used to substitute the positional markers during expansion.

ALGORITHM

Pass I — Processing Macro Definitions

1. Start. Initialize MNT, MDT, and their index counters (MNT_index = 0, MDT_index = 0) to empty/zero.
2. Read the next line of the source program.
3. If the line is not "MACRO", copy it as is to an intermediate file and repeat step 2 (this handles non-macro lines outside definitions, until a macro definition begins).
4. If the line is "MACRO", read the next line — this is the macro prototype (macro name and formal parameter list).
5. Enter the macro name into MNT along with the current MDT_index and the number of formal parameters; store the formal parameters (e.g., &A, &B) in a temporary parameter list for substitution.
6. Read the next line of the macro body. While the line is not "MEND":
 - Replace every occurrence of a formal parameter in the line with its positional index marker (e.g., &A -> #1, &B -> #2).
 - Store the modified line in the MDT at MDT_index, and increment MDT_index.
 - Read the next line and repeat.
7. When "MEND" is encountered, store a MEND entry in the MDT, increment MDT_index, and go back to step 2 to continue scanning for further macro definitions or normal code.
8. Repeat until the end of the source program is reached.
9. Stop Pass I. The MNT and MDT are now completely built.

Pass II — Expanding Macro Calls

1. Read the next line of the intermediate file (output of Pass I).

2. If the line is not a macro call (its opcode/name is not present in MNT), copy the line as is to the final expanded output.
3. If the line is a macro call, search MNT for the macro name to obtain its MDT starting index and number of parameters.
4. Extract the actual arguments from the macro call and store them in the Argument List Array (ALA), positionally matched to the formal parameters.
5. Starting from the MDT index found in MNT, read successive MDT entries. For each entry, replace every positional marker (#1, #2, ...) with the corresponding actual argument from the ALA, and write the resulting statement to the expanded output.
6. Continue copying and substituting MDT entries until a "MEND" entry is encountered in the MDT; then stop expanding this call and resume reading the intermediate file from step 1.
7. Repeat until the end of the intermediate file is reached.
8. Stop. The final expanded (macro-free) program is now available as output.

PROGRAM

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX 50

/* ----- Macro Name Table ----- */
struct mnt {
    char name[15];
    int mdt_index;
    int nparams;
} MNT[MAX];
int mnt_count = 0;

/* ----- Macro Definition Table ----- */
char MDT[MAX][100];
int mdt_count = 0;

/* formal parameter list of the macro currently being defined */
char formal[10][10];
int nformal = 0;

int findMNT(char *name) {
    int i;
    for (i = 0; i < mnt_count; i++)
        if (strcmp(MNT[i].name, name) == 0)
            return i;
    return -1;
}
```

```

}

/* replace every occurrence of formal parameters in 'line' with #1,#2,... */
void substituteFormals(char *line, char *out) {
    char word[30];
    int i = 0, j = 0, k;
    out[0] = '\0';
    while (line[i] != '\0') {
        if (line[i] == ' ' || line[i] == ',') {
            char sep = line[i];
            strcat(out, "");
            /* copy separator */
            char tmp[2] = { sep, '\0' };
            strcat(out, tmp);
            i++;
            continue;
        }
        j = 0;
        while (line[i] != '\0' && line[i] != ' ' && line[i] != ',') {
            word[j++] = line[i++];
        }
        word[j] = '\0';
        int found = -1;
        for (k = 0; k < nformal; k++)
            if (strcmp(word, formal[k]) == 0) { found = k; break; }
        if (found != -1) {
            char pos[6];
            sprintf(pos, "#%d", found + 1);
            strcat(out, pos);
        } else {
            strcat(out, word);
        }
    }
}

/* ----- PASS I : build MNT and MDT, write intermediate file ----- */
void passOne(FILE *src, FILE *inter) {
    char line[100], token1[30];

    while (fgets(line, sizeof(line), src)) {
        line[strcspn(line, "\n")] = 0;
        sscanf(line, "%s", token1);

        if (strcmp(token1, "MACRO") == 0) {
            /* next line is the macro prototype: NAME &P1,&P2 */

```

```

char proto[100], name[20], plist[80];
fgets(proto, sizeof(proto), src);
proto[strcspn(proto, "\n")] = 0;
sscanf(proto, "%s %[^\\n]", name, plist);

nformal = 0;
char *tok = strtok(plist, ",");
while (tok != NULL) {
    strcpy(formal[nformal++], tok);
    tok = strtok(NULL, ",");
}

strcpy(MNT[mnt_count].name, name);
MNT[mnt_count].mdt_index = mdt_count;
MNT[mnt_count].nparams = nformal;
mnt_count++;

char body[100];
while (fgets(body, sizeof(body), src)) {
    body[strcspn(body, "\n")] = 0;
    char btok[20];
    sscanf(body, "%s", btok);
    if (strcmp(btok, "MEND") == 0) {
        strcpy(MDT[mdt_count++], "MEND");
        break;
    }
    char subst[100];
    substituteFormals(body, subst);
    strcpy(MDT[mdt_count++], subst);
}
} else {
    /* ordinary line: copy to intermediate file for Pass II */
    fprintf(inter, "%s\\n", line);
}
}
}

/* ----- PASS II : expand macro calls ----- */
void passTwo(FILE *inter, FILE *out) {
    char line[100], token1[30];

    while (fgets(line, sizeof(line), inter)) {
        line[strcspn(line, "\n")] = 0;
        sscanf(line, "%s", token1);

```

```

int midx = findMNT(token1);
if (midx == -1) {
    fprintf(out, "%s\n", line);
} else {
    /* extract actual arguments */
    char rest[80], actual[10][20];
    int nactual = 0;
    if (sscanf(line, "%*s %[^\n]", rest) == 1) {
        char *tok = strtok(rest, ",");
        while (tok != NULL) {
            strcpy(actual[nactual++], tok);
            tok = strtok(NULL, ",");
        }
    }

    int p = MNT[midx].mdt_index;
    while (strcmp(MDT[p], "MEND") != 0) {
        char expanded[120] = "";
        char *src = MDT[p];
        int i = 0, j;
        while (*src) {
            if (*src == '#') {
                int num = atoi(src + 1);
                strcat(expanded, actual[num - 1]);
                src++;
                while (*src >= '0' && *src <= '9') src++;
            } else {
                char tmp[2] = { *src, '\0' };
                strcat(expanded, tmp);
                src++;
            }
        }
        fprintf(out, "%s\n", expanded);
        p++;
    }
}

int main() {
    FILE *src = fopen("input.asm", "r");
    FILE *inter = fopen("intermediate.txt", "w+");
    FILE *out = fopen("expanded.asm", "w");

    if (!src || !inter || !out) {

```

```

    printf("Error opening files.\n");
    return 1;
}

passOne(src, inter);
rewind(inter);
passTwo(inter, out);

fclose(src); fclose(inter); fclose(out);

printf("----- MACRO NAME TABLE (MNT) ----- \n");
printf("NAME\tMDT_INDEX\tNPARAMS\n");
{ int i;
  for (i = 0; i < mnt_count; i++)
    printf("%-10s\t%d\t%d\n", MNT[i].name, MNT[i].mdt_index, MNT[i].nparams);
}

printf("\n----- MACRO DEFINITION TABLE (MDT) ----- \n");
{ int i;
  for (i = 0; i < mdt_count; i++)
    printf("%d\t%s\n", i, MDT[i]);
}

printf("\nExpansion complete. See expanded.asm for the final macro-free program.\n");
return 0;
}

```

PROCEDURE

1. Open a text editor / IDE (e.g., Turbo C++, GCC, Code::Blocks) and create a new C source file, for example `two_pass_macro_processor.c`.
2. Type the program given above and save the file.
3. In the same directory, create an input file named `input.asm` containing a sample assembly program with one or more macro definitions and calls, for example:


```

      MACRO   INCR
      &A,&B   MOVER AREG,&A   ADD AREG,&B   MOVEM AREG,&A   MEND
START 100   INCR X,Y   INCR P,Q   STOP   END

```
4. Compile the program using the command: `gcc two_pass_macro_processor.c -o macroproc`
5. Run the compiled executable: `./macroproc` (or `macroproc.exe` on Windows)
6. Observe the Macro Name Table (MNT) and Macro Definition Table (MDT) printed on the screen after Pass I processing.
7. Open the generated file `expanded.asm` and verify that every macro call (e.g., `INCR X,Y`) has been replaced by the fully expanded, parameter-substituted macro body.

8. Compare the expanded output with the expected expansion for the sample program to confirm correctness.

OUTPUT

Sample input.asm:

```
MACRO
  INCR &A,&B
  MOVER AREG,&A
  ADD AREG,&B
  MOVEM AREG,&A
MEND
START 100
INCR X,Y
INCR P,Q
STOP
END
```

Console output — Macro Name Table (MNT) and Macro Definition Table (MDT):

----- MACRO NAME TABLE (MNT) -----

NAME	MDT_INDEX	NPARAMS
INCR	0	2

----- MACRO DEFINITION TABLE (MDT) -----

```
0  MOVER AREG,#1
1  ADD AREG,#2
2  MOVEM AREG,#1
3  MEND
```

Expansion complete. See expanded.asm for the final macro-free program.

Contents of expanded.asm (final macro-free program):

```
START 100
MOVER AREG,X
ADD AREG,Y
MOVEM AREG,X
MOVER AREG,P
ADD AREG,Q
MOVEM AREG,P
STOP
END
```

Note: Each call to INCR is expanded independently, with the formal parameters &A and &B (stored as positional markers #1 and #2 in the MDT) substituted by the actual arguments supplied at each call — (X, Y) for the first call and (P, Q) for the second.

MODEL VIVA QUESTIONS

1. What is a macro?
2. Why does a macro processor need two passes?
3. What is the MNT (Macro Name Table)?
4. What is the MDT (Macro Definition Table)?
- 5 What is the ALA (Argument List Array)?

RESULT

Thus, a C program to implement a Two Pass Macro Processor — which builds the Macro Name Table (MNT) and Macro Definition Table (MDT) in Pass I, and expands every macro call in the source program into its parameter-substituted macro body in Pass II — was successfully written, executed, and verified, and the output was found to be correct.

EXP.NO.5**IMPLEMENT A SINGLE PASS MACRO PROCESSOR.****AIM**

To write, execute, and study a C program that implements a Single Pass Macro Processor which processes macro definitions and expands macro calls present in a given assembly language program in a single scan of the source program, provided every macro is defined before it is called.

LEARNING OUTCOME

- Understand the concept of macros, macro definition, macro call, and macro expansion in assembly language programming.
- Learn to construct and use the data structures required by a macro processor: Macro Name Table (MNT), Macro Definition Table (MDT), and Argument List Array (ALA), and build/use them simultaneously in one scan.
- Understand the difference between a single pass and a two pass macro processor, and the restriction a single pass processor imposes — every macro must be defined before its first call, since definitions and expansions are handled in the same scan.
- Learn how positional/keyword parameters in a macro body are replaced by index markers (e.g., #1, #2) at definition time, and substituted back with actual arguments immediately at the point of call.
- Understand how a single pass macro processor can transparently handle nested macro calls (a macro call inside another macro's body), since by the time a macro is invoked its own body has already been fully defined and stored.
- Gain the ability to translate a source program containing macro calls into an expanded, macro-free program in a single scan, and develop skills in implementing systems programming concepts using the C programming language.

DESCRIPTION

A macro is a unit of a program that is defined once, given a name, and can be repeatedly invoked (called) by that name at different points in a program. Each time the macro is called, it is replaced by the sequence of statements that form its body — this process is called macro expansion. Macros allow programmers to avoid repeating the same block of code and to parameterize that code using formal parameters that are replaced by actual arguments at each call.

A macro processor is the system software responsible for detecting macro definitions and macro calls in a source program and performing macro expansion. Unlike a Two Pass Macro Processor, which first builds the complete Macro Name Table (MNT) and Macro Definition Table (MDT) in Pass I and only then expands calls in Pass II, a Single Pass Macro Processor performs both tasks together in a single scan of the source program:

- While scanning, if a MACRO directive is encountered, the processor reads the macro prototype and body up to MEND, replaces formal parameters with positional markers (#1,

#2, ...), and immediately enters this information into the MNT and MDT — exactly as in Pass I of a two pass processor.

- If, instead, an ordinary statement is encountered whose opcode matches a name already present in the MNT, it is treated as a macro call: the processor immediately fetches the macro's body from the MDT, substitutes the actual arguments (via the Argument List Array, ALA) for the positional markers, and writes the expanded statements to the output right away.
- If the opcode does not match any macro name, the statement is simply copied to the output unchanged.

Because expansion happens immediately, in the same pass as definition processing, a single pass macro processor works correctly only if every macro is defined before it is ever called in the source program (forward references to macro definitions are not allowed). This is the key trade-off compared to a two pass macro processor. On the other hand, since a called macro's own body may itself contain calls to other, previously defined macros, nested macro calls are handled naturally — when the outer macro's body is expanded, any inner macro call appearing in it is recognized (because that inner macro was necessarily already defined earlier) and expanded in turn.

The processor uses three key data structures:

- MNT (Macro Name Table): stores the macro name, the starting index (MDT pointer) of its definition, and the number of formal parameters.
- MDT (Macro Definition Table): stores the body of every macro definition, line by line, with formal parameters replaced by positional index markers (#1, #2, ...), terminated by a MEND entry.
- ALA (Argument List Array): temporarily holds the actual arguments supplied in a macro call, indexed by parameter position, used to substitute the positional markers during expansion.

ALGORITHM

1. Start. Initialize MNT, MDT, and their index counters (MNT_index = 0, MDT_index = 0) to empty/zero.
2. Read the next line of the source program.
3. If the line is "MACRO":
 - Read the next line — the macro prototype (macro name and formal parameter list) — and record the formal parameters (e.g., &A, &B) in a temporary parameter list.
 - Enter the macro name into MNT along with the current MDT_index and the number of formal parameters.
 - Read successive lines of the macro body. For each line that is not "MEND": replace every occurrence of a formal parameter with its positional index marker (e.g., &A -> #1, &B -> #2), store the modified line in the MDT, and increment MDT_index.

- When "MEND" is encountered, store a MEND entry in the MDT, increment MDT_index, and resume normal scanning from step 2.
4. Else, if the line is not "MACRO", search the MNT for the line's opcode (the first token of the line):
- If the opcode is not found in MNT, the line is an ordinary statement — copy it as is to the output.
 - If the opcode is found in MNT, the line is a macro call: extract the actual arguments from the call and store them in the Argument List Array (ALA), positionally matched to the formal parameters; then, starting from the MDT index recorded in MNT, read successive MDT entries, substitute every positional marker (#1, #2, ...) with the corresponding actual argument from the ALA, and write each resulting statement to the output, continuing until a "MEND" entry is reached in the MDT.
 - If any statement generated during this expansion is itself a call to another macro already present in MNT, expand it in turn using the same substitution procedure (handling nested macro calls).
5. Repeat steps 2 to 4 until the end of the source program is reached.
6. Stop. The final expanded (macro-free) program is now available as output, produced in a single scan of the source.

PROGRAM

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX 50

/* ----- Macro Name Table ----- */
struct mnt {
    char name[15];
    int mdt_index;
    int nparams;
} MNT[MAX];
int mnt_count = 0;

/* ----- Macro Definition Table ----- */
char MDT[MAX][100];
int mdt_count = 0;

/* formal parameter list of the macro currently being defined */
char formal[10][10];
int nformal = 0;
```

```
FILE *out; /* single expanded output file */

int findMNT(char *name) {
    int i;
    for (i = 0; i < mnt_count; i++)
        if (strcmp(MNT[i].name, name) == 0)
            return i;
    return -1;
}

/* replace every occurrence of formal parameters in 'line' with #1,#2,... */
void substituteFormals(char *line, char *result) {
    char word[30];
    int i = 0, j, k;
    result[0] = '\0';
    while (line[i] != '\0') {
        if (line[i] == ' ' || line[i] == ',') {
            char sep[2] = { line[i], '\0' };
            strcat(result, sep);
            i++;
            continue;
        }
        j = 0;
        while (line[i] != '\0' && line[i] != ' ' && line[i] != ',')
            word[j++] = line[i++];
        word[j] = '\0';

        int found = -1;
        for (k = 0; k < nformal; k++)
            if (strcmp(word, formal[k]) == 0) { found = k; break; }
        if (found != -1) {
            char pos[6];
            sprintf(pos, "%d", found + 1);
            strcat(result, pos);
        } else {
            strcat(result, word);
        }
    }
}

/* expand a macro call given its MNT index and the raw argument string */
void expandCall(int midx, char *argstr) {
    char actual[10][20];
    int nactual = 0;
```

```

if (argstr != NULL && strlen(argstr) > 0) {
    char *tok = strtok(argstr, ",");
    while (tok != NULL) {
        strcpy(actual[nactual++], tok);
        tok = strtok(NULL, ",");
    }
}

int p = MNT[midx].mdt_index;
while (strcmp(MDT[p], "MEND") != 0) {
    char expanded[120] = "";
    char *src = MDT[p];
    while (*src) {
        if (*src == '#') {
            int num = atoi(src + 1);
            strcat(expanded, actual[num - 1]);
            src++;
            while (*src >= '0' && *src <= '9') src++;
        } else {
            char tmp[2] = { *src, '\0' };
            strcat(expanded, tmp);
            src++;
        }
    }
}

/* check if this expanded line is itself a call to another macro */
char first[30], rest[80] = "";
sscanf(expanded, "%s %[^\\n]", first, rest);
int innerIdx = findMNT(first);
if (innerIdx != -1) {
    expandCall(innerIdx, rest); /* nested macro call */
} else {
    fprintf(out, "%s\\n", expanded);
}
p++;
}
}

/* ----- SINGLE PASS: definitions processed and calls expanded together ----- */
void singlePass(FILE *src) {
    char line[100], token1[30];

    while (fgets(line, sizeof(line), src)) {
        line[strcspn(line, "\\n")] = 0;
        sscanf(line, "%s", token1);
    }
}

```

```

if (strcmp(token1, "MACRO") == 0) {
    char proto[100], name[20], plist[80];
    fgets(proto, sizeof(proto), src);
    proto[strcspn(proto, "\n")] = 0;
    sscanf(proto, "%s %[^\\n]", name, plist);

    nformal = 0;
    char *tok = strtok(plist, ",");
    while (tok != NULL) {
        strcpy(formal[nformal++], tok);
        tok = strtok(NULL, ",");
    }

    strcpy(MNT[mnt_count].name, name);
    MNT[mnt_count].mdt_index = mdt_count;
    MNT[mnt_count].nparams = nformal;
    mnt_count++;

    char body[100];
    while (fgets(body, sizeof(body), src)) {
        body[strcspn(body, "\n")] = 0;
        char btok[20];
        sscanf(body, "%s", btok);
        if (strcmp(btok, "MEND") == 0) {
            strcpy(MDT[mdt_count++], "MEND");
            break;
        }
        char subst[100];
        substituteFormals(body, subst);
        strcpy(MDT[mdt_count++], subst);
    }
} else {
    int midx = findMNT(token1);
    if (midx == -1) {
        fprintf(out, "%s\\n", line);    /* ordinary statement */
    } else {
        char rest[80] = "";
        sscanf(line, "%*s %[^\\n]", rest);
        expandCall(midx, rest);        /* immediate expansion */
    }
}
}
}

```

```

int main() {
    FILE *src = fopen("input.asm", "r");
    out = fopen("expanded.asm", "w");

    if (!src || !out) {
        printf("Error opening files.\n");
        return 1;
    }

    singlePass(src);

    fclose(src);
    fclose(out);

    printf("----- MACRO NAME TABLE (MNT) ----- \n");
    printf("NAME\t\tMDT_INDEX\tNPARAMS\n");
    { int i;
      for (i = 0; i < mnt_count; i++)
          printf("%-10s\t%d\t\t%d\n", MNT[i].name, MNT[i].mdt_index, MNT[i].nparams);
    }

    printf("\n----- MACRO DEFINITION TABLE (MDT) ----- \n");
    { int i;
      for (i = 0; i < mdt_count; i++)
          printf("%d\t%s\n", i, MDT[i]);
    }

    printf("\nExpansion complete (single scan). See expanded.asm for the final macro-free
program.\n");
    return 0;
}

```

PROCEDURE

1. Open a text editor / IDE (e.g., Turbo C++, GCC, Code::Blocks) and create a new C source file, for example `single_pass_macro_processor.c`.
2. Type the program given above and save the file.
3. In the same directory, create an input file named `input.asm` containing a sample assembly program with macro definitions placed before their calls, for example:

```

MACRO INCR
&A,&B    MOVER AREG,&A    ADD AREG,&B    MOVEM AREG,&A    MEND
START 100    INCR X,Y    INCR P,Q    STOP    END

```
4. Compile the program using the command: `gcc single_pass_macro_processor.c -o macroproc1`
5. Run the compiled executable: `./macroproc1` (or `macroproc1.exe` on Windows)

6. Observe the Macro Name Table (MNT) and Macro Definition Table (MDT) printed on the screen after the single scan completes.
7. Open the generated file expanded.asm and verify that every macro call (e.g., INCR X,Y) has already been replaced by the fully expanded, parameter-substituted macro body — produced without a second pass over the source.
8. Optionally, modify the input so that a macro call appears before its MACRO...MEND definition, run the program again, and observe that the call is not recognized/expanded — this demonstrates the single pass restriction that every macro must be defined before it is called.

OUTPUT

Sample input.asm:

```
MACRO
INCR &A,&B
MOVER AREG,&A
ADD AREG,&B
MOVEM AREG,&A
MEND
START 100
INCR X,Y
INCR P,Q
STOP
END
```

Console output — Macro Name Table (MNT) and Macro Definition Table (MDT):

```
----- MACRO NAME TABLE (MNT) -----
NAME      MDT_INDEX  NPARAMS
INCR      0          2
```

```
----- MACRO DEFINITION TABLE (MDT) -----
0  MOVER AREG,#1
1  ADD AREG,#2
2  MOVEM AREG,#1
3  MEND
```

Expansion complete (single scan). See expanded.asm for the final macro-free program.
Contents of expanded.asm (final macro-free program, produced in one scan):

```
START 100
MOVER AREG,X
ADD AREG,Y
MOVEM AREG,X
MOVER AREG,P
ADD AREG,Q
MOVEM AREG,P
```

STOP
END

Note: Unlike the two pass version, here the macro call INCR X,Y is expanded immediately, as soon as it is read — right after the MACRO...MEND definition of INCR has already been recorded in the MNT/MDT earlier in the same scan. If INCR X,Y had appeared before the MACRO...MEND definition in the source, the single pass processor would not have recognized it as a macro call, since MNT would not yet contain an entry for INCR at that point.

MODEL VIVA QUESTIONS

1. What is a macro?
2. What is macro expansion?
3. How does a single pass macro processor differ from a two pass macro processor?
4. What is the key restriction of a single pass macro processor?
5. Why can a single pass macro processor still correctly handle nested macro calls?
6. What is the MNT (Macro Name Table)?

RESULT

Thus, a C program to implement a Single Pass Macro Processor — which builds the Macro Name Table (MNT) and Macro Definition Table (MDT) and expands every macro call into its parameter-substituted body, all within a single scan of the source program — was successfully written, executed, and verified, and the output was found to be correct, subject to the requirement that every macro be defined before it is called.

EXP NO.6**IMPLEMENT AN ABSOLUTE LOADER.****AIM**

To write, execute, and study a C program that implements an Absolute Loader which reads an object program (in Header, Text, and End record format) produced by an assembler and loads its machine code into memory at the exact absolute addresses specified in the object program, and then transfers control to the program's starting execution address.

LEARNING OUTCOME

- Understand the role of a loader as the system software that places a translated (object) program into main memory and prepares it for execution.
- Learn the standard structure of an object program: Header record, one or more Text records, and an End record, and the information each field of these records carries.
- Understand what an Absolute Loader is and why it is the simplest type of loader — it performs only the 'loading' function and does not perform relocation or linking, since all addresses in the object code are already absolute.
- Learn the difference between an absolute loader and more advanced loaders such as a relocating loader or a linking loader, and identify the limitations of an absolute loader (e.g., the programmer/assembler must fix the load address in advance, and multiple independently assembled programs cannot easily be combined).
- Gain the ability to parse a hexadecimal object program, convert it into actual machine instructions/data in a simulated memory array, and transfer control to the correct execution start address.
- Develop skills in implementing systems programming concepts using the C programming language.

DESCRIPTION

A loader is a system program that takes the object program generated by an assembler (or compiler) and loads it into main memory, making it ready for execution. Depending on how much processing they perform, loaders are classified into several types: Absolute Loader, Relocating Loader (Relocating Bootstrap Loader), Direct Linking Loader, and Dynamic Linking Loader. The Absolute Loader is the simplest of these.

In an absolute loader scheme, the assembler itself assigns absolute (fixed) machine addresses to every instruction and data item while generating the object program — the loader's only job is to read this object program and place each byte of machine code exactly at the address specified in it; no address modification (relocation) or external symbol resolution (linking) is required at load time. This makes the absolute loader very simple and fast, but it has the drawback that the starting address of every program must be fixed at assembly time; if two programs are assembled to overlapping addresses, they cannot be loaded and run together, and combining independently assembled subprograms is difficult.

The object program processed by an absolute loader typically consists of three types of records:

- Header Record (H): contains the program name, the starting (load) address of the program, and the total length of the program in bytes. Format: H^programe^startaddr^length.
- Text Record (T): contains a starting address, the length (in bytes) of the object code in that record, and the actual object code itself in hexadecimal form. Format: T^startaddr^length^object-code. A program may have several text records.
- End Record (E): marks the end of the object program and specifies the address at which execution should begin (the first executable instruction of the program). Format: E^execution-start-address.

The absolute loader reads the Header record to identify the program and confirm the starting address and length, then reads each Text record and copies the object code it contains into the simulated memory array at the exact addresses given, byte by byte. When the End record is reached, the loader reads the execution start address from it and transfers control to that address (in an actual system, by setting the program counter; in this simulated implementation, by printing/indicating the transfer of control and displaying the loaded memory contents).

ALGORITHM

1. Start. Declare a memory array large enough to represent the machine's address space and initialize it (e.g., to zero).
2. Open the object program file for reading.
3. Read the first record. If it is a Header record (starts with 'H'), extract the program name, the starting address, and the length of the program; display this information.
4. Repeat step 5 for every subsequent record until an End record (starts with 'E') is found.
5. If the current record is a Text record (starts with 'T'):
 - Extract the starting address of this text record and the length (number of bytes) of object code it contains.
 - Read the object code as a sequence of two-hexadecimal-digit byte values.
 - Store each byte, in order, into the memory array at consecutive locations beginning at the record's starting address (i.e., $\text{memory}[\text{startaddr} + i] = \text{byte}[i]$).
6. When an End record (starts with 'E') is encountered, extract the execution start address given in it.
7. Display the final contents of the loaded memory region (address and corresponding byte value) to verify that the object code has been placed correctly.
8. Transfer control to the execution start address read from the End record (in this simulation, simply report that control has been transferred to that address).
9. Stop.

PROGRAM

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
#define MEMSIZE 2000 /* simulated memory (addresses 0 .. MEMSIZE-1) */

unsigned char memory[MEMSIZE];

int hexToInt(char *s, int len) {
    char buf[10];
    strncpy(buf, s, len);
    buf[len] = '\0';
    return (int) strtol(buf, NULL, 16);
}

int main() {
    FILE *fp;
    char line[100];
    char progname[10];
    int startaddr = 0, proglen = 0, execaddr = 0;

    fp = fopen("input.obj", "r");
    if (fp == NULL) {
        printf("Cannot open input.obj\n");
        return 1;
    }

    memset(memory, 0, sizeof(memory));

    while (fgets(line, sizeof(line), fp)) {
        line[strcspn(line, "\r\n")] = 0;
        if (strlen(line) == 0) continue;

        if (line[0] == 'H') {
            /* H^PROGNAME^STARTADDR^LENGTH (fields separated by ^ or spaces) */
            char *tok = strtok(line + 1, "^ \t");
            strcpy(progname, tok);
            tok = strtok(NULL, "^ \t");
            startaddr = hexToInt(tok, strlen(tok));
            tok = strtok(NULL, "^ \t");
            proglen = hexToInt(tok, strlen(tok));

            printf("Header Record : Program = %s, Start Address = %04X, Length = %04X\n",
                progname, startaddr, proglen);
        }
        else if (line[0] == 'T') {
            /* T^STARTADDR^LEN^OBJECTCODE(hex pairs, no separator) */
            char *tok = strtok(line + 1, "^ \t");
```

```

    int taddr = hexToInt(tok, strlen(tok));
    tok = strtok(NULL, "^\\t");
    int tlen = hexToInt(tok, strlen(tok));
    tok = strtok(NULL, "^\\t"); /* remaining hex object code string */

    printf("Text Record   : Start Address = %04X, Length = %02X, Code = %s\\n",
           taddr, tlen, tok);

    int i;
    for (i = 0; i < tlen; i++) {
        int byteval = hexToInt(tok + (2 * i), 2);
        memory[taddr + i] = (unsigned char) byteval;
    }
}
else if (line[0] == 'E') {
    /* E^EXECSTARTADDR */
    char *tok = strtok(line + 1, "^\\t");
    execaddr = hexToInt(tok, strlen(tok));
    printf("End Record   : Execution Start Address = %04X\\n", execaddr);
}
}
fclose(fp);

printf("\\n----- LOADED MEMORY CONTENTS -----\\n");
printf("ADDRESS\\tCONTENT (HEX)\\n");
{ int i;
  for (i = startaddr; i < startaddr + proglen; i++)
      printf("%04X\\t%02X\\n", i, memory[i]);
}

printf("\\nControl transferred to execution start address %04X.\\n", execaddr);
printf("Program \\\"%s\\\" loaded successfully by the Absolute Loader.\\n", progname);

return 0;
}

```

PROCEDURE

1. Open a text editor / IDE (e.g., Turbo C++, GCC, Code::Blocks) and create a new C source file, for example `absolute_loader.c`.
2. Type the program given above and save the file.
3. In the same directory, create an object program file named `input.obj` containing Header, Text, and End records in the format described, for example:


```

H^COPY^1000^002A
T^1000^09^141033482039101036   T^1009^0D^102A48101336000000000F
T^1016^0C^0C1041000000000C0C0C0C   E^1000
      
```

4. Compile the program using the command: `gcc absolute_loader.c -o absloader`
5. Run the compiled executable: `./absloader` (or `absloader.exe` on Windows)
6. Observe the details of the Header, Text, and End records as they are parsed, printed to the screen for verification.
7. Observe the dump of the loaded memory contents (address and hexadecimal byte value) for the program's entire address range.
8. Verify that the loaded bytes at each address exactly match the object code given in the corresponding Text record, and note the reported execution start address.

OUTPUT

Sample input.obj:

```
H^COPY^1000^002A
T^1000^09^141033482039101036
T^1009^0D^102A48101336000000000F
T^1016^0C^0C10410000000000C0C0C0C
E^1000
```

Sample console output:

```
Header Record : Program = COPY, Start Address = 1000, Length = 002A
Text Record   : Start Address = 1000, Length = 09, Code = 141033482039101036
Text Record   : Start Address = 1009, Length = 0D, Code = 102A48101336000000000F
Text Record   : Start Address = 1016, Length = 0C, Code = 0C10410000000000C0C0C0C
End Record    : Execution Start Address = 1000
```

----- LOADED MEMORY CONTENTS -----

ADDRESS CONTENT (HEX)

```
1000  14
1001  10
1002  33
1003  48
1004  20
1005  39
1006  10
1007  10
1008  36
1009  10
100A  2A
...   ... (remaining bytes continue similarly up to address 1029)
```

Control transferred to execution start address 1000.

Program "COPY" loaded successfully by the Absolute Loader.

Note: Each pair of hexadecimal digits in a Text record's object code is stored as one byte at consecutive memory addresses starting from that record's given start address; the loader performs no relocation, since every address in the object program is already absolute.

MODEL VIVA QUESTIONS

1. What is a loader?
2. What is an Absolute Loader?
3. What are the different types of loaders?
4. What functions does a general loader perform?
5. Which of these functions does an Absolute Loader actually perform?
6. What are the three types of records found in the object program used here?
memory addresses, resulting in a corrupted or incorrectly loaded program.

RESULT

Thus, a C program to implement an Absolute Loader — which reads the Header, Text, and End records of an object program, loads the machine code into simulated memory at the exact absolute addresses specified, and transfers control to the given execution start address — was successfully written, executed, and verified, and the output was found to be correct.

EXP.NO.7**IMPLEMENT A RELOCATING LOADER****AIM**

To write, execute, and study a C program that implements a Relocating Loader which reads an object program (assembled relative to address 0, in Header, Text, Modification, and End record format), loads it into memory starting at any specified actual load address by adjusting every address-sensitive field using a computed relocation factor, and then transfers control to the relocated execution start address.

LEARNING OUTCOME

- Understand the role of a loader as the system software that places a translated (object) program into main memory and prepares it for execution.
- Understand what relocation means, and why it is needed — a program assembled relative to address 0 must have every address-dependent field adjusted before it can run correctly at a different actual load address.
- Learn the concept of the relocation factor: the difference between the actual load address supplied at load time and the assembly-time (assumed) starting address of the program.
- Learn the structure and purpose of a Modification (M) record, which identifies exactly which fields of the object code are address-sensitive and therefore require the relocation factor to be added to them.
- Understand how a Relocating Loader differs from an Absolute Loader — an absolute loader loads code only at fixed addresses already present in the object program, whereas a relocating loader can load the same object program at any available memory address.
- Gain the ability to parse a hexadecimal object program with relocation information, apply the relocation factor selectively to the correct fields, and load the adjusted machine code into a simulated memory array.
- Develop skills in implementing systems programming concepts using the C programming language.

DESCRIPTION

A loader is a system program that takes the object program generated by an assembler (or compiler) and loads it into main memory, making it ready for execution. An Absolute Loader can only load a program at the fixed address for which it was assembled. A Relocating Loader removes this restriction: it allows the same assembled object program to be loaded and executed starting at any available address in memory, by adjusting (relocating) every address-sensitive field of the object code by a relocation factor.

To make this possible, the assembler generates the object program assuming the program will be loaded starting at address 0 (i.e., all addresses in the Header and Text records are relative to 0), and additionally produces Modification (M) records that tell the loader exactly which fields in the Text records contain address values that must be adjusted at load time — for example, the target address of a branch instruction, or the address of a variable — since these values depend on where the program actually ends up in memory.

At load time, the user (or operating system) specifies the actual address at which the program is to be loaded. The loader computes the relocation factor as:

- Relocation Factor (R) = Actual Load Address – Assembly-time Starting Address (usually 0)

The loader then reads the Text records and copies the object code into memory starting at (relative address + R) for each byte, exactly as an absolute loader would. After all Text records are loaded, the loader processes each Modification record: for the field specified (given by its starting address and length in the object program), it extracts the current hexadecimal value stored there, adds the relocation factor R to it, and writes the adjusted value back into that field in memory — this is what makes every branch target, address constant, and other address-sensitive value correct for the new load location. Finally, on reaching the End record, the loader adds the relocation factor to the assembly-time execution start address and transfers control to this relocated address.

The object program processed by this relocating loader consists of four types of records:

- Header Record (H): program name, assembly-time starting address (normally 000000), and total program length. Format: H^progrname^startaddr^length.
- Text Record (T): a relative starting address, the length of the object code in that record, and the object code itself in hexadecimal. Format: T^startaddr^length^object-code.
- Modification Record (M): the relative address of a field that needs adjustment, and the length of that field in half-bytes (hexadecimal digits); the loader adds the relocation factor to the value stored there. Format: M^address^halfbyte-length.
- End Record (E): the assembly-time execution start address, to which the relocation factor is added before transferring control. Format: E^execstartaddr.

ALGORITHM

1. Start. Declare a memory array (represented at the level of individual hexadecimal digits/nibbles for ease of field extraction) and initialize it to zero.
2. Open the object program file for reading.
3. Read the Header record; extract the program name, the assembly-time starting address, and the length of the program.
4. Obtain the actual load address at which the program is to be placed in memory (supplied by the user or by the operating system).
5. Compute the relocation factor: $R = \text{Actual Load Address} - \text{Assembly-time Starting Address}$.
6. Repeat step 7 for every Text record until a record other than 'T' is encountered.

7. For each Text record: extract its relative starting address and length, read the hexadecimal object code, and store each byte into the memory array at (relative address + R), byte by byte.
8. Repeat step 9 for every Modification record until the End record is reached.
9. For each Modification record: extract the relative address and half-byte length of the field to be modified; read the current hexadecimal value stored at (relative address + R) in memory for that length; add the relocation factor R to this value; and write the adjusted value back into the same field in memory.
10. When the End record is reached, extract the assembly-time execution start address and compute the relocated execution address as (execution start address + R).
11. Display the final contents of the loaded (and relocated) memory region to verify correctness.
12. Transfer control to the relocated execution address (in this simulation, report that control has been transferred to that address).
13. Stop.

PROGRAM

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MEMSIZE 4000    /* simulated memory, in NIBBLES (hex digits) */

char memNibble[MEMSIZE]; /* each element holds one hex digit '0'-'F' */

int hexToInt(char *s, int len) {
    char buf[10];
    strncpy(buf, s, len);
    buf[len] = '\0';
    return (int) strtol(buf, NULL, 16);
}

/* store a hex-digit string of length 'len' starting at nibble position 'pos' */
void storeNibbles(int pos, char *hexstr, int len) {
    int i;
    for (i = 0; i < len; i++)
        memNibble[pos + i] = hexstr[i];
}

/* read 'len' nibbles starting at nibble position 'pos' as an integer value */
int readField(int pos, int len) {
    char buf[10];
    int i;
    for (i = 0; i < len; i++) buf[i] = memNibble[pos + i];
}
```

```

    buf[len] = '\0';
    return (int) strtol(buf, NULL, 16);
}

/* write an integer value back into 'len' nibbles starting at 'pos' (zero-padded) */
void writeField(int pos, int len, int value) {
    char fmt[10], buf[10];
    sprintf(fmt, "%%0%dX", len);
    sprintf(buf, fmt, value);
    storeNibbles(pos, buf, len);
}

int main() {
    FILE *fp;
    char line[100];
    char progname[10];
    int asmStart = 0, proglen = 0, execStart = 0, loadAddr, R;

    fp = fopen("input.obj", "r");
    if (fp == NULL) {
        printf("Cannot open input.obj\n");
        return 1;
    }

    memset(memNibble, '0', sizeof(memNibble));

    printf("Enter the actual load address (hex): ");
    scanf("%x", &loadAddr);

    while (fgets(line, sizeof(line), fp)) {
        line[strcspn(line, "\r\n")] = 0;
        if (strlen(line) == 0) continue;

        if (line[0] == 'H') {
            char *tok = strtok(line + 1, "^\\t");
            strcpy(progname, tok);
            tok = strtok(NULL, "^\\t");
            asmStart = hexToInt(tok, strlen(tok));
            tok = strtok(NULL, "^\\t");
            proglen = hexToInt(tok, strlen(tok));

            R = loadAddr - asmStart; /* relocation factor */

            printf("Header Record    : Program = %s, Assembly Start = %04X, Length = %04X\n",
                progname, asmStart, proglen);
        }
    }
}

```

```

        printf("Relocation Factor R = Load Address - Assembly Start = %04X - %04X
= %04X\n",
            loadAddr, asmStart, R);
    }
    else if (line[0] == 'T') {
        char *tok = strtok(line + 1, "^\\t");
        int taddr = hexToInt(tok, strlen(tok));
        tok = strtok(NULL, "^\\t");
        int tlen = hexToInt(tok, strlen(tok));    /* length in bytes */
        tok = strtok(NULL, "^\\t");              /* hex object code */

        int destByte = taddr + R;
        printf("Text Record      : RelAddr = %04X -> LoadedAt = %04X, Length = %02X\n",
            taddr, destByte, tlen);

        storeNibbles(destByte * 2, tok, tlen * 2);
    }
    else if (line[0] == 'M') {
        char *tok = strtok(line + 1, "^\\t");
        int maddr = hexToInt(tok, strlen(tok));
        tok = strtok(NULL, "^\\t");
        int mlen = hexToInt(tok, strlen(tok));    /* length in nibbles */

        int destByte = maddr + R;
        int pos = destByte * 2;
        int oldVal = readField(pos, mlen);
        int newVal = oldVal + R;
        writeField(pos, mlen, newVal);

        printf("Modification Record : RelAddr = %04X -> LoadedAt = %04X, Len(nibbles)
= %d, "
            "Old = %0*X, New = %0*X\n",
            maddr, destByte, mlen, mlen, oldVal, mlen, newVal);
    }
    else if (line[0] == 'E') {
        char *tok = strtok(line + 1, "^\\t");
        int e = hexToInt(tok, strlen(tok));
        execStart = e + R;
        printf("End Record      : Assembly Exec Addr = %04X -> Relocated Exec Addr
= %04X\n",
            e, execStart);
    }
}
fclose(fp);

```

```

printf("\n----- LOADED (RELOCATED) MEMORY CONTENTS -----
\n");
printf("ADDRESS\tCONTENT (HEX BYTE)\n");
{ int i;
  for (i = loadAddr; i < loadAddr + proglen; i++) {
    int val = readField(i * 2, 2);
    printf("%04X\t%02X\n", i, val);
  }
}

printf("\nControl transferred to relocated execution address %04X.\n", execStart);
printf("Program \"%s\" loaded and relocated successfully by the Relocating Loader.\n",
progname);

return 0;
}

```

PROCEDURE

1. Open a text editor / IDE (e.g., Turbo C++, GCC, Code::Blocks) and create a new C source file, for example `relocating_loader.c`.
2. Type the program given above and save the file.
3. In the same directory, create an object program file named `input.obj` containing Header, Text, Modification, and End records, assembled relative to address 0, for example:
H^COPY^000000^002A T^000000^09^141033482039101036
T^000009^0D^102A48101336000000000F T^000016^0C^0C1041000000000C0C0C0C
M^000004^05 M^00000A^05 E^000000
4. Compile the program using the command: `gcc relocating_loader.c -o reloader`
5. Run the compiled executable: `./reloader` (or `reloader.exe` on Windows)
6. When prompted, enter the actual load address in hexadecimal (for example, 2000) at which the program should be placed in memory.
7. Observe the computed relocation factor, and the details of each Header, Text, Modification, and End record as they are processed.
8. Observe the dump of the loaded memory contents at the new (relocated) addresses, and verify that the fields identified by the Modification records have been correctly adjusted by the relocation factor.
9. Re-run the program with a different load address and confirm that the same object program can now be relocated and loaded correctly at that new address as well.

OUTPUT

Sample input.obj:

H^COPY^000000^002A

T^000000^09^141033482039101036

T^000009^0D^102A48101336000000000F

T^000016^0C^0C1041000000000C0C0C0C

M^000004^05

M^00000A^05

E^000000

Sample console output (entered load address: 2000):

Enter the actual load address (hex): 2000

Header Record : Program = COPY, Assembly Start = 0000, Length = 002A

Relocation Factor R = Load Address - Assembly Start = 2000 - 0000 = 2000

Text Record : RelAddr = 0000 -> LoadedAt = 2000, Length = 09

Text Record : RelAddr = 0009 -> LoadedAt = 2009, Length = 0D

Text Record : RelAddr = 0016 -> LoadedAt = 2016, Length = 0C

Modification Record : RelAddr = 0004 -> LoadedAt = 2004, Len(nibbles) = 5, Old = 10336, New = 12336

Modification Record : RelAddr = 000A -> LoadedAt = 200A, Len(nibbles) = 5, Old = 02A48, New = 22A48

End Record : Assembly Exec Addr = 0000 -> Relocated Exec Addr = 2000

----- LOADED (RELOCATED) MEMORY CONTENTS -----

ADDRESS CONTENT (HEX BYTE)

2000 14

2001 10

2002 33

2003 48

2004 20

2005 39

2006 10

2007 10

2008 36

2009 10

... .. (remaining bytes continue similarly up to address 2029)

Control transferred to relocated execution address 2000.

Program "COPY" loaded and relocated successfully by the Relocating Loader.

Note: The relocation factor R (2000 in this run) is added to every address in the Text records to decide where each byte is loaded, and is additionally added to the value stored inside each field named by a Modification record, so that address-sensitive operands remain correct at the new load location; running the loader again with a different load address (e.g., 3000) would relocate the same object program correctly to that new location instead.

MODEL VIVA QUESTIONS

1. What is a loader?
2. What is a Relocating Loader?
3. What is relocation, and why is it necessary?
4. What is the relocation factor, and how is it calculated?
5. What is a Modification (M) record, and what does it contain?
6. Why isn't the relocation factor simply added to every byte of the object code?

RESULT

Thus, a C program to implement a Relocating Loader — which reads the Header, Text, Modification, and End records of an object program assembled relative to address 0, computes the relocation factor for a given actual load address, loads the machine code accordingly, adjusts every address-sensitive field using the Modification records, and transfers control to the relocated execution address — was successfully written, executed, and verified, and the output was found to be correct.